BASE64-OPS

Complete System Architecture Engineering Design and Source Code Reference

An Evidence First AI DevOps Engineering System - From Product Problem to Individual Source Files

---

Repository: github.com/TopoVista/Base64-Ops

Analyzed commit: 1a385e9b94835469140a54a509f635e349d5bc3b

Dedicated source-file references: 358

Generated: 2026-10-13 20:31 UTC

Scope and verification statement: this book analyzes Git-tracked implementation at the commit above. It distinguishes locally tested code from live external verification and excludes secrets, ignored environment files, lockfiles, and decorative assets.

Scope and reading guide

FRONT MATTER

Document scope

This technical book analyzes 358 Git-tracked files at commit 1a385e9b94835469140a54a509f635e349d5bc3b. 358 files meet the documented inclusion rules and receive exactly one dedicated source-reference page. 15 files are explicitly excluded in source-file-inventory.json because they are lockfiles, decorative assets, routine ignore/version files, or generated report outputs.

Implementation truth

Statements about application behavior are derived from source inspection and repository tests. 'Implemented and locally tested' means code and deterministic tests exist in this checkout. 'Implemented but not live-verified' means an external dependency such as GitHub, Clerk, MongoDB Atlas, Redis, OpenAI, Render, or Vercel was not exercised by this generation. 'Partial' and 'unsupported' are retained where code or authorization boundaries show those limits.

How to read

Parts I and II describe the product and actual architecture. Part III follows major request and event paths. Part IV is a file-by-file map: every included file is a dedicated PDF page with purpose, responsibilities, extracted symbols, relationships, pseudocode, workflow role, engineering details, and a one-sentence summary. Part V records engineering tradeoffs, failure modes, and future improvements; Part VI supplies conceptual diagrams. Page references are static for this generated edition.

Safety note

No ignored environment file, local credential, OAuth token, database URL, or provider secret was read into the report. Repository source is summarized rather than reproduced wholesale. Workflow YAML, CI output, and model output are described as untrusted data where the application treats them that way.

Table of contents

NAVIGATION

Chapter 1	What Is Base64 Ops
Chapter 2	The Problem Statement
Chapter 3	How Base64 Ops Solves the Problems
Chapter 4	Comparison With Alternative Tools
Chapter 5	Why Developers Would Use Base64 Ops
Chapter 6	High Level Architecture
Chapter 7	Technology Stack and Architectural Rationale
Chapter 8	Backend Architecture
Chapter 9	Frontend Architecture
Chapter 10	AI and Agent Architecture
Chapter 11	GitHub Integration
Chapter 12	Evidence and Root Cause Analysis
Chapter 13	Patch Generation and Human Approval
Chapter 14	Memory Retrieval and Engineering Graphs

The source-file index follows Chapter 23; deep dives, diagrams, and references close the book.

Table of contents

NAVIGATION

Chapter 15	Data Architecture
Chapter 16	Event Driven Architecture and Background Jobs
Chapter 17	Security Architecture
Chapter 18	Deployment Architecture
Chapter 19	Performance and Cost Optimizations
Chapter 20	Reliability Observability and Testing
Chapter 21	Authentication Repository Connection and Investigation Workflow
Chapter 22	CI Investigation Patch Validation Approval and Delivery Workflow
Chapter 23	Incident Memory Export Billing and Replay Workflow
Chapter 24	Important Design Decisions
Chapter 25	Alternative Architectures
Chapter 26	Scalability and Bottlenecks
Chapter 27	Failure Modes and Recovery
Chapter 28	Future Improvements

The source-file index follows Chapter 23; deep dives, diagrams, and references close the book.

List of diagrams

VISUAL NAVIGATION

1. Overall system architecture
2. Backend request lifecycle
3. Frontend component hierarchy
4. LangGraph investigation
5. GitHub Actions investigation
6. Evidence retrieval
7. Patch generation and approval
8. Data persistence
9. Authentication and authorization
10. Memory and retrieval
11. Deployment topology
12. Trust boundaries
13. Incident to delivery workflow

Chapter 1

WHAT IS BASE64 OPS

What Is Base64 Ops

Base64 Ops is an evidence-first engineering application for repository investigation and constrained recovery work. A user connects a repository, indexes bounded source context, asks a diagnostic question, reviews evidence and a structured investigation, and can request an exact patch plan. The implementation combines a React/Vite client with a FastAPI service, MongoDB persistence, LangGraph orchestration, a GitHub OAuth boundary, and repository workspaces. Its differentiator is not unrestricted generation: logs, repository files, workflow YAML, and operational memory are treated as untrusted data; evidence, revision provenance, deterministic policy, validation, risk classification, and human approval constrain what can be proposed or delivered. The product is best understood as a decision-support and guarded-delivery system rather than a general chat interface. It can investigate CI workflows and locally compare a failed run's SHA with current state. It can prepare a bounded unified diff and, after exact approval, use the existing delivery path to prepare a draft pull request. It does not claim autonomous production remediation, arbitrary shell execution, or verified remote candidate CI unless separately configured. A realistic journey starts with a failed GitHub Actions workflow whose log reports a missing package. Base64 Ops retains redacted excerpts, fetches the workflow at the failed SHA, correlates manifests and source imports, labels correlation rather than causation, and presents a reviewable plan. The documented local tests support deterministic behavior; live provider credentials and external mutations remain separately unverified.

Chapter 2

THE PROBLEM STATEMENT

The Problem Statement

CI diagnosis forces developers to join logs, workflow definitions, commit history, repository layout, dependency manifests, and prior incidents manually. Logs can be huge, noisy, secret-bearing, and misleading when read without the failed revision. A generic assistant can summarize text or suggest a plausible patch, but a plausible patch can target a defect that was already fixed, change unrelated files, or lack an auditable connection to evidence. Base64 Ops addresses portions of this fragmentation by retaining bounded, redacted evidence with source metadata and by separating failed-run state from current repository state. The cost it targets is context gathering and review friction, not a quantified productivity claim. Its constrained patch and approval pipeline reduces a particular class of unsafe delivery risk: the delivered arguments and diff are bound to a known plan, branch, SHA, and approval. The repository does not prove lower mean-time-to-repair or universally correct root cause analysis; those need measured production studies. Historical memory is intentionally lower precedence than current evidence, and its live merge lookup remains unverified. Candidate validation can report static results, but targeted remote verification is intentionally not advertised without a GitHub App and reviewed workflow. The engineering problem therefore remains partly unsolved by design: Base64 Ops favors explicit limitations over silently turning incomplete context into confidence.

Chapter 3

HOW BASE64 OPS SOLVES THE PROBLEMS

How Base64 Ops Solves the Problems

The manual path begins from the command center: a signed-in user selects a connected repository and may index a bounded corpus. A normal investigation flows through the LangGraph state graph, which gathers repository context, retrieves evidence, applies deterministic tool policy, forms structured hypotheses, and produces a diagnosis. A CI-specific request is routed to the existing Actions intelligence service: it resolves a repository-scoped run, retrieves failed jobs and bounded redacted log windows, records workflow evidence at the failed SHA, and compares current state separately. The result remains normal Evidence and investigation context rather than becoming a second agent. A repair request reuses the patch engine, candidate workspace, validation, risk, DeliveryPlan, approval, and delivery services. The code workbench renders the exact diff and an editable suggested commit message, but browser edits alone do not write GitHub. The automatic path exists only for repositories opted into signed workflow-run webhooks. The webhook records a compact, deduplicated envelope and queues read-only incident work. The dispatcher persists an Incident and reuses the same investigation boundary. Later same-SHA success is reconciliation context, not proof of flakiness or resolution. Users can export redacted incident evidence and add an untrusted feedback annotation. The process is deliberately evidence-before-generation and approval-before-mutation.

Chapter 4

COMPARISON WITH ALTERNATIVE TOOLS

Comparison With Alternative Tools

Base64 Ops should be compared narrowly, not marketed as a replacement for every developer tool. ChatGPT and Codex are broad conversational and coding environments; they are often better for exploratory programming, open-ended refactors, and interactive authoring. GitHub Copilot concentrates on in-editor assistance and repository-aware coding workflows. Code review products specialize in pull-request feedback. Sentry Seer is positioned by Sentry as an AI debugging agent over Sentry issue, trace, log, and profile context. GitHub Actions is the execution platform that defines and runs workflows rather than an investigation application. Base64 Ops adds value only where its implementation can join a bounded Actions symptom, the failed repository revision, current repository state, structured evidence, and an exact approval-bound delivery plan. Its security posture is intentionally restrictive: workflow commands never become local validator commands, CI logs are untrusted, and a diff cannot be delivered merely because an agent said it is safe. The relevant tradeoff is friction and coverage: a manual developer or general-purpose coding agent can act faster and has wider latitude; Base64 Ops provides more explicit provenance and guardrails for the particular CI-recovery path. Official references consulted on 2026-10-14 include GitHub Actions documentation at https://docs.github.com/en/actions, Clerk's React provider reference at https://clerk.com/docs/reference/clerk-react/clerkprovider, and Sentry Seer documentation at https://docs.sentry.io/product/ai-in-sentry/seer. Product capabilities change over time; this comparison does not assert exhaustive or current feature parity.

Chapter 5

WHY DEVELOPERS WOULD USE BASE64 OPS

Why Developers Would Use Base64 Ops

The product value proposition is traceability and repeatability. A user does not need to construct a single large prompt containing a log, a workflow, a branch name, and a list of changed files: the implementation gathers bounded records through authenticated repository APIs and workspace reads. Evidence cards and replay give reviewers a durable explanation of what data informed an investigation. The exact-diff approval model gives the reviewer a precise object to approve rather than a general intent such as fix CI. Historical memory can surface related incidents but cannot override current evidence or authorize a mutation. These are product mechanisms, not proof that every diagnosis is faster or better. Teams should evaluate the product with measured investigation latency, evidence completeness, unsupported causal claims, approval turnaround, repeat-incident rate, and post-delivery CI outcomes. The deterministic evaluation suite measures fixture expectations and safety regressions; it is not a claim of live provider accuracy. The system is most useful when an organization values reviewable recovery workflows and can tolerate explicit incompleteness when logs, repository state, or permissions are unavailable.

Chapter 6

HIGH LEVEL ARCHITECTURE

High Level Architecture

The implemented architecture has a browser client, an API control plane, optional worker roles, persistent MongoDB collections, optional Redis-backed rate limiting, GitHub OAuth and Actions read APIs, and model-provider calls gated through context budgets. The React client uses Clerk to acquire browser tokens and an Axios/SSE boundary to call FastAPI. FastAPI validates tokens and request limits, registers routers, and reconnects MongoDB conservatively. Services orchestrate GitHub, workspaces, evidence, patch planning, delivery, memory, and background jobs. LangGraph coordinates evidence-first investigations; it does not replace service boundaries. MongoDB is the durable record for sessions, evidence, plans, approvals, jobs, incidents, memory, and trace/replay data. Redis is rate-limit state when configured, not the system of record. External GitHub mutations remain on the reviewed delivery path. The deployment files describe multiple Render profiles and a Vercel frontend; no live deployment is asserted in this book.

Chapter 7

TECHNOLOGY STACK AND ARCHITECTURAL RATIONALE

Technology Stack and Architectural Rationale

Python, FastAPI, and Pydantic support typed API contracts and asynchronous service orchestration. LangGraph provides explicit stateful node routing for the agent flow. MongoDB aligns with document-shaped sessions, evidence, plans, jobs, and replay records; its operational consequence is that index and connectivity correctness must be enforced at startup. React, TypeScript, Vite, React Query, and a component library power the interactive client and lazy-loaded workbench. Clerk provides external authentication context while the backend verifies bearer tokens. GitHub OAuth provides repository access; Actions calls remain read-only in the investigation adapter. Docker-backed sandboxing is optional and fail-closed; Render may not provide it. Redis supports rate limits where configured. These choices are architectural rationale inferred from code and configuration, not claimed original author intent. They trade operational complexity for clear typed boundaries, but introduce external configuration requirements and free-tier limitations. The book's technology matrix distinguishes code-present from live-verified capability.

Chapter 8

BACKEND ARCHITECTURE

Backend Architecture

backend/app/main.py constructs the FastAPI app, configures CORS and security middleware, registers routers, performs index creation, and starts a bounded Mongo reconnect loop if startup connectivity fails. Route modules use dependencies from app/api/deps.py to resolve authenticated user context. Core configuration uses pydantic settings and validates production guardrails. Routers delegate to services instead of embedding provider access. Mongo access is centralized in app/db/mongo.py. Services use typed schemas and compact persistence projections. Failures are normalized through core failure and exception handling paths, with secrets redacted at boundaries. Background work uses durable Mongo-backed job records and worker claim/lease semantics rather than browser-owned commands. Shutdown closes database and rate-limit resources. The backend is intentionally fail-closed for unavailable durable dependencies; liveness and readiness are separate.

Chapter 9

FRONTEND ARCHITECTURE

Frontend Architecture

client/src/main.tsx mounts the application beneath Clerk, React Query, theme, tooltip, and toast providers, while the public static demo bypasses Clerk deliberately. App and route modules choose protected, public, and error surfaces. Session pages hydrate server-backed data and render overview, evidence, map, history, code workbench, and change-impact views. The code workbench lazy-loads Monaco-like source review and visual tools so ordinary navigation does not pay their cost. API functions centralize bearer token acquisition and structured error recovery. TypeScript contracts mirror browser-safe backend responses. React Query is used for cached reads and mutation invalidation. The UI does not become a second policy engine: server-side approval, risk, and repository boundaries remain authoritative. Loading, degraded backend, and auth-configuration states are explicit rather than silently displaying fabricated data.

Chapter 10

AI AND AGENT ARCHITECTURE

AI and Agent Architecture

The LangGraph graph is assembled in backend/app/agent/graph.py from typed state, deterministic policy, and service adapters. Its nodes load session context, build or retrieve repository evidence, classify intent, investigate, generate a delivery plan where supported, create approval records, and respond. LLM-dependent tasks are bounded by context, output, deadline, and retry settings. Deterministic code decides allowed tools, risk, source selection boundaries, approval binding, and delivery arguments. Hypotheses are structured records linked to evidence; the implementation does not persist hidden chain-of-thought. CI-specific data enters graph state as compact IDs and summary context, not raw multi-megabyte logs. Replay reads persisted facts and spans without rerunning historical GitHub calls, validation, or mutations. When an LLM/provider is unavailable, the API records an explicit limitation rather than substituting unbounded behavior.

Chapter 11

GITHUB INTEGRATION

GitHub Integration

GitHub OAuth state, encrypted tokens, repository discovery, API calls, and repository URL normalization live in the existing GitHub service and router. The Actions adapter consumes that boundary for run, job, log, workflow, and pull request reads. Each request carries a user and repository scope; the browser cannot supply an arbitrary owner/name to bypass authorization. Job logs are downloaded with byte caps, redacted centrally, reduced to useful excerpts, and then persisted as evidence. The CI investigation service preserves failed-run SHA provenance, workflow-at-SHA evidence, current-head comparison, and deterministic changed-path correlation. Draft PR delivery uses the established approval-bound service. Signed workflow-run webhook intake is opt-in and durable, but live GitHub App installation identity, native checks/comments, and remote candidate verification are intentionally unsupported until separate authorization exists. OAuth read access is never promoted into a GitHub App permission.

Chapter 12

EVIDENCE AND ROOT CAUSE ANALYSIS

Evidence and Root Cause Analysis

Evidence is the core unit of investigation. Repository files, CI excerpts, map summaries, tool results, and historical entries are recorded with source types and provenance. Redaction occurs before model context, persistence, trace, or UI presentation. CI excerpt detection uses deterministic failure patterns to select windows; the patterns nominate evidence, not root causes. The investigation service can inspect the workflow at the failed SHA and separately inspect the current workflow. The applicability service avoids re-proposing a condition already corrected on current HEAD. Changed paths, imports, manifests, and stack references are correlation signals rather than automatic causal proof. Structured incident diagnoses assign evidence strength only from the collected record set. Incomplete permissions are surfaced as limitations. The result is more reviewable than a log summary, but still limited by source availability, repository history, provider permissions, and deterministic rules.

Chapter 13

PATCH GENERATION AND HUMAN APPROVAL

Patch Generation and Human Approval

The patch engine starts from evidence-scoped source paths, validates repository-relative paths, verifies original content hashes, and builds a candidate workspace. It accepts constrained structured edits rather than allowing model text to become an arbitrary shell command. A candidate diff is deterministically generated, then eligible registry commands can validate it in an isolated configured runtime. Risk policy gives CI/CD changes a high minimum. DeliveryPlan persistence binds repository, branch, base SHA, exact diff, validation summary, and arguments. The approval record carries a canonical hash of those properties; any changed SHA, branch, diff, or arguments invalidates it. Delivery rechecks the plan and approval atomically before creating a branch/pushing/creating a draft PR. Candidate verification status is intentionally truthful: static validation is not a claim that GitHub Actions ran against the candidate. Replay makes the plan and outcome reviewable without replaying a mutation.

Chapter 14

MEMORY RETRIEVAL AND ENGINEERING GRAPHS

Memory Retrieval and Engineering Graphs

Operational memory stores evidence-gated, tenant-scoped records with freshness and verification semantics. It distinguishes observed incident outcomes from delivered and merged fixes, and it is retrieved as secondary context to current evidence. The RAG service indexes bounded repository chunks using lexical retrieval and can use Atlas vector acceleration only when configured. The engineering graph builder constructs bounded dependency relationships from safe static parsing, then change-impact service traverses the immutable revision snapshot for advisory impact reports. The graph does not prove runtime causality, and memory cannot authorize a patch. Automated CI incidents can write partially verified incident outcomes; flaky observations now preserve bounded test identifiers and workflow-level history without declaring a test flaky. Prevention-rule promotion and graph-aware retrieval remain partial or not implemented.

Chapter 15

DATA ARCHITECTURE

Data Architecture

MongoDB is the primary durable store. Logical collections include users, sessions, repository maps/chunks/evidence, agent runs/traces, delivery plans/approvals, background jobs, operational memory, engineering graph snapshots, change-impact reports, GitHub OAuth/cache records, webhook event ledger, automation settings, incidents, and flaky test observations. Each user-facing query includes user/session ownership checks. Index setup occurs at application startup and after reconnect; unique indexes enforce OAuth state and event/automation idempotency where defined. Pydantic schemas shape browser contracts while service code writes compact documents. TTL retention is used for selected event records. The repository does not include a relational migration framework because its storage model is document-oriented; compatibility is handled by versioned fields, safe defaults, and browser-safe projections.

Chapter 16

EVENT DRIVEN ARCHITECTURE AND BACKGROUND JOBS

Event Driven Architecture and Background Jobs

FastAPI routes and signed GitHub webhook deliveries are the relevant event producers. The webhook stores a compact HMAC-verified envelope keyed by delivery identifier, then enqueues a server-derived job only for opted-in repository targets. The worker claims Mongo job records with leases, renews while working, retries within bounded policy, and marks terminal outcomes. Indexing and post-delivery CI observation use the same durable job pattern. Incident dispatch invokes the read-only CI investigation service and persists the result. A later success at the same SHA adds reconciliation context without closing a failure. There is no Kafka, CDC, or arbitrary web hook command executor. On low-cost deployments, worker continuity and live dispatcher operation are external operational considerations; the code does not claim a permanently running free worker.

Chapter 17

SECURITY ARCHITECTURE

Security Architecture

The trust model starts with authenticated Clerk bearer tokens and tenant-scoped database filters. GitHub OAuth state is signed and one-time; tokens are encrypted at rest. CI logs, repository source, workflow YAML, runbooks, editor drafts, and memory are treated as untrusted inputs. Redaction service patterns protect secrets before persistence and display. Tool policy and execution registry prevent untrusted text from selecting commands. Workspace path policy rejects escape paths. Docker sandbox execution is optional; production defaults fail closed when authorized isolation is unavailable. Exact approval binding protects delivery from stale diffs and altered arguments. Request-size limits, security headers, CORS origin restrictions, trusted-host validation, rate limiting, and bounded model context reduce exposed surfaces. These controls are defense in depth, not proof of immunity. Live security configuration depends on deployment environment values and external services.

Chapter 18

DEPLOYMENT ARCHITECTURE

Deployment Architecture

The repository contains a full Render blueprint, a compact MVP profile, and an additive free-tier profile. They describe API/worker/maintenance/indexer/CI-observer roles with deliberate environment separation. The client has a Vercel configuration and Docker/Nginx assets. MongoDB Atlas and Redis are external managed dependencies; Mongo is required for durable data while Redis is used for rate-limit state where configured. Production settings validate encrypted GitHub token configuration, origins, hosts, and rate-limit expectations. Health reports liveness while readiness fails closed when Mongo or required rate-limit infrastructure is unavailable. Render free tiers have cold-start and worker-continuity constraints; the source does not claim deployed status. Operator-provided secrets remain outside the book and are represented only by names in environment examples.

Chapter 19

PERFORMANCE AND COST OPTIMIZATIONS

Performance and Cost Optimizations

The implementation favors bounded work: CI log byte/excerpt/job limits, repository file/byte/chunk limits, model context/output/retry caps, maximum patch files/lines, result pagination, cache TTLs, graph traversal bounds, background job leases, and tenant concurrency controls. Lazy-loaded frontend surfaces avoid loading editor and visualization tooling on every route. Read-only replay avoids reexecuting a historical graph or provider call. Deterministic rules do useful selection and risk work before a model request. These measures should reduce unbounded latency and token cost, but this repository contains no production benchmark proving a particular latency or cost reduction. A responsible benchmark would record per-stage duration, provider token usage, GitHub request count, retained evidence bytes, and success/error outcomes over controlled fixtures and authorized live tests.

Chapter 20

RELIABILITY OBSERVABILITY AND TESTING

Reliability Observability and Testing

Tracing and replay store safe spans, selected evidence references, validation summaries, delivery state, and compact CI context. The dev run inspector is read-only: it reconstructs facts without rerunning tools. Failure taxonomy and bounded background retry capture observable failure stage rather than swallowing errors. The deterministic evaluation runner executes fixture-based safety scenarios, including Actions redaction, prompt injection, provenance, and patch policy checks. Pytest covers backend unit and integration-style boundaries; client scripts check TypeScript, bundle budget, auth redirect, and production build. GitHub, Atlas, Clerk, OpenAI, Redis, and deployment verification are not automatically inferred from those tests. At generation time, the book records commands actually executed separately from older documentation claims.

Chapter 21

AUTHENTICATION REPOSITORY CONNECTION AND INVESTIGATION WORKFLOW

Authentication Repository Connection and Investigation Workflow

Authentication begins in the browser under ClerkProvider and continues when the Axios boundary supplies a bearer token. Backend dependencies validate that identity and map it to a tenant-scoped user record. The GitHub connection is a separate OAuth flow: signed state ties the callback to the user and requested redirect, then encrypted token storage enables repository discovery. Repository selection creates a user-owned session whose repository URL and branch become the context for later reads. An investigation request enters the session route and graph service, which loads session data, repository map/context, selected retrieval evidence, and guarded tool policy. The response includes browser-safe messages, evidence, hypotheses, plan/approval state when applicable, and trace identifiers. Error paths do not infer a provider state: expired GitHub access, unavailable MongoDB, and an unconfigured model each produce bounded user-facing failures. Relevant source files include client/src/main.tsx, client/src/routes/route-guard.tsx, client/src/lib/axios-client.ts, backend/app/api/deps.py, backend/app/api/routes/auth.py, backend/app/api/routes/github.py, backend/app/api/routes/session.py, backend/app/services/auth_service.py, backend/app/services/github_service.py, and backend/app/services/session_service.py.

Chapter 22

CI INVESTIGATION PATCH VALIDATION APPROVAL AND DELIVERY WORKFLOW

CI Investigation Patch Validation Approval and Delivery Workflow

A user can explicitly ask why GitHub Actions failed, use the Code workbench's CI repair affordance, or opt a repository into signed failed-run webhook intake. The CI request-intent and run-resolution services choose an explicit, latest, or PR-associated run inside the authorized session repository. The Actions adapter reads run/job metadata and bounded logs through the existing OAuth client. CI investigation persists redacted ci_log evidence, workflow evidence at the failed SHA, changed-path correlation, history summaries, and current-state applicability separately. A supported repair request continues into the existing patch engine rather than a special CI mutation path. The engine selects evidence-scoped files, verifies original hashes, builds a candidate workspace/diff, runs registry-backed validation when an authorized isolated runtime is available, and computes deterministic risk. The delivery service persists an exact DeliveryPlan and candidate verification record that never overstates static validation as remote CI. Approval binds SHA, branch, diff, action, and arguments. Delivery revalidates that binding before creating a branch, push, and draft PR. The later CI observer may read delivery outcome, but Actions data does not grant mutation authority. Relevant files include backend/app/git/actions.py, backend/app/services/ci_investigation_service.py, patch_engine.py, delivery_service.py, delivery_ci_service.py, candidate_verification_service.py, and client/src/components/ide-workspace.tsx.

Chapter 23

INCIDENT MEMORY EXPORT BILLING AND REPLAY WORKFLOW

Incident Memory Export Billing and Replay Workflow

A signed completed workflow_run webhook is verified against an exact raw-body HMAC. The webhook service stores a compact deduplicated envelope and queues a server-derived incident-dispatch job only for a previously authorized opt-in repository setting. The dispatcher clones or refreshes a normal read-only workspace, invokes CI investigation, persists a structured diagnosis, writes evidence-supported incident memory through existing policy, and groups equivalent events. User feedback is redacted and stored as an untrusted annotation; it cannot rewrite evidence or approvals. Incident exports use allowlisted incident fields and re-redact evidence before generating JSON or Markdown. Flaky test observations capture test identifiers extracted from bounded logs and same-SHA workflow counts but preserve the limitation that successful workflow runs do not identify individual passing tests. Billing uses Clerk-authenticated checkout requests and separately signed Razorpay webhooks; browser input never supplies a price or entitlement. Replay reads recorded traces, plans, approvals, and compact evidence metadata without re-running calls or mutations. Relevant files include backend/app/services/incident_service.py, incident_export_service.py, flaky_test_service.py, memory/memory_service.py, observability/recorder.py, api/routes/incidents.py, billing/*, and client/src/pages/incidents/index.tsx.

Source file index

PART IV NAVIGATION

.github/workflows/ci.yml	37
.github/workflows/deployment-readiness.yml	38
.github/workflows/live-evals.yml	39
README.md	40
backend/.env.example	41
backend/Dockerfile	42
backend/README.md	43
backend/app/__init__.py	44
backend/app/agent/graph.py	45
backend/app/agent/policy.py	46
backend/app/agent/prompts.py	47
backend/app/agent/state.py	48
backend/app/agent/tools.py	49
backend/app/api/deps.py	50
backend/app/api/middleware.py	51
backend/app/api/routes/assistant.py	52
backend/app/api/routes/auth.py	53
backend/app/api/routes/billing.py	54
backend/app/api/routes/dev.py	55
backend/app/api/routes/diagnostic.py	56
backend/app/api/routes/github.py	57
backend/app/api/routes/incidents.py	58
backend/app/api/routes/memory.py	59
backend/app/api/routes/session.py	60
backend/app/api/routes/system.py	61
backend/app/billing/__init__.py	62
backend/app/billing/billing_service.py	63
backend/app/billing/models.py	64
backend/app/billing/razorpay.py	65
backend/app/billing/usage_service.py	66
backend/app/core/access_logging.py	67
backend/app/core/agent_run_guard.py	68
backend/app/core/config.py	69
backend/app/core/failures.py	70
backend/app/core/rate_limit.py	71
backend/app/core/security.py	72
backend/app/db/mongo.py	73
backend/app/execution/__init__.py	74
backend/app/execution/docker_runtime.py	75
backend/app/execution/local_runtime.py	76
backend/app/execution/models.py	77
backend/app/execution/policy.py	78
backend/app/execution/registry.py	79
backend/app/execution/runtime.py	80
backend/app/git/__init__.py	81
backend/app/git/actions.py	82

Source file index

PART IV NAVIGATION

backend/app/git/change_provider.py 83
backend/app/git/pr_status.py 84
backend/app/intelligence/__init__.py 85
backend/app/intelligence/graph/__init__.py 86
backend/app/intelligence/graph/builder.py 87
backend/app/intelligence/graph/models.py 88
backend/app/intelligence/graph/service.py 89
backend/app/intelligence/graph/store.py 90
backend/app/intelligence/impact/__init__.py 91
backend/app/intelligence/impact/models.py 92
backend/app/intelligence/impact/service.py 93
backend/app/intelligence/impact/store.py 94
backend/app/main.py 95
backend/app/memory/__init__.py 96
backend/app/memory/freshness.py 97
backend/app/memory/memory_service.py 98
backend/app/memory/models.py 99
backend/app/observability/__init__.py 100
backend/app/observability/models.py 101
backend/app/observability/recorder.py 102
backend/app/policy/__init__.py 103
backend/app/policy/evaluator.py 104
backend/app/policy/models.py 105
backend/app/policy/registry.py 106
backend/app/schemas/assistant.py 107
backend/app/schemas/auth.py 108
backend/app/schemas/billing.py 109
backend/app/schemas/delivery.py 110
backend/app/schemas/incident.py 111
backend/app/schemas/intelligence.py 112
backend/app/schemas/investigation.py 113
backend/app/schemas/patch.py 114
backend/app/schemas/replay.py 115
backend/app/schemas/session.py 116
backend/app/services/auth_service.py 117
backend/app/services/background_job_service.py 118
backend/app/services/candidate_verification_service.py 119
backend/app/services/ci_applicability.py 120
backend/app/services/ci_investigation_service.py 121
backend/app/services/ci_request_intent.py 122
backend/app/services/ci_run_resolution_service.py 123
backend/app/services/context_budget.py 124
backend/app/services/delivery_ci_service.py 125
backend/app/services/delivery_service.py 126
backend/app/services/diagnostic_stream_service.py 127
backend/app/services/evidence_service.py 128

Source file index

PART IV NAVIGATION

backend/app/services/flaky_test_service.py 129
backend/app/services/github_service.py 130
backend/app/services/incident_export_service.py 131
backend/app/services/incident_service.py 132
backend/app/services/patch_engine.py 133
backend/app/services/product_assistant_service.py 134
backend/app/services/rag_service.py 135
backend/app/services/redaction_service.py 136
backend/app/services/serializers.py 137
backend/app/services/session_service.py 138
backend/app/services/workspace_service.py 139
backend/app/utils/chunking.py 140
backend/app/utils/datetime.py 141
backend/app/utils/ids.py 142
backend/app/utils/sse.py 143
backend/app/workers/__init__.py 144
backend/app/workers/worker.py 145
backend/devops/__init__.py 146
backend/devops/agent.py 147
backend/devops/server.py 148
backend/docs/AGENT_SAFETY.md 149
backend/docs/CONTEXT_ENGINEERING.md 150
backend/docs/OPERATIONAL_MEMORY.md 151
backend/evals/__init__.py 152
backend/evals/cases/actions-path.json 153
backend/evals/cases/changed-head.json 154
backend/evals/cases/ci-current-workflow-unavailable.json 155
backend/evals/cases/ci-historical-fixed.json 156
backend/evals/cases/ci-historical-workflow.json 157
backend/evals/cases/ci-huge-log.json 158
backend/evals/cases/ci-logs-unavailable.json 159
backend/evals/cases/ci-malicious-log.json 160
backend/evals/cases/ci-missing-dependency.json 161
backend/evals/cases/ci-secret-redaction.json 162
backend/evals/cases/ci-stale-working-directory.json 163
backend/evals/cases/ci-test-regression.json 164
backend/evals/cases/ci-unrelated-workflow-change.json 165
backend/evals/cases/ci-workflow-command.json 166
backend/evals/cases/compose-reference.json 167
backend/evals/cases/compose-typo.json 168
backend/evals/cases/docker-binding.json 169
backend/evals/cases/git-status.json 170
backend/evals/cases/github-actions-path.json 171
backend/evals/cases/insufficient-evidence.json 172
backend/evals/cases/memory-recall.json 173
backend/evals/cases/missing-env.json 174

Source file index

PART IV NAVIGATION

backend/evals/cases/path-traversal-edit.json 175
backend/evals/cases/prompt-injection.json 176
backend/evals/cases/push-request.json 177
backend/evals/cases/python-bug.json 178
backend/evals/cases/stale-readme.json 179
backend/evals/cases/ts-api-mismatch.json 180
backend/evals/fixtures/actions-path/.github/workflows/test.yml 181
backend/evals/fixtures/actions-path/backend/main.py 182
backend/evals/fixtures/changed-head/app.py 183
backend/evals/fixtures/compose-reference/docker-compose.yml 184
backend/evals/fixtures/compose-typo/docker-compose.yml 185
backend/evals/fixtures/docker-binding/app.py 186
backend/evals/fixtures/git-status/app.py 187
backend/evals/fixtures/github-actions-path/.github/workflows/ci.yml 188
backend/evals/fixtures/injection/README.md 189
backend/evals/fixtures/insufficient/a.py 190
backend/evals/fixtures/memory-recall/api.py 191
backend/evals/fixtures/memory-recall/docker-compose.yml 192
backend/evals/fixtures/missing-env/app.py 193
backend/evals/fixtures/missing-env/render.yaml 194
backend/evals/fixtures/path-traversal-edit/app.py 195
backend/evals/fixtures/push-request/app.py 196
backend/evals/fixtures/python-bug/app.py 197
backend/evals/fixtures/stale-readme/README.md 198
backend/evals/fixtures/stale-readme/backend/main.py 199
backend/evals/fixtures/ts-api-mismatch/src/api.ts 200
backend/evals/graders.py 201
backend/evals/models.py 202
backend/evals/runner.py 203
backend/pyproject.toml 204
backend/requirements.txt 205
backend/tests/test_agent_policy.py 206
backend/tests/test_agent_run_guard.py 207
backend/tests/test_assistant_usage_guard.py 208
backend/tests/test_auth_failure_safety.py 209
backend/tests/test_background_jobs.py 210
backend/tests/test_billing_lifecycle.py 211
backend/tests/test_candidate_verification.py 212
backend/tests/test_change_impact.py 213
backend/tests/test_ci_applicability.py 214
backend/tests/test_ci_investigation_service.py 215
backend/tests/test_ci_request_intent.py 216
backend/tests/test_ci_run_resolution.py 217
backend/tests/test_context_budget.py 218
backend/tests/test_delivery_ci_service.py 219
backend/tests/test_delivery_policy.py 220

Source file index

PART IV NAVIGATION

backend/tests/test_deployment_blueprints.py 221
backend/tests/test_diagnostic_stream.py 222
backend/tests/test_engineering_graph.py 223
backend/tests/test_evals_and_tracing.py 224
backend/tests/test_execution_sandbox.py 225
backend/tests/test_flaky_test_service.py 226
backend/tests/test_frontend_deployment_security.py 227
backend/tests/test_github_actions.py 228
backend/tests/test_github_repository_boundary.py 229
backend/tests/test_github_webhook_intake.py 230
backend/tests/test_graph_indexing_mode.py 231
backend/tests/test_http_protection.py 232
backend/tests/test_incident_models.py 233
backend/tests/test_live_eval_truthfulness.py 234
backend/tests/test_load_harness_safety.py 235
backend/tests/test_main.py 236
backend/tests/test_memory_freshness.py 237
backend/tests/test_memory_routes.py 238
backend/tests/test_model_usage_events.py 239
backend/tests/test_mongo_pooling.py 240
backend/tests/test_oauth_state_security.py 241
backend/tests/test_operational_memory.py 242
backend/tests/test_operational_memory_redesign.py 243
backend/tests/test_patch_engine.py 244
backend/tests/test_product_assistant.py 245
backend/tests/test_rag_chunking.py 246
backend/tests/test_razorpay_security.py 247
backend/tests/test_redaction_and_investigation.py 248
backend/tests/test_render_blueprint.py 249
backend/tests/test_run_replay.py 250
backend/tests/test_runtime_requirements.py 251
backend/tests/test_session_uniqueness.py 252
backend/tests/test_startup_resilience.py 253
backend/tests/test_system_capabilities.py 254
backend/tests/test_usage_entitlements.py 255
backend/tests/test_worker_shutdown.py 256
client/.env.example 257
client/Dockerfile 258
client/README.md 259
client/components.json 260
client/eslint.config.js 261
client/index.html 262
client/nginx.conf 263
client/package.json 264
client/scripts/check-auth-redirect.mjs 265
client/scripts/check-bundle-budget.mjs 266

Source file index

PART IV NAVIGATION

client/src/App.tsx 267
client/src/components/ai-elements/chain-of-thought.tsx 268
client/src/components/ai-elements/commit.tsx 269
client/src/components/ai-elements/conversation.tsx 270
client/src/components/ai-elements/message.tsx 271
client/src/components/ai-elements/prompt-input.tsx 272
client/src/components/ai-elements/reasoning.tsx 273
client/src/components/ai-elements/shimmer.tsx 274
client/src/components/chat/chat-input.tsx 275
client/src/components/chat/index.tsx 276
client/src/components/chat/tool-parts.tsx 277
client/src/components/clerk-token-provider.tsx 278
client/src/components/diagnostic-dashboard.tsx 279
client/src/components/diff-viewer.tsx 280
client/src/components/ide-workspace.tsx 281
client/src/components/loader.tsx 282
client/src/components/logo.tsx 283
client/src/components/mode-toggle.tsx 284
client/src/components/product-guide.tsx 285
client/src/components/sidebar/chat-sessions.tsx 286
client/src/components/sidebar/index.tsx 287
client/src/components/sidebar/navitems.tsx 288
client/src/components/sidebar/session-search-dialog.tsx 289
client/src/components/theme-provider.tsx 290
client/src/components/ui/alert.tsx 291
client/src/components/ui/avatar.tsx 292
client/src/components/ui/badge.tsx 293
client/src/components/ui/button-group.tsx 294
client/src/components/ui/button.tsx 295
client/src/components/ui/collapsible.tsx 296
client/src/components/ui/command.tsx 297
client/src/components/ui/dialog.tsx 298
client/src/components/ui/dropdown-menu.tsx 299
client/src/components/ui/empty.tsx 300
client/src/components/ui/hover-card.tsx 301
client/src/components/ui/input-group.tsx 302
client/src/components/ui/input.tsx 303
client/src/components/ui/label.tsx 304
client/src/components/ui/popover.tsx 305
client/src/components/ui/scroll-area.tsx 306
client/src/components/ui/select.tsx 307
client/src/components/ui/separator.tsx 308
client/src/components/ui/sheet.tsx 309
client/src/components/ui/sidebar.tsx 310
client/src/components/ui/skeleton.tsx 311
client/src/components/ui/sonner.tsx 312

Source file index

PART IV NAVIGATION

client/src/components/ui/spinner.tsx 313
client/src/components/ui/table.tsx 314
client/src/components/ui/textarea.tsx 315
client/src/components/ui/tooltip.tsx 316
client/src/components/vscode-editor.tsx 317
client/src/devops/ui/diagnostic-dashboard.tsx 318
client/src/hooks/use-agent-session.ts 319
client/src/hooks/use-mobile.ts 320
client/src/hooks/use-user.ts 321
client/src/index.css 322
client/src/layouts/app-layout.tsx 323
client/src/layouts/base-layout.tsx 324
client/src/lib/api.ts 325
client/src/lib/auth-redirect.ts 326
client/src/lib/axios-client.ts 327
client/src/lib/env.ts 328
client/src/lib/utils.ts 329
client/src/main.tsx 330
client/src/pages/auth/sign-in.tsx 331
client/src/pages/auth/sign-up.tsx 332
client/src/pages/billing/index.tsx 333
client/src/pages/demo/index.tsx 334
client/src/pages/dev/DevRunInspector.tsx 335
client/src/pages/error-boundary/index.tsx 336
client/src/pages/guide/index.tsx 337
client/src/pages/home/index.tsx 338
client/src/pages/incidents/index.tsx 339
client/src/pages/not-found/index.tsx 340
client/src/pages/session/index.tsx 341
client/src/routes/index.tsx 342
client/src/routes/route-guard.tsx 343
client/src/routes/route.ts 344
client/src/types/agent.type.ts 345
client/src/types/auth.type.ts 346
client/src/types/github.type.ts 347
client/src/types/incident.type.ts 348
client/src/types/session.type.ts 349
client/tsconfig.app.json 350
client/tsconfig.json 351
client/tsconfig.node.json 352
client/vercel.json 353
client/vite.config.ts 354
docker-compose.yml 355
docs/AGENT_SAFETY.md 356
docs/APPROVAL_MODEL.md 357
docs/ARCHITECTURE.md 358

Source file index

PART IV NAVIGATION

docs/DELIVERY_WORKFLOW.md	359
docs/DEMO_SCRIPT.md	360
docs/DEPLOYMENT.md	361
docs/DIAGNOSTIC_DASHBOARD.md	362
docs/ENGINEERING_GRAPH.md	363
docs/EVALUATION.md	364
docs/EXECUTION_SANDBOX.md	365
docs/FAILURE_HANDLING.md	366
docs/FEATURE_PARITY_MATRIX.md	367
docs/GITHUB_ACTIONS_INTELLIGENCE.md	368
docs/IMPLEMENTATION_AUDIT.md	369
docs/IMPLEMENTATION_GAP_ANALYSIS.md	370
docs/IMPLEMENTATION_PROGRESS.md	371
docs/IMPLEMENTATION_STATUS.md	372
docs/INTERVIEW_TALK_TRACK.md	373
docs/LIVE_EVALUATION.md	374
docs/OBSERVABILITY.md	375
docs/OPERATIONAL_MEMORY.md	376
docs/PATCH_ENGINE.md	377
docs/PRODUCTION_READINESS.md	378
docs/PRODUCTION_READINESS_AUDIT.md	379
docs/PRODUCT_STRATEGY.md	380
docs/SAAS_SECURITY_AND_SCALE.md	381
docs/SYSTEM_DESIGN.md	382
docs/THREAT_MODEL.md	383
load/k6/README.md	384
load/k6/authenticated-session-read.js	385
load/k6/public-health.js	386
render.free.yaml	387
render.mvp.yaml	388
render.yaml	389
scripts/demo.ps1	390
scripts/demo.sh	391
scripts/generate_engineering_book.py	392
scripts/verify.ps1	393
scripts/verify.sh	394

Source file reference

CONTINUOUS INTEGRATION CONFIGURATION

FILE REFERENCE:

.github/workflows/ci.yml

Category: Continuous integration configuration | Kind: YAML configuration | Lines: 103 | SHA-256: 5d76068fba46d04a...

Purpose

This YAML configuration is part of the continuous integration configuration layer. It supplies the repository behavior or operational contract associated with name, on, permissions.

Key responsibilities

It exists to provide a coherent YAML configuration boundary for Ci. Its extracted size is 103 lines and 260 lexical tokens; the source digest begins 5d76068fba46.

Important symbols

Important exported or declared symbols/sections identified by static inspection: name, on, permissions, concurrency, jobs. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, .github/workflows/deployment-readiness.yml, .github/workflows/live-evals.yml. The containing category is Continuous integration configuration.

Simplified pseudocode

```
READ .github/workflows/ci.yml from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when GitHub Actions evaluates repository automation. Base64 Ops may inspect it as untrusted repository evidence but never promotes its run commands to local validators.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: .github/workflows/ci.yml is a YAML configuration that owns a focused part of continuous integration configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

CONTINUOUS INTEGRATION CONFIGURATION

FILE REFERENCE:

.github/workflows/deployment-readiness.yml

Category: Continuous integration configuration | Kind: YAML configuration | Lines: 48 | SHA-256: ceaec5bb7d72a409...

Purpose

This YAML configuration is part of the continuous integration configuration layer. It supplies the repository behavior or operational contract associated with name, on, permissions.

Key responsibilities

It exists to provide a coherent YAML configuration boundary for Deployment Readiness. Its extracted size is 48 lines and 125 lexical tokens; the source digest begins ceaec5bb7d72.

Important symbols

Important exported or declared symbols/sections identified by static inspection: name, on, permissions, jobs. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Continuous integration configuration.

Simplified pseudocode

```
READ .github/workflows/deployment-readiness.yml from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when GitHub Actions evaluates repository automation. Base64 Ops may inspect it as untrusted repository evidence but never promotes its run commands to local validators.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: .github/workflows/deployment-readiness.yml is a YAML configuration that owns a focused part of continuous integration configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

CONTINUOUS INTEGRATION CONFIGURATION

FILE REFERENCE:

.github/workflows/live-evals.yml

Category: Continuous integration configuration | Kind: YAML configuration | Lines: 53 | SHA-256: 85b6e6225e4a79c4...

Purpose

This YAML configuration is part of the continuous integration configuration layer. It supplies the repository behavior or operational contract associated with name, on, permissions.

Key responsibilities

It exists to provide a coherent YAML configuration boundary for Live Evals. Its extracted size is 53 lines and 155 lexical tokens; the source digest begins 85b6e6225e4a.

Important symbols

Important exported or declared symbols/sections identified by static inspection: name, on, permissions, concurrency, jobs. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/IMPLEMENTATION_STATUS.md, docs/LIVE_EVALUATION.md. The containing category is Continuous integration configuration.

Simplified pseudocode

```
READ .github/workflows/live-evals.yml from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when GitHub Actions evaluates repository automation. Base64 Ops may inspect it as untrusted repository evidence but never promotes its run commands to local validators.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: .github/workflows/live-evals.yml is a YAML configuration that owns a focused part of continuous integration configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

REPOSITORY REFERENCE

FILE REFERENCE:

README.md

Category: Repository reference | Kind: documentation | Lines: 83 | SHA-256: 75f47836d84213bd...

Purpose

This documentation file records the repository's current guidance for Readme. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Readme. Its extracted size is 83 lines and 525 lexical tokens; the source digest begins 75f47836d842.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Base64 Ops, What it does, Why it is different, Flagship demo, Architecture, Quick start, Evaluation and verification. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/docs/CONTEXT_ENGINEERING.md, backend/evals/cases/prompt-injection.json, backend/evals/cases/stale-readme.json, backend/tests/test_load_harness_safety.py. The containing category is Repository reference.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: README.md is a documentation that owns a focused part of repository reference. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

PROJECT CONFIGURATION

FILE REFERENCE:

backend/.env.example

Category: Project configuration | Kind: configuration | Lines: 146 | SHA-256: 63b77f686a844137...

Purpose

This configuration is part of the project configuration layer. It supplies the repository behavior or operational contract associated with its declared configuration and content structure.

Key responsibilities

It exists to provide a coherent configuration boundary for .Env. Its extracted size is 146 lines and 854 lexical tokens; the source digest begins 63b77f686a84.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/README.md. The containing category is Project configuration.

Simplified pseudocode

```
READ backend/.env.example from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: backend/.env.example is a configuration that owns a focused part of project configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DEPLOYMENT CONFIGURATION

FILE REFERENCE:

backend/Dockerfile

Category: Deployment configuration | Kind: configuration | Lines: 19 | SHA-256: 7e4c2e05d2eeb5ba...

Purpose

This configuration is part of the deployment configuration layer. It supplies the repository behavior or operational contract associated with its declared configuration and content structure.

Key responsibilities

It exists to provide a coherent configuration boundary for Dockerfile. Its extracted size is 19 lines and 67 lexical tokens; the source digest begins 7e4c2e05d2ee.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/execution/docker_runtime.py, backend/app/services/workspace_service.py, backend/tests/test_agent_policy.py, backend/tests/test_redaction_and_investigation.py. The containing category is Deployment configuration.

Simplified pseudocode

```
READ backend/Dockerfile from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: backend/Dockerfile is a configuration that owns a focused part of deployment configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

REPOSITORY REFERENCE

FILE REFERENCE:

backend/README.md

Category: Repository reference | Kind: documentation | Lines: 27 | SHA-256: f1600ae0ad40b5b6...

Purpose

This documentation file records the repository's current guidance for Readme. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Readme. Its extracted size is 27 lines and 122 lexical tokens; the source digest begins f1600ae0ad40.

Important symbols

Important exported or declared symbols/sections identified by static inspection: LangGraph DevOps Agent API, Local setup, RAG, DevOps safety. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/docs/CONTEXT_ENGINEERING.md, backend/evals/cases/prompt-injection.json, backend/evals/cases/stale-readme.json, backend/tests/test_load_harness_safety.py. The containing category is Repository reference.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: backend/README.md is a documentation that owns a focused part of repository reference. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/__init__.py

Category: Backend platform | Kind: Python module | Lines: 1 | SHA-256: f68b575ce78a8ad5...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around its declared configuration and content structure and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Init . Its extracted size is 1 lines and 5 lexical tokens; the source digest begins f68b575ce78a.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/tools.py, backend/app/api/middleware.py, backend/app/billing/billing_service.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/__init__.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/__init__.py is a Python module that owns a focused part of backend platform. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

LANGGRAPH AGENT ORCHESTRATION

FILE REFERENCE:

backend/app/agent/graph.py

Category: LangGraph agent orchestration | Kind: Python module | Lines: 1520 | SHA-256: ff4c66dc9c9a158d...

Purpose

This Python module contributes to the langgraph agent orchestration subsystem. It defines focused behavior around _provider_usage_event, _extract_memory_candidates, AgentGraph and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Graph. Its extracted size is 1520 lines and 5424 lexical tokens; the source digest begins ff4c66dc9c9a.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _provider_usage_event, _extract_memory_candidates, AgentGraph. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/agent/policy.py, backend/app/agent/prompts.py, backend/app/agent/state.py, backend/app/agent/tools.py, backend/app/core/config.py. Potential consumers found through bounded textual/import analysis: backend/.env.example, backend/app/agent/state.py, backend/app/api/routes/session.py, backend/app/billing/usage_service.py. The containing category is LangGraph agent orchestration.

Simplified pseudocode

```
READ backend/app/agent/graph.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/agent/graph.py is a Python module that owns a focused part of langgraph agent orchestration. Related files: backend/app/agent/policy.py, backend/app/agent/prompts.py, backend/app/agent/state.py, backend/app/agent/tools.py, backend/app/core/config.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

LANGGRAPH AGENT ORCHESTRATION

FILE REFERENCE:

backend/app/agent/policy.py

Category: LangGraph agent orchestration | Kind: Python module | Lines: 79 | SHA-256: 4fc1c38976284db7...

Purpose

This Python module contributes to the langgraph agent orchestration subsystem. It defines focused behavior around MUTATING_KEYWORDS, ACTION_TOOL_NAMES, is_risky_prompt and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Policy. Its extracted size is 79 lines and 224 lexical tokens; the source digest begins 4fc1c3897628.

Important symbols

Important exported or declared symbols/sections identified by static inspection: MUTATING_KEYWORDS, ACTION_TOOL_NAMES, is_risky_prompt, choose_action, action_requires_approval. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/agent/state.py. Potential consumers found through bounded textual/import analysis: README.md, backend/app/agent/graph.py, backend/app/core/agent_run_guard.py, backend/app/core/failures.py. The containing category is LangGraph agent orchestration.

Simplified pseudocode

```
READ backend/app/agent/policy.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/agent/policy.py is a Python module that owns a focused part of langgraph agent orchestration. Related files: backend/app/agent/state.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

LANGGRAPH AGENT ORCHESTRATION

FILE REFERENCE:

backend/app/agent/prompts.py

Category: LangGraph agent orchestration | Kind: Python module | Lines: 35 | SHA-256: 9e326f92cfec5739...

Purpose

This Python module contributes to the langgraph agent orchestration subsystem. It defines focused behavior around SYSTEM_PROMPT, build_answer_prompt and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Prompts. Its extracted size is 35 lines and 217 lexical tokens; the source digest begins 9e326f92cfec.

Important symbols

Important exported or declared symbols/sections identified by static inspection: SYSTEM_PROMPT, build_answer_prompt. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/state.py, backend/app/api/routes/assistant.py, backend/app/schemas/assistant.py. The containing category is LangGraph agent orchestration.

Simplified pseudocode

```
READ backend/app/agent/prompts.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/agent/prompts.py is a Python module that owns a focused part of langgraph agent orchestration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

LANGGRAPH AGENT ORCHESTRATION

FILE REFERENCE:

backend/app/agent/state.py

Category: LangGraph agent orchestration | Kind: Python module | Lines: 66 | SHA-256: 4b1e1e449535680d...

Purpose

This Python module contributes to the langgraph agent orchestration subsystem. It defines focused behavior around AgentAction, AgentState and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for State. Its extracted size is 66 lines and 277 lexical tokens; the source digest begins 4b1e1e449535.

Important symbols

Important exported or declared symbols/sections identified by static inspection: AgentAction, AgentState. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/taste.md, README.md, backend/.env.example, backend/app/agent/graph.py. The containing category is LangGraph agent orchestration.

Simplified pseudocode

```
READ backend/app/agent/state.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/agent/state.py is a Python module that owns a focused part of langgraph agent orchestration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

LANGGRAPH AGENT ORCHESTRATION

FILE REFERENCE:

backend/app/agent/tools.py

Category: LangGraph agent orchestration | Kind: Python module | Lines: 45 | SHA-256: 07f611fcd41aec0c...

Purpose

This Python module contributes to the langgraph agent orchestration subsystem. It defines focused behavior around DevOpsToolbox and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Tools. Its extracted size is 45 lines and 180 lexical tokens; the source digest begins 07f611fcd41a.

Important symbols

Important exported or declared symbols/sections identified by static inspection: DevOpsToolbox. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/services/workspace_service.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/policy.py, backend/app/core/config.py, backend/evals/cases/actions-path.json. The containing category is LangGraph agent orchestration.

Simplified pseudocode

```
READ backend/app/agent/tools.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/agent/tools.py is a Python module that owns a focused part of langgraph agent orchestration. Related files: backend/app/services/workspace_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/api/deps.py

Category: Backend platform | Kind: Python module | Lines: 192 | SHA-256: fafa5bcf4c1732c2...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around MAX_CLERK_BEARER_TOKEN_CHARS, ensure_user_indexes, _authorized_parties and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Deps. Its extracted size is 192 lines and 778 lexical tokens; the source digest begins fafa5bcf4c17.

Important symbols

Important exported or declared symbols/sections identified by static inspection: MAX_CLERK_BEARER_TOKEN_CHARS, ensure_user_indexes, _authorized_parties, _get_jwks_client, _decode_clerk_token, get_current_user, get_current_user_id. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py, backend/app/db/mongo.py, backend/app/utils/datetime.py, backend/app/utils/ids.py. Potential consumers found through bounded textual/import analysis: backend/app/api/routes/assistant.py, backend/app/api/routes/auth.py, backend/app/api/routes/billing.py, backend/app/api/routes/dev.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/api/deps.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/api/deps.py is a Python module that owns a focused part of backend platform. Related files: backend/app/core/config.py, backend/app/db/mongo.py, backend/app/utils/datetime.py, backend/app/utils/ids.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/api/middleware.py

Category: Backend platform | Kind: Python module | Lines: 216 | SHA-256: d05a741c8b22d6bc...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around _EXPENSIVE_PATHS, _BYPASS_PATHS, _EXPENSIVE_SESSION_SUFFIXES and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Middleware. Its extracted size is 216 lines and 818 lexical tokens; the source digest begins d05a741c8b22.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _EXPENSIVE_PATHS, _BYPASS_PATHS, _EXPENSIVE_SESSION_SUFFIXES, declared_body_exceeds_limit, trusted_client_ip, is_expensive_path, RequestBodyTooLarge. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py, backend/app/core/rate_limit.py. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/app/main.py, backend/tests/test_http_protection.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/api/middleware.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/api/middleware.py is a Python module that owns a focused part of backend platform. Related files: backend/app/core/config.py, backend/app/core/rate_limit.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND API ROUTES

FILE REFERENCE:

backend/app/api/routes/assistant.py

Category: Backend API routes | Kind: Python module | Lines: 41 | SHA-256: 3c10695dcb058e14...

Purpose

This router exposes a tenant-aware FastAPI boundary for Assistant. It validates the request context, delegates domain work to existing services, and returns browser-safe data rather than implementing business logic in the route itself.

Key responsibilities

It exists to provide a coherent Python module boundary for Assistant. Its extracted size is 41 lines and 150 lexical tokens; the source digest begins 3c10695dcb05.

Important symbols

Important exported or declared symbols/sections identified by static inspection: starter_prompts, chat. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/api/deps.py, backend/app/billing/usage_service.py, backend/app/core/agent_run_guard.py, backend/app/schemas/assistant.py, backend/app/services/product_assistant_service.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/api/middleware.py, backend/app/main.py, backend/app/schemas/session.py. The containing category is Backend API routes.

Simplified pseudocode

```
READ backend/app/api/routes/assistant.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates at the authenticated HTTP boundary, before service coordination and after browser-safe response projection.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/api/routes/assistant.py is a Python module that owns a focused part of backend api routes. Related files: backend/app/api/deps.py, backend/app/billing/usage_service.py, backend/app/core/agent_run_guard.py, backend/app/schemas/assistant.py, backend/app/services/product_assistant_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND API ROUTES

FILE REFERENCE:

backend/app/api/routes/auth.py

Category: Backend API routes | Kind: Python module | Lines: 40 | SHA-256: c612b2cf62b164cc...

Purpose

This router exposes a tenant-aware FastAPI boundary for Auth. It validates the request context, delegates domain work to existing services, and returns browser-safe data rather than implementing business logic in the route itself.

Key responsibilities

It exists to provide a coherent Python module boundary for Auth. Its extracted size is 40 lines and 170 lexical tokens; the source digest begins c612b2cf62b1.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _me_response, sync_user, get_me. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/api/deps.py, backend/app/db/mongo.py, backend/app/services/serializers.py. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, .github/workflows/ci.yml, README.md. The containing category is Backend API routes.

Simplified pseudocode

```
READ backend/app/api/routes/auth.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates at the authenticated HTTP boundary, before service coordination and after browser-safe response projection.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/api/routes/auth.py is a Python module that owns a focused part of backend api routes. Related files: backend/app/api/deps.py, backend/app/db/mongo.py, backend/app/services/serializers.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND API ROUTES

FILE REFERENCE:

backend/app/api/routes/billing.py

Category: Backend API routes | Kind: Python module | Lines: 40 | SHA-256: 64d289492e6bb08d...

Purpose

This router exposes a tenant-aware FastAPI boundary for Billing. It validates the request context, delegates domain work to existing services, and returns browser-safe data rather than implementing business logic in the route itself.

Key responsibilities

It exists to provide a coherent Python module boundary for Billing. Its extracted size is 40 lines and 183 lexical tokens; the source digest begins 64d289492e6b.

Important symbols

Important exported or declared symbols/sections identified by static inspection: entitlement, checkout, razorpay_webhook. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/api/deps.py, backend/app/billing/billing_service.py, backend/app/billing/razorpay.py, backend/app/core/config.py, backend/app/schemas/billing.py. Potential consumers found through bounded textual/import analysis: backend/.env.example, backend/app/agent/graph.py, backend/app/api/routes/assistant.py, backend/app/api/routes/diagnostic.py. The containing category is Backend API routes.

Simplified pseudocode

```
READ backend/app/api/routes/billing.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates at the authenticated HTTP boundary, before service coordination and after browser-safe response projection.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/api/routes/billing.py is a Python module that owns a focused part of backend api routes. Related files: backend/app/api/deps.py, backend/app/billing/billing_service.py, backend/app/billing/razorpay.py, backend/app/core/config.py, backend/app/schemas/billing.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND API ROUTES

FILE REFERENCE:

backend/app/api/routes/dev.py

Category: Backend API routes | Kind: Python module | Lines: 15 | SHA-256: f7f5042d36593f4d...

Purpose

This router exposes a tenant-aware FastAPI boundary for Dev. It validates the request context, delegates domain work to existing services, and returns browser-safe data rather than implementing business logic in the route itself.

Key responsibilities

It exists to provide a coherent Python module boundary for Dev. Its extracted size is 15 lines and 52 lexical tokens; the source digest begins f7f5042d3659.

Important symbols

Important exported or declared symbols/sections identified by static inspection: get_dev_run_inspector. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/api/deps.py, backend/app/schemas/replay.py, backend/app/services/session_service.py. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/.env.example, backend/README.md. The containing category is Backend API routes.

Simplified pseudocode

```
READ backend/app/api/routes/dev.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates at the authenticated HTTP boundary, before service coordination and after browser-safe response projection.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/api/routes/dev.py is a Python module that owns a focused part of backend api routes. Related files: backend/app/api/deps.py, backend/app/schemas/replay.py, backend/app/services/session_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND API ROUTES

FILE REFERENCE:

backend/app/api/routes/diagnostic.py

Category: Backend API routes | Kind: Python module | Lines: 60 | SHA-256: 6f6e6ab706a3cc4a...

Purpose

This router exposes a tenant-aware FastAPI boundary for Diagnostic. It validates the request context, delegates domain work to existing services, and returns browser-safe data rather than implementing business logic in the route itself.

Key responsibilities

It exists to provide a coherent Python module boundary for Diagnostic. Its extracted size is 60 lines and 203 lexical tokens; the source digest begins 6f6e6ab706a3.

Important symbols

Important exported or declared symbols/sections identified by static inspection: stream, apply_approved_patch. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/api/deps.py, backend/app/billing/usage_service.py, backend/app/core/agent_run_guard.py, backend/app/schemas/session.py, backend/app/services/diagnostic_stream_service.py. Potential consumers found through bounded textual/import analysis: backend/README.md, backend/app/agent/graph.py, backend/app/agent/prompts.py, backend/app/api/middleware.py. The containing category is Backend API routes.

Simplified pseudocode

```
READ backend/app/api/routes/diagnostic.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates at the authenticated HTTP boundary, before service coordination and after browser-safe response projection.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/api/routes/diagnostic.py is a Python module that owns a focused part of backend api routes. Related files: backend/app/api/deps.py, backend/app/billing/usage_service.py, backend/app/core/agent_run_guard.py, backend/app/schemas/session.py, backend/app/services/diagnostic_stream_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND API ROUTES

FILE REFERENCE:

backend/app/api/routes/github.py

Category: Backend API routes | Kind: Python module | Lines: 52 | SHA-256: 9d677be1aeea811a...

Purpose

This router exposes a tenant-aware FastAPI boundary for Github. It validates the request context, delegates domain work to existing services, and returns browser-safe data rather than implementing business logic in the route itself.

Key responsibilities

It exists to provide a coherent Python module boundary for Github. Its extracted size is 52 lines and 198 lexical tokens; the source digest begins 9d677be1aeea.

Important symbols

Important exported or declared symbols/sections identified by static inspection: connect, callback, webhook, repos, disconnect. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/api/deps.py, backend/app/services/github_service.py, backend/app/services/incident_service.py. Potential consumers found through bounded textual/import analysis: .commandcode/taste/taste.md, .github/workflows/ci.yml, .github/workflows/deployment-readiness.yml, .github/workflows/live-evals.yml. The containing category is Backend API routes.

Simplified pseudocode

```
READ backend/app/api/routes/github.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates at the authenticated HTTP boundary, before service coordination and after browser-safe response projection.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/api/routes/github.py is a Python module that owns a focused part of backend api routes. Related files: backend/app/api/deps.py, backend/app/services/github_service.py, backend/app/services/incident_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND API ROUTES

FILE REFERENCE:

backend/app/api/routes/incidents.py

Category: Backend API routes | Kind: Python module | Lines: 265 | SHA-256: 194277a35c7d89b5...

Purpose

This router exposes a tenant-aware FastAPI boundary for Incidents. It validates the request context, delegates domain work to existing services, and returns browser-safe data rather than implementing business logic in the route itself.

Key responsibilities

It exists to provide a coherent Python module boundary for Incidents. Its extracted size is 265 lines and 816 lexical tokens; the source digest begins 194277a35c7d.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _settings_response, _incident_response, _flaky_observation_response, configure_automation, automation_settings, list_incidents, list_flaky_test_observations. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/api/deps.py, backend/app/db/mongo.py, backend/app/schemas/incident.py, backend/app/services/incident_export_service.py, backend/app/services/incident_service.py. Potential consumers found through bounded textual/import analysis: README.md, backend/app/main.py, backend/app/schemas/incident.py, backend/app/services/incident_service.py. The containing category is Backend API routes.

Simplified pseudocode

```
READ backend/app/api/routes/incidents.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates at the authenticated HTTP boundary, before service coordination and after browser-safe response projection.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/api/routes/incidents.py is a Python module that owns a focused part of backend api routes. Related files: backend/app/api/deps.py, backend/app/db/mongo.py, backend/app/schemas/incident.py, backend/app/services/incident_export_service.py, backend/app/services/incident_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND API ROUTES

FILE REFERENCE:

backend/app/api/routes/memory.py

Category: Backend API routes | Kind: Python module | Lines: 224 | SHA-256: 99462647319e38bb...

Purpose

This router exposes a tenant-aware FastAPI boundary for Memory. It validates the request context, delegates domain work to existing services, and returns browser-safe data rather than implementing business logic in the route itself.

Key responsibilities

It exists to provide a coherent Python module boundary for Memory. Its extracted size is 224 lines and 761 lexical tokens; the source digest begins 99462647319e.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _repository_id_from_url, get_staleness_report, trigger_staleness_check, delete_memory_entry, refresh_memory_entry, promote_memory_entry, list_memory_entries. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/api/routes/auth.py, backend/app/core/config.py, backend/app/memory/memory_service.py, backend/app/services/workspace_service.py, backend/app/db/mongo.py. Potential consumers found through bounded textual/import analysis: README.md, backend/app/agent/graph.py, backend/app/agent/state.py, backend/app/api/routes/incidents.py. The containing category is Backend API routes.

Simplified pseudocode

```
READ backend/app/api/routes/memory.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates at the authenticated HTTP boundary, before service coordination and after browser-safe response projection.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/api/routes/memory.py is a Python module that owns a focused part of backend api routes. Related files: backend/app/api/routes/auth.py, backend/app/core/config.py, backend/app/memory/memory_service.py, backend/app/services/workspace_service.py, backend/app/db/mongo.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND API ROUTES

FILE REFERENCE:

backend/app/api/routes/session.py

Category: Backend API routes | Kind: Python module | Lines: 263 | SHA-256: d955976036b0fcf2...

Purpose

This router exposes a tenant-aware FastAPI boundary for Session. It validates the request context, delegates domain work to existing services, and returns browser-safe data rather than implementing business logic in the route itself.

Key responsibilities

It exists to provide a coherent Python module boundary for Session. Its extracted size is 263 lines and 884 lexical tokens; the source digest begins d955976036b0.

Important symbols

Important exported or declared symbols/sections identified by static inspection: all_sessions, chat, approval, reindex, sources, evidence, refresh_ci. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/api/deps.py, backend/app/billing/usage_service.py, backend/app/core/agent_run_guard.py, backend/app/schemas/intelligence.py, backend/app/schemas/replay.py. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/.env.example, backend/app/agent/graph.py. The containing category is Backend API routes.

Simplified pseudocode

```
READ backend/app/api/routes/session.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates at the authenticated HTTP boundary, before service coordination and after browser-safe response projection.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/api/routes/session.py is a Python module that owns a focused part of backend api routes. Related files: backend/app/api/deps.py, backend/app/billing/usage_service.py, backend/app/core/agent_run_guard.py, backend/app/schemas/intelligence.py, backend/app/schemas/replay.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND API ROUTES

FILE REFERENCE:

backend/app/api/routes/system.py

Category: Backend API routes | Kind: Python module | Lines: 66 | SHA-256: eb4ae5f1c57324fa...

Purpose

This router exposes a tenant-aware FastAPI boundary for System. It validates the request context, delegates domain work to existing services, and returns browser-safe data rather than implementing business logic in the route itself.

Key responsibilities

It exists to provide a coherent Python module boundary for System. Its extracted size is 66 lines and 158 lexical tokens; the source digest begins eb4ae5f1c573.

Important symbols

Important exported or declared symbols/sections identified by static inspection: get_system_capabilities. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py, backend/app/execution/docker_runtime.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/execution/__init__.py, backend/app/intelligence/graph/models.py, backend/app/main.py. The containing category is Backend API routes.

Simplified pseudocode

```
READ backend/app/api/routes/system.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates at the authenticated HTTP boundary, before service coordination and after browser-safe response projection.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/api/routes/system.py is a Python module that owns a focused part of backend api routes. Related files: backend/app/core/config.py, backend/app/execution/docker_runtime.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/billing/__init__.py

Category: Backend platform | Kind: Python module | Lines: 1 | SHA-256: 08511684fb5c21a6...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around its declared configuration and content structure and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Init . Its extracted size is 1 lines and 11 lexical tokens; the source digest begins 08511684fb5c.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/tools.py, backend/app/api/middleware.py, backend/app/billing/billing_service.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/billing/__init__.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/billing/__init__.py is a Python module that owns a focused part of backend platform. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/billing/billing_service.py

Category: Backend platform | Kind: Python module | Lines: 277 | SHA-256: 77055c4f4fc15f35...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around provider_event_created_at, BillingService and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Billing Service. Its extracted size is 277 lines and 1020 lexical tokens; the source digest begins 77055c4f4fc1.

Important symbols

Important exported or declared symbols/sections identified by static inspection: provider_event_created_at, BillingService. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/billing/models.py, backend/app/billing/razorpay.py, backend/app/billing/usage_service.py, backend/app/core/config.py, backend/app/db/mongo.py. Potential consumers found through bounded textual/import analysis: backend/app/api/routes/billing.py, backend/app/main.py, backend/tests/test_billing_lifecycle.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/billing/billing_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/billing/billing_service.py is a Python module that owns a focused part of backend platform. Related files: backend/app/billing/models.py, backend/app/billing/razorpay.py, backend/app/billing/usage_service.py, backend/app/core/config.py, backend/app/db/mongo.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/billing/models.py

Category: Backend platform | Kind: Python module | Lines: 29 | SHA-256: fbc3275e241f3d09...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around SubscriptionTier, SubscriptionStatus, UsageEntitlement and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Models. Its extracted size is 29 lines and 51 lexical tokens; the source digest begins fbc3275e241f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: SubscriptionTier, SubscriptionStatus, UsageEntitlement. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/app/agent/graph.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/billing/models.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/billing/models.py is a Python module that owns a focused part of backend platform. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/billing/razorpay.py

Category: Backend platform | Kind: Python module | Lines: 143 | SHA-256: b88ac1099f186896...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around _BASE_URL, _TRUSTED_CHECKOUT_HOSTS, verify_webhook_signature and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Razorpay. Its extracted size is 143 lines and 505 lexical tokens; the source digest begins b88ac1099f18.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _BASE_URL, _TRUSTED_CHECKOUT_HOSTS, verify_webhook_signature, trusted_checkout_url, HostedSubscription, RazorpayClient. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/billing/models.py, backend/app/core/config.py. Potential consumers found through bounded textual/import analysis: backend/app/api/routes/billing.py, backend/app/billing/billing_service.py, backend/app/core/config.py, backend/tests/test_billing_lifecycle.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/billing/razorpay.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/billing/razorpay.py is a Python module that owns a focused part of backend platform. Related files: backend/app/billing/models.py, backend/app/core/config.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/billing/usage_service.py

Category: Backend platform | Kind: Python module | Lines: 236 | SHA-256: 205720d736eaebf4...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around monthly_period_key, resolve_tier, agent_run_limit and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Usage Service. Its extracted size is 236 lines and 821 lexical tokens; the source digest begins 205720d736ea.

Important symbols

Important exported or declared symbols/sections identified by static inspection: monthly_period_key, resolve_tier, agent_run_limit, model_token_limit, request_key_hash, reported_token_count, UsageService. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/billing/models.py, backend/app/core/config.py, backend/app/db/mongo.py, backend/app/utils/datetime.py. Potential consumers found through bounded textual/import analysis: backend/app/api/routes/assistant.py, backend/app/api/routes/diagnostic.py, backend/app/api/routes/session.py, backend/app/billing/billing_service.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/billing/usage_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/billing/usage_service.py is a Python module that owns a focused part of backend platform. Related files: backend/app/billing/models.py, backend/app/core/config.py, backend/app/db/mongo.py, backend/app/utils/datetime.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/core/access_logging.py

Category: Backend platform | Kind: Python module | Lines: 44 | SHA-256: dfe890cd5047e19c...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around redact_access_path_query, AccessLogQueryRedactionFilter, configure_access_log_redaction and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Access Logging. Its extracted size is 44 lines and 207 lexical tokens; the source digest begins dfe890cd5047.

Important symbols

Important exported or declared symbols/sections identified by static inspection: redact_access_path_query, AccessLogQueryRedactionFilter, configure_access_log_redaction. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/main.py, backend/tests/test_http_protection.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/core/access_logging.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/core/access_logging.py is a Python module that owns a focused part of backend platform. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/core/agent_run_guard.py

Category: Backend platform | Kind: Python module | Lines: 136 | SHA-256: 6759302e687bf17b...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around _tenant_key, LeaseStore, InMemoryLeaseStore and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Agent Run Guard. Its extracted size is 136 lines and 548 lexical tokens; the source digest begins 6759302e687b.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _tenant_key, LeaseStore, InMemoryLeaseStore, RedisLeaseStore, AgentRunLease, AgentRunGuard. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py. Potential consumers found through bounded textual/import analysis: backend/app/api/routes/assistant.py, backend/app/api/routes/diagnostic.py, backend/app/api/routes/session.py, backend/tests/test_agent_run_guard.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/core/agent_run_guard.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/core/agent_run_guard.py is a Python module that owns a focused part of backend platform. Related files: backend/app/core/config.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/core/config.py

Category: Backend platform | Kind: Python module | Lines: 307 | SHA-256: a5203583f1e0943c...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around _is_fernet_key, Settings, get_settings and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Config. Its extracted size is 307 lines and 1967 lexical tokens; the source digest begins a5203583f1e0.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _is_fernet_key, Settings, get_settings. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/README.md. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/core/config.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/core/config.py is a Python module that owns a focused part of backend platform. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/core/failures.py

Category: Backend platform | Kind: Python module | Lines: 16 | SHA-256: 0ca668c912599b9a...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around FailureType, is_retryable and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Failures. Its extracted size is 16 lines and 45 lexical tokens; the source digest begins 0ca668c91259.

Important symbols

Important exported or declared symbols/sections identified by static inspection: FailureType, is_retryable. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: README.md, backend/app/agent/graph.py, backend/app/services/candidate_verification_service.py, backend/app/services/ci_applicability.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/core/failures.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/core/failures.py is a Python module that owns a focused part of backend platform. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/core/rate_limit.py

Category: Backend platform | Kind: Python module | Lines: 132 | SHA-256: e16f4df51e186210...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around RateLimitDecision, RateLimitStore, InMemoryRateLimitStore and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Rate Limit. Its extracted size is 132 lines and 571 lexical tokens; the source digest begins e16f4df51e18.

Important symbols

Important exported or declared symbols/sections identified by static inspection: RateLimitDecision, RateLimitStore, InMemoryRateLimitStore, RedisRateLimitStore, rate_limit_key. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/api/middleware.py, backend/app/core/agent_run_guard.py, backend/app/core/config.py, backend/app/main.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/core/rate_limit.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/core/rate_limit.py is a Python module that owns a focused part of backend platform. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/core/security.py

Category: Backend platform | Kind: Python module | Lines: 144 | SHA-256: 42c0e2c0d654df8b...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around _password_digest, hash_password, verify_password and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Security. Its extracted size is 144 lines and 576 lexical tokens; the source digest begins 42c0e2c0d654.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _password_digest, hash_password, verify_password, create_access_token, decode_access_token, _legacy_fernet, _fernet. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py. Potential consumers found through bounded textual/import analysis: backend/app/core/config.py, backend/app/schemas/investigation.py, backend/app/services/auth_service.py, backend/app/services/delivery_service.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/core/security.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/core/security.py is a Python module that owns a focused part of backend platform. Related files: backend/app/core/config.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/db/mongo.py

Category: Backend platform | Kind: Python module | Lines: 64 | SHA-256: cdcd625c182713bd...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around MongoUnavailableError, connect_mongo, close_mongo and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Mongo. Its extracted size is 64 lines and 199 lexical tokens; the source digest begins cdcd625c1827.

Important symbols

Important exported or declared symbols/sections identified by static inspection: MongoUnavailableError, connect_mongo, close_mongo, get_db, mongo_is_connected. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, backend/.env.example, backend/README.md, backend/app/agent/graph.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/db/mongo.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/db/mongo.py is a Python module that owns a focused part of backend platform. Related files: backend/app/core/config.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

EXECUTION SAFETY

FILE REFERENCE:

backend/app/execution/__init__.py

Category: Execution safety | Kind: Python module | Lines: 1 | SHA-256: 0e2f57fe5fe3bc78...

Purpose

This Python module contributes to the execution safety subsystem. It defines focused behavior around its declared configuration and content structure and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Init . Its extracted size is 1 lines and 6 lexical tokens; the source digest begins 0e2f57fe5fe3.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/tools.py, backend/app/api/middleware.py, backend/app/billing/billing_service.py. The containing category is Execution safety.

Simplified pseudocode

```
READ backend/app/execution/__init__.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/execution/__init__.py is a Python module that owns a focused part of execution safety. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

EXECUTION SAFETY

FILE REFERENCE:

backend/app/execution/docker_runtime.py

Category: Execution safety | Kind: Python module | Lines: 111 | SHA-256: 3bec59db926980b5...

Purpose

This Python module contributes to the execution safety subsystem. It defines focused behavior around DockerSandboxRuntime and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Docker Runtime. Its extracted size is 111 lines and 370 lexical tokens; the source digest begins 3bec59db9269.

Important symbols

Important exported or declared symbols/sections identified by static inspection: DockerSandboxRuntime. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/execution/models.py, backend/app/services/redaction_service.py. Potential consumers found through bounded textual/import analysis: backend/app/api/routes/system.py, backend/app/execution/runtime.py, backend/tests/test_execution_sandbox.py, docs/PRODUCTION_READINESS_AUDIT.md. The containing category is Execution safety.

Simplified pseudocode

```
READ backend/app/execution/docker_runtime.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/execution/docker_runtime.py is a Python module that owns a focused part of execution safety. Related files: backend/app/execution/models.py, backend/app/services/redaction_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

EXECUTION SAFETY

FILE REFERENCE:

backend/app/execution/local_runtime.py

Category: Execution safety | Kind: Python module | Lines: 108 | SHA-256: e363ab2b7c310349...

Purpose

This Python module contributes to the execution safety subsystem. It defines focused behavior around RestrictedLocalRuntime and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Local Runtime. Its extracted size is 108 lines and 339 lexical tokens; the source digest begins e363ab2b7c31.

Important symbols

Important exported or declared symbols/sections identified by static inspection: RestrictedLocalRuntime. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/execution/models.py, backend/app/execution/policy.py, backend/app/services/redaction_service.py. Potential consumers found through bounded textual/import analysis: backend/app/execution/runtime.py. The containing category is Execution safety.

Simplified pseudocode

```
READ backend/app/execution/local_runtime.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/execution/local_runtime.py is a Python module that owns a focused part of execution safety. Related files: backend/app/execution/models.py, backend/app/execution/policy.py, backend/app/services/redaction_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

EXECUTION SAFETY

FILE REFERENCE:

backend/app/execution/models.py

Category: Execution safety | Kind: Python module | Lines: 47 | SHA-256: 926ae34444dfd880...

Purpose

This Python module contributes to the execution safety subsystem. It defines focused behavior around ExecutionRequest, ExecutionResult, ValidatorDefinition and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Models. Its extracted size is 47 lines and 140 lexical tokens; the source digest begins 926ae34444df.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ExecutionRequest, ExecutionResult, ValidatorDefinition. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/app/agent/graph.py. The containing category is Execution safety.

Simplified pseudocode

```
READ backend/app/execution/models.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/execution/models.py is a Python module that owns a focused part of execution safety. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

EXECUTION SAFETY

FILE REFERENCE:

backend/app/execution/policy.py

Category: Execution safety | Kind: Python module | Lines: 83 | SHA-256: da9920197058a50e...

Purpose

This Python module contributes to the execution safety subsystem. It defines focused behavior around ExecutionPolicyError, ExecutionPolicy and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Policy. Its extracted size is 83 lines and 296 lexical tokens; the source digest begins da9920197058.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ExecutionPolicyError, ExecutionPolicy. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/services/delivery_service.py. Potential consumers found through bounded textual/import analysis: README.md, backend/app/agent/graph.py, backend/app/agent/policy.py, backend/app/core/agent_run_guard.py. The containing category is Execution safety.

Simplified pseudocode

```
READ backend/app/execution/policy.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/execution/policy.py is a Python module that owns a focused part of execution safety. Related files: backend/app/services/delivery_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

EXECUTION SAFETY

FILE REFERENCE:

backend/app/execution/registry.py

Category: Execution safety | Kind: Python module | Lines: 98 | SHA-256: d207786f0d5a81ba...

Purpose

This Python module contributes to the execution safety subsystem. It defines focused behavior around get_validator and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Registry. Its extracted size is 98 lines and 191 lexical tokens; the source digest begins d207786f0d5a.

Important symbols

Important exported or declared symbols/sections identified by static inspection: get_validator. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/execution/models.py. Potential consumers found through bounded textual/import analysis: backend/app/execution/runtime.py, backend/app/policy/__init__.py, backend/app/policy/evaluator.py, backend/app/services/candidate_verification_service.py. The containing category is Execution safety.

Simplified pseudocode

```
READ backend/app/execution/registry.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/execution/registry.py is a Python module that owns a focused part of execution safety. Related files: backend/app/execution/models.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

EXECUTION SAFETY

FILE REFERENCE:

backend/app/execution/runtime.py

Category: Execution safety | Kind: Python module | Lines: 114 | SHA-256: d06ae5d2472d2320...

Purpose

This Python module contributes to the execution safety subsystem. It defines focused behavior around ExecutionRuntimeManager and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Runtime. Its extracted size is 114 lines and 414 lexical tokens; the source digest begins d06ae5d2472d.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ExecutionRuntimeManager. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py, backend/app/execution/docker_runtime.py, backend/app/execution/local_runtime.py, backend/app/execution/models.py, backend/app/execution/policy.py. Potential consumers found through bounded textual/import analysis: backend/.env.example, backend/app/agent/graph.py, backend/app/api/routes/system.py, backend/app/core/config.py. The containing category is Execution safety.

Simplified pseudocode

```
READ backend/app/execution/runtime.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/execution/runtime.py is a Python module that owns a focused part of execution safety. Related files: backend/app/core/config.py, backend/app/execution/docker_runtime.py, backend/app/execution/local_runtime.py, backend/app/execution/models.py, backend/app/execution/policy.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/git/__init__.py

Category: Backend platform | Kind: Python module | Lines: 0 | SHA-256: e3b0c44298fc1c14...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around its declared configuration and content structure and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Init . Its extracted size is 0 lines and 0 lexical tokens; the source digest begins e3b0c44298fc.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/tools.py, backend/app/api/middleware.py, backend/app/billing/billing_service.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/git/__init__.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/git/__init__.py is a Python module that owns a focused part of backend platform. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/git/actions.py

Category: Backend platform | Kind: Python module | Lines: 353 | SHA-256: 6421d22b88cd3b0d...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around CIFailureCategory, WorkflowRunSummary, WorkflowJobSummary and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Actions. Its extracted size is 353 lines and 1475 lexical tokens; the source digest begins 6421d22b88cd.

Important symbols

Important exported or declared symbols/sections identified by static inspection: CIFailureCategory, WorkflowRunSummary, WorkflowJobSummary, CILogExcerpt, CIWorkflowHistory, CIInvestigationContext, _SIGNAL. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py, backend/app/services/redaction_service.py, backend/app/services/github_service.py. Potential consumers found through bounded textual/import analysis: .github/workflows/ci.yml, .github/workflows/deployment-readiness.yml, .github/workflows/live-evals.yml, README.md. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/git/actions.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/git/actions.py is a Python module that owns a focused part of backend platform. Related files: backend/app/core/config.py, backend/app/services/redaction_service.py, backend/app/services/github_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/git/change_provider.py

Category: Backend platform | Kind: Python module | Lines: 385 | SHA-256: 0025ac23c9147ea6...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around RenamedPath, ChangedPathsResult, _parse_name_status and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Change Provider. Its extracted size is 385 lines and 1221 lexical tokens; the source digest begins 0025ac23c914.

Important symbols

Important exported or declared symbols/sections identified by static inspection: RenamedPath, ChangedPathsResult, _parse_name_status, _normalize, GitChangeProvider. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/services/github_service.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/memory/freshness.py, backend/app/services/ci_investigation_service.py, backend/tests/test_ci_investigation_service.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/git/change_provider.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/git/change_provider.py is a Python module that owns a focused part of backend platform. Related files: backend/app/services/github_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/git/pr_status.py

Category: Backend platform | Kind: Python module | Lines: 154 | SHA-256: ca74d93a7ba5a982...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around PullRequestStatus, PullRequestStatusProvider and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Pr Status. Its extracted size is 154 lines and 432 lexical tokens; the source digest begins ca74d93a7ba5.

Important symbols

Important exported or declared symbols/sections identified by static inspection: PullRequestStatus, PullRequestStatusProvider. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/services/github_service.py. Potential consumers found through bounded textual/import analysis: backend/app/memory/memory_service.py, backend/tests/test_memory_freshness.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/git/pr_status.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/git/pr_status.py is a Python module that owns a focused part of backend platform. Related files: backend/app/services/github_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ENGINEERING INTELLIGENCE

FILE REFERENCE:

backend/app/intelligence/__init__.py

Category: Engineering intelligence | Kind: Python module | Lines: 6 | SHA-256: a702340f7a91281a...

Purpose

This Python module contributes to the engineering intelligence subsystem. It defines focused behavior around its declared configuration and content structure and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Init . Its extracted size is 6 lines and 35 lexical tokens; the source digest begins a702340f7a91.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/tools.py, backend/app/api/middleware.py, backend/app/billing/billing_service.py. The containing category is Engineering intelligence.

Simplified pseudocode

```
READ backend/app/intelligence/__init__.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/intelligence/__init__.py is a Python module that owns a focused part of engineering intelligence. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ENGINEERING INTELLIGENCE

FILE REFERENCE:

backend/app/intelligence/graph/__init__.py

Category: Engineering intelligence | Kind: Python module | Lines: 19 | SHA-256: be2be8a516635831...

Purpose

This Python module contributes to the engineering intelligence subsystem. It defines focused behavior around its declared configuration and content structure and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Init . Its extracted size is 19 lines and 25 lexical tokens; the source digest begins be2be8a51663.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/intelligence/graph/models.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/tools.py, backend/app/api/middleware.py, backend/app/billing/billing_service.py. The containing category is Engineering intelligence.

Simplified pseudocode

```
READ backend/app/intelligence/graph/__init__.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/intelligence/graph/__init__.py is a Python module that owns a focused part of engineering intelligence. Related files: backend/app/intelligence/graph/models.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ENGINEERING INTELLIGENCE

FILE REFERENCE:

backend/app/intelligence/graph/builder.py

Category: Engineering intelligence | Kind: Python module | Lines: 542 | SHA-256: 8baf2a1df9a39c74...

Purpose

This Python module contributes to the engineering intelligence subsystem. It defines focused behavior around MAX_GRAPH_FILES, MAX_GRAPH_NODES, MAX_GRAPH_EDGES and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Builder. Its extracted size is 542 lines and 1706 lexical tokens; the source digest begins 8baf2a1df9a3.

Important symbols

Important exported or declared symbols/sections identified by static inspection: MAX_GRAPH_FILES, MAX_GRAPH_NODES, MAX_GRAPH_EDGES, _SOURCE_SUFFIXES, _TYPESCRIPT_SUFFIXES, _WORKFLOW_PREFIX, _IMPORT_RE. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/intelligence/graph/models.py, backend/app/utils/datetime.py, backend/app/utils/ids.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/execution/models.py, backend/app/execution/registry.py, backend/app/intelligence/graph/service.py. The containing category is Engineering intelligence.

Simplified pseudocode

```
READ backend/app/intelligence/graph/builder.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/intelligence/graph/builder.py is a Python module that owns a focused part of engineering intelligence. Related files: backend/app/intelligence/graph/models.py, backend/app/utils/datetime.py, backend/app/utils/ids.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ENGINEERING INTELLIGENCE

FILE REFERENCE:

backend/app/intelligence/graph/models.py

Category: Engineering intelligence | Kind: Python module | Lines: 134 | SHA-256: 48d3f835eee0e508...

Purpose

This Python module contributes to the engineering intelligence subsystem. It defines focused behavior around GraphNodeType, GraphRelationType, EpistemicStatus and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Models. Its extracted size is 134 lines and 360 lexical tokens; the source digest begins 48d3f835eee0.

Important symbols

Important exported or declared symbols/sections identified by static inspection: GraphNodeType, GraphRelationType, EpistemicStatus, GraphDiscoveryMethod, GraphNode, GraphEdge, GraphCoverage. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/app/agent/graph.py. The containing category is Engineering intelligence.

Simplified pseudocode

```
READ backend/app/intelligence/graph/models.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/intelligence/graph/models.py is a Python module that owns a focused part of engineering intelligence. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ENGINEERING INTELLIGENCE

FILE REFERENCE:

backend/app/intelligence/graph/service.py

Category: Engineering intelligence | Kind: Python module | Lines: 103 | SHA-256: e5ab5d132db2ce99...

Purpose

This Python module contributes to the engineering intelligence subsystem. It defines focused behavior around DEFAULT_NEIGHBORHOOD_LIMIT, MAX_NEIGHBORHOOD_LIMIT, MAX_NEIGHBORHOOD_DEPTH and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Service. Its extracted size is 103 lines and 429 lexical tokens; the source digest begins e5ab5d132db2.

Important symbols

Important exported or declared symbols/sections identified by static inspection: DEFAULT_NEIGHBORHOOD_LIMIT, MAX_NEIGHBORHOOD_LIMIT, MAX_NEIGHBORHOOD_DEPTH, EngineeringGraphService. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/intelligence/graph/builder.py, backend/app/intelligence/graph/models.py, backend/app/intelligence/graph/store.py, backend/app/services/workspace_service.py. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/app/agent/graph.py, backend/app/agent/tools.py. The containing category is Engineering intelligence.

Simplified pseudocode

```
READ backend/app/intelligence/graph/service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/intelligence/graph/service.py is a Python module that owns a focused part of engineering intelligence. Related files: backend/app/intelligence/graph/builder.py, backend/app/intelligence/graph/models.py, backend/app/intelligence/graph/store.py, backend/app/services/workspace_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ENGINEERING INTELLIGENCE

FILE REFERENCE:

backend/app/intelligence/graph/store.py

Category: Engineering intelligence | Kind: Python module | Lines: 93 | SHA-256: 581ca25ade34f807...

Purpose

This Python module contributes to the engineering intelligence subsystem. It defines focused behavior around EngineeringGraphStore and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Store. Its extracted size is 93 lines and 313 lexical tokens; the source digest begins 581ca25ade34.

Important symbols

Important exported or declared symbols/sections identified by static inspection: EngineeringGraphStore. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/db/mongo.py, backend/app/intelligence/graph/models.py. Potential consumers found through bounded textual/import analysis: README.md, backend/.env.example, backend/app/agent/graph.py, backend/app/api/middleware.py. The containing category is Engineering intelligence.

Simplified pseudocode

```
READ backend/app/intelligence/graph/store.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/intelligence/graph/store.py is a Python module that owns a focused part of engineering intelligence. Related files: backend/app/db/mongo.py, backend/app/intelligence/graph/models.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ENGINEERING INTELLIGENCE

FILE REFERENCE:

backend/app/intelligence/impact/__init__.py

Category: Engineering intelligence | Kind: Python module | Lines: 12 | SHA-256: 1125fbdd952046a6...

Purpose

This Python module contributes to the engineering intelligence subsystem. It defines focused behavior around its declared configuration and content structure and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Init . Its extracted size is 12 lines and 63 lexical tokens; the source digest begins 1125fbdd9520.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/intelligence/impact/models.py, backend/app/intelligence/impact/service.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/tools.py, backend/app/api/middleware.py, backend/app/billing/billing_service.py. The containing category is Engineering intelligence.

Simplified pseudocode

```
READ backend/app/intelligence/impact/__init__.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/intelligence/impact/__init__.py is a Python module that owns a focused part of engineering intelligence. Related files: backend/app/intelligence/impact/models.py, backend/app/intelligence/impact/service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ENGINEERING INTELLIGENCE

FILE REFERENCE:

backend/app/intelligence/impact/models.py

Category: Engineering intelligence | Kind: Python module | Lines: 93 | SHA-256: fb250cd3d43c9359...

Purpose

This Python module contributes to the engineering intelligence subsystem. It defines focused behavior around ChangeImpactStatus, ImpactRelationship, ImpactTarget and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Models. Its extracted size is 93 lines and 297 lexical tokens; the source digest begins fb250cd3d43c.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ChangeImpactStatus, ImpactRelationship, ImpactTarget, ChangeImpactAnalysis, ChangeImpactResponse. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/intelligence/graph/models.py. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/app/agent/graph.py. The containing category is Engineering intelligence.

Simplified pseudocode

```
READ backend/app/intelligence/impact/models.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/intelligence/impact/models.py is a Python module that owns a focused part of engineering intelligence. Related files: backend/app/intelligence/graph/models.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ENGINEERING INTELLIGENCE

FILE REFERENCE:

backend/app/intelligence/impact/service.py

Category: Engineering intelligence | Kind: Python module | Lines: 243 | SHA-256: af076a01f5045e3a...

Purpose

This Python module contributes to the engineering intelligence subsystem. It defines focused behavior around DEFAULT_IMPACT_DEPTH, DEFAULT_IMPACT_LIMIT, MAX_IMPACT_DEPTH and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Service. Its extracted size is 243 lines and 1012 lexical tokens; the source digest begins af076a01f504.

Important symbols

Important exported or declared symbols/sections identified by static inspection: DEFAULT_IMPACT_DEPTH, DEFAULT_IMPACT_LIMIT, MAX_IMPACT_DEPTH, MAX_IMPACT_LIMIT, ChangeImpactService. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/intelligence/graph/models.py, backend/app/intelligence/impact/models.py, backend/app/intelligence/impact/store.py, backend/app/schemas/delivery.py, backend/app/utils/datetime.py. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/app/agent/graph.py, backend/app/agent/tools.py. The containing category is Engineering intelligence.

Simplified pseudocode

```
READ backend/app/intelligence/impact/service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/intelligence/impact/service.py is a Python module that owns a focused part of engineering intelligence. Related files: backend/app/intelligence/graph/models.py, backend/app/intelligence/impact/models.py, backend/app/intelligence/impact/store.py, backend/app/schemas/delivery.py, backend/app/utils/datetime.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ENGINEERING INTELLIGENCE

FILE REFERENCE:

backend/app/intelligence/impact/store.py

Category: Engineering intelligence | Kind: Python module | Lines: 91 | SHA-256: c829f41e79ff5d25...

Purpose

This Python module contributes to the engineering intelligence subsystem. It defines focused behavior around ChangeImpactStore and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Store. Its extracted size is 91 lines and 293 lexical tokens; the source digest begins c829f41e79ff.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ChangeImpactStore. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/db/mongo.py, backend/app/intelligence/impact/models.py. Potential consumers found through bounded textual/import analysis: README.md, backend/.env.example, backend/app/agent/graph.py, backend/app/api/middleware.py. The containing category is Engineering intelligence.

Simplified pseudocode

```
READ backend/app/intelligence/impact/store.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/intelligence/impact/store.py is a Python module that owns a focused part of engineering intelligence. Related files: backend/app/db/mongo.py, backend/app/intelligence/impact/models.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/main.py

Category: Backend platform | Kind: Python module | Lines: 239 | SHA-256: 583768453abce8d5...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around _ensure_database_indexes, _retry_mongo_connection, lifespan and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Main. Its extracted size is 239 lines and 1078 lexical tokens; the source digest begins 583768453abc.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _ensure_database_indexes, _retry_mongo_connection, lifespan, health, readiness, app_exception_handler. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/api/deps.py, backend/app/api/middleware.py, backend/app/billing/billing_service.py, backend/app/billing/usage_service.py, backend/app/core/access_logging.py. Potential consumers found through bounded textual/import analysis: .commandcode/taste/taste.md, .github/workflows/ci.yml, .github/workflows/deployment-readiness.yml, README.md. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/main.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/main.py is a Python module that owns a focused part of backend platform. Related files: backend/app/api/deps.py, backend/app/api/middleware.py, backend/app/billing/billing_service.py, backend/app/billing/usage_service.py, backend/app/core/access_logging.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

OPERATIONAL MEMORY

FILE REFERENCE:

backend/app/memory/__init__.py

Category: Operational memory | Kind: Python module | Lines: 1 | SHA-256: 97dfb65d41795b88...

Purpose

This Python module contributes to the operational memory subsystem. It defines focused behavior around its declared configuration and content structure and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Init . Its extracted size is 1 lines and 5 lexical tokens; the source digest begins 97dfb65d4179.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/tools.py, backend/app/api/middleware.py, backend/app/billing/billing_service.py. The containing category is Operational memory.

Simplified pseudocode

```
READ backend/app/memory/__init__.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/memory/__init__.py is a Python module that owns a focused part of operational memory. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

OPERATIONAL MEMORY

FILE REFERENCE:

backend/app/memory/freshness.py

Category: Operational memory | Kind: Python module | Lines: 394 | SHA-256: 54c592608f56c891...

Purpose

This Python module contributes to the operational memory subsystem. It defines focused behavior around MemoryFreshnessResult, _cache_key, MemoryFreshnessService and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Freshness. Its extracted size is 394 lines and 1197 lexical tokens; the source digest begins 54c592608f56.

Important symbols

Important exported or declared symbols/sections identified by static inspection: MemoryFreshnessResult, _cache_key, MemoryFreshnessService. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/git/change_provider.py, backend/app/memory/models.py, backend/app/utils/datetime.py, backend/app/db/mongo.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/api/routes/memory.py, backend/app/memory/memory_service.py, backend/app/memory/models.py. The containing category is Operational memory.

Simplified pseudocode

```
READ backend/app/memory/freshness.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/memory/freshness.py is a Python module that owns a focused part of operational memory. Related files: backend/app/git/change_provider.py, backend/app/memory/models.py, backend/app/utils/datetime.py, backend/app/db/mongo.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

OPERATIONAL MEMORY

FILE REFERENCE:

backend/app/memory/memory_service.py

Category: Operational memory | Kind: Python module | Lines: 1480 | SHA-256: 99a48ef918330a6a...

Purpose

This Python module contributes to the operational memory subsystem. It defines focused behavior around _CONTEXT_CAP_DEFAULT, _MIN_CONFIDENCE_DEFAULT, _HISTORICAL_CAP_DEFAULT and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Memory Service. Its extracted size is 1480 lines and 4306 lexical tokens; the source digest begins 99a48ef91833.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _CONTEXT_CAP_DEFAULT, _MIN_CONFIDENCE_DEFAULT, _HISTORICAL_CAP_DEFAULT, _applicability_matches, _affected_paths_match, _to_document, _freshness_from_status. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/db/mongo.py, backend/app/memory/models.py, backend/app/services/redaction_service.py, backend/app/utils/datetime.py, backend/app/utils/ids.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/api/routes/memory.py, backend/app/services/incident_service.py, backend/tests/test_memory_freshness.py. The containing category is Operational memory.

Simplified pseudocode

```
READ backend/app/memory/memory_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/memory/memory_service.py is a Python module that owns a focused part of operational memory. Related files: backend/app/db/mongo.py, backend/app/memory/models.py, backend/app/services/redaction_service.py, backend/app/utils/datetime.py, backend/app/utils/ids.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

OPERATIONAL MEMORY

FILE REFERENCE:

backend/app/memory/models.py

Category: Operational memory | Kind: Python module | Lines: 376 | SHA-256: 6723a99da30c2e7a...

Purpose

This Python module contributes to the operational memory subsystem. It defines focused behavior around OperationalMemoryKind, MemoryProvenance, MemoryApplicability and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Models. Its extracted size is 376 lines and 1302 lexical tokens; the source digest begins 6723a99da30c.

Important symbols

Important exported or declared symbols/sections identified by static inspection: OperationalMemoryKind, MemoryProvenance, MemoryApplicability, OperationalMemoryEntry, MemoryMatch, MemorySummary, MemoryEntryCandidate. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/app/agent/graph.py. The containing category is Operational memory.

Simplified pseudocode

```
READ backend/app/memory/models.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/memory/models.py is a Python module that owns a focused part of operational memory. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/observability/__init__.py

Category: Backend platform | Kind: Python module | Lines: 3 | SHA-256: 537ce3e227c63715...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around its declared configuration and content structure and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Init . Its extracted size is 3 lines and 8 lexical tokens; the source digest begins 537ce3e227c6.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/observability/recorder.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/tools.py, backend/app/api/middleware.py, backend/app/billing/billing_service.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/observability/__init__.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/observability/__init__.py is a Python module that owns a focused part of backend platform. Related files: backend/app/observability/recorder.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/observability/models.py

Category: Backend platform | Kind: Python module | Lines: 35 | SHA-256: adbab9d1b97daec6...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around RunTrace, TraceSpan and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Models. Its extracted size is 35 lines and 117 lexical tokens; the source digest begins adbab9d1b97d.

Important symbols

Important exported or declared symbols/sections identified by static inspection: RunTrace, TraceSpan. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/app/agent/graph.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/observability/models.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/observability/models.py is a Python module that owns a focused part of backend platform. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/observability/recorder.py

Category: Backend platform | Kind: Python module | Lines: 95 | SHA-256: a906ef0ac4c9966b...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around TraceExporter, LocalTraceExporter, MongoTraceExporter and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Recorder. Its extracted size is 95 lines and 360 lexical tokens; the source digest begins a906ef0ac4c9.

Important symbols

Important exported or declared symbols/sections identified by static inspection: TraceExporter, LocalTraceExporter, MongoTraceExporter, TraceRecorder. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/db/mongo.py, backend/app/observability/models.py, backend/app/services/redaction_service.py, backend/app/utils/datetime.py, backend/app/utils/ids.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/observability/__init__.py, backend/app/services/session_service.py, backend/tests/test_evals_and_tracing.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/observability/recorder.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/observability/recorder.py is a Python module that owns a focused part of backend platform. Related files: backend/app/db/mongo.py, backend/app/observability/models.py, backend/app/services/redaction_service.py, backend/app/utils/datetime.py, backend/app/utils/ids.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/policy/__init__.py

Category: Backend platform | Kind: Python module | Lines: 5 | SHA-256: 8118046a399ebf76...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around its declared configuration and content structure and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Init . Its extracted size is 5 lines and 24 lexical tokens; the source digest begins 8118046a399e.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/policy/evaluator.py, backend/app/policy/models.py, backend/app/policy/registry.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/tools.py, backend/app/api/middleware.py, backend/app/billing/billing_service.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/policy/__init__.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/policy/__init__.py is a Python module that owns a focused part of backend platform. Related files: backend/app/policy/evaluator.py, backend/app/policy/models.py, backend/app/policy/registry.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/policy/evaluator.py

Category: Backend platform | Kind: Python module | Lines: 5 | SHA-256: 3af3837a40a1d7b7...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around requires_approval and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Evaluator. Its extracted size is 5 lines and 15 lexical tokens; the source digest begins 3af3837a40a1.

Important symbols

Important exported or declared symbols/sections identified by static inspection: requires_approval. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/policy/registry.py. Potential consumers found through bounded textual/import analysis: backend/app/policy/__init__.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/policy/evaluator.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/policy/evaluator.py is a Python module that owns a focused part of backend platform. Related files: backend/app/policy/registry.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/policy/models.py

Category: Backend platform | Kind: Python module | Lines: 20 | SHA-256: 1977bebd9a18f33b...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around ToolRisk, ToolPolicy and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Models. Its extracted size is 20 lines and 46 lexical tokens; the source digest begins 1977bebd9a18.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ToolRisk, ToolPolicy. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/app/agent/graph.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/policy/models.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/policy/models.py is a Python module that owns a focused part of backend platform. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/policy/registry.py

Category: Backend platform | Kind: Python module | Lines: 30 | SHA-256: 1b4ce9d5e2d192cd...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around get_policy and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Registry. Its extracted size is 30 lines and 177 lexical tokens; the source digest begins 1b4ce9d5e2d1.

Important symbols

Important exported or declared symbols/sections identified by static inspection: get_policy. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/policy/models.py. Potential consumers found through bounded textual/import analysis: backend/app/execution/runtime.py, backend/app/policy/__init__.py, backend/app/policy/evaluator.py, backend/app/services/candidate_verification_service.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/policy/registry.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/policy/registry.py is a Python module that owns a focused part of backend platform. Related files: backend/app/policy/models.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND DOMAIN CONTRACTS

FILE REFERENCE:

backend/app/schemas/assistant.py

Category: Backend domain contracts | Kind: Python module | Lines: 19 | SHA-256: 2319ddb74a897202...

Purpose

This Python module contributes to the backend domain contracts subsystem. It defines focused behavior around AssistantChatRequest, AssistantChatResponse and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Assistant. Its extracted size is 19 lines and 110 lexical tokens; the source digest begins 2319ddb74a89.

Important symbols

Important exported or declared symbols/sections identified by static inspection: AssistantChatRequest, AssistantChatResponse. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/api/middleware.py, backend/app/api/routes/assistant.py, backend/app/main.py. The containing category is Backend domain contracts.

Simplified pseudocode

```
READ backend/app/schemas/assistant.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/schemas/assistant.py is a Python module that owns a focused part of backend domain contracts. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND DOMAIN CONTRACTS

FILE REFERENCE:

backend/app/schemas/auth.py

Category: Backend domain contracts | Kind: Python module | Lines: 29 | SHA-256: 94ad49bc45d7b7b6...

Purpose

This Python module contributes to the backend domain contracts subsystem. It defines focused behavior around RegisterRequest, LoginRequest, UserResponse and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Auth. Its extracted size is 29 lines and 68 lexical tokens; the source digest begins 94ad49bc45d7.

Important symbols

Important exported or declared symbols/sections identified by static inspection: RegisterRequest, LoginRequest, UserResponse, AuthResponse. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, .github/workflows/ci.yml, README.md. The containing category is Backend domain contracts.

Simplified pseudocode

```
READ backend/app/schemas/auth.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/schemas/auth.py is a Python module that owns a focused part of backend domain contracts. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND DOMAIN CONTRACTS

FILE REFERENCE:

backend/app/schemas/billing.py

Category: Backend domain contracts | Kind: Python module | Lines: 18 | SHA-256: a1e107fd555eb828...

Purpose

This Python module contributes to the backend domain contracts subsystem. It defines focused behavior around CreateCheckoutRequest, CheckoutResponse, EntitlementResponse and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Billing. Its extracted size is 18 lines and 31 lexical tokens; the source digest begins a1e107fd555e.

Important symbols

Important exported or declared symbols/sections identified by static inspection: CreateCheckoutRequest, CheckoutResponse, EntitlementResponse. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/billing/models.py. Potential consumers found through bounded textual/import analysis: backend/.env.example, backend/app/agent/graph.py, backend/app/api/routes/assistant.py, backend/app/api/routes/billing.py. The containing category is Backend domain contracts.

Simplified pseudocode

```
READ backend/app/schemas/billing.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/schemas/billing.py is a Python module that owns a focused part of backend domain contracts. Related files: backend/app/billing/models.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND DOMAIN CONTRACTS

FILE REFERENCE:

backend/app/schemas/delivery.py

Category: Backend domain contracts | Kind: Python module | Lines: 82 | SHA-256: dbd3e43b09c9c903...

Purpose

This Python module contributes to the backend domain contracts subsystem. It defines focused behavior around ProposedFileChange, ValidationStep, ValidationResult and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Delivery. Its extracted size is 82 lines and 225 lexical tokens; the source digest begins dbd3e43b09c9.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ProposedFileChange, ValidationStep, ValidationResult, DeliveryPlan, ApprovalRecord, DeliveryResult. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: README.md, backend/.env.example, backend/app/agent/graph.py, backend/app/agent/state.py. The containing category is Backend domain contracts.

Simplified pseudocode

```
READ backend/app/schemas/delivery.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/schemas/delivery.py is a Python module that owns a focused part of backend domain contracts. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND DOMAIN CONTRACTS

FILE REFERENCE:

backend/app/schemas/incident.py

Category: Backend domain contracts | Kind: Python module | Lines: 106 | SHA-256: d58c3318cff210f0...

Purpose

This Python module contributes to the backend domain contracts subsystem. It defines focused behavior around IncidentStatus, EvidenceStrength, IncidentDiagnosis and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Incident. Its extracted size is 106 lines and 325 lexical tokens; the source digest begins d58c3318cff2.

Important symbols

Important exported or declared symbols/sections identified by static inspection: IncidentStatus, EvidenceStrength, IncidentDiagnosis, RepositoryAutomationSettingsRequest, RepositoryAutomationSettingsResponse, IncidentFeedbackRequest, FlakyTestObservationSummary. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: README.md, backend/app/agent/graph.py, backend/app/api/routes/github.py, backend/app/api/routes/incidents.py. The containing category is Backend domain contracts.

Simplified pseudocode

```
READ backend/app/schemas/incident.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/schemas/incident.py is a Python module that owns a focused part of backend domain contracts. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND DOMAIN CONTRACTS

FILE REFERENCE:

backend/app/schemas/intelligence.py

Category: Backend domain contracts | Kind: Python module | Lines: 120 | SHA-256: 70b265a33c2b89b0...

Purpose

This Python module contributes to the backend domain contracts subsystem. It defines focused behavior around GraphNodePublic, GraphEdgePublic, GraphSnapshotStatus and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Intelligence. Its extracted size is 120 lines and 293 lexical tokens; the source digest begins 70b265a33c2b.

Important symbols

Important exported or declared symbols/sections identified by static inspection: GraphNodePublic, GraphEdgePublic, GraphSnapshotStatus, GraphNodeDetailResponse. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/intelligence/graph/models.py, backend/app/intelligence/impact/models.py. Potential consumers found through bounded textual/import analysis: README.md, backend/app/api/routes/session.py, backend/app/intelligence/__init__.py, backend/app/intelligence/graph/__init__.py. The containing category is Backend domain contracts.

Simplified pseudocode

```
READ backend/app/schemas/intelligence.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/schemas/intelligence.py is a Python module that owns a focused part of backend domain contracts. Related files: backend/app/intelligence/graph/models.py, backend/app/intelligence/impact/models.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND DOMAIN CONTRACTS

FILE REFERENCE:

backend/app/schemas/investigation.py

Category: Backend domain contracts | Kind: Python module | Lines: 71 | SHA-256: 6aa15107b4656520...

Purpose

This Python module contributes to the backend domain contracts subsystem. It defines focused behavior around EvidenceItem, Hypothesis, ActionProposal and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Investigation. Its extracted size is 71 lines and 197 lexical tokens; the source digest begins 6aa15107b465.

Important symbols

Important exported or declared symbols/sections identified by static inspection: EvidenceItem, Hypothesis, ActionProposal, InvestigationResult. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: README.md, backend/.env.example, backend/app/agent/graph.py, backend/app/agent/prompts.py. The containing category is Backend domain contracts.

Simplified pseudocode

```
READ backend/app/schemas/investigation.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/schemas/investigation.py is a Python module that owns a focused part of backend domain contracts. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND DOMAIN CONTRACTS

FILE REFERENCE:

backend/app/schemas/patch.py

Category: Backend domain contracts | Kind: Python module | Lines: 33 | SHA-256: 2ec73457ffbc9c94...

Purpose

This Python module contributes to the backend domain contracts subsystem. It defines focused behavior around FileEditIntent, PatchProposal, ChangeSurface and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Patch. Its extracted size is 33 lines and 87 lexical tokens; the source digest begins 2ec73457ffbc.

Important symbols

Important exported or declared symbols/sections identified by static inspection: FileEditIntent, PatchProposal, ChangeSurface, RepositoryCapability. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .github/workflows/live-evals.yml, README.md, backend/app/agent/graph.py, backend/app/agent/policy.py. The containing category is Backend domain contracts.

Simplified pseudocode

```
READ backend/app/schemas/patch.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/schemas/patch.py is a Python module that owns a focused part of backend domain contracts. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND DOMAIN CONTRACTS

FILE REFERENCE:

backend/app/schemas/replay.py

Category: Backend domain contracts | Kind: Python module | Lines: 250 | SHA-256: 0b967b6f488830ba...

Purpose

This Python module contributes to the backend domain contracts subsystem. It defines focused behavior around ReplayPublicModel, ReplaySession, ReplayRepository and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Replay. Its extracted size is 250 lines and 883 lexical tokens; the source digest begins 0b967b6f4888.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ReplayPublicModel, ReplaySession, ReplayRepository, ReplayInvestigation, ReplayTimelineItem, ReplayToolResult, ReplayTrace. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: README.md, backend/.env.example, backend/app/api/routes/dev.py, backend/app/api/routes/session.py. The containing category is Backend domain contracts.

Simplified pseudocode

```
READ backend/app/schemas/replay.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/schemas/replay.py is a Python module that owns a focused part of backend domain contracts. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND DOMAIN CONTRACTS

FILE REFERENCE:

backend/app/schemas/session.py

Category: Backend domain contracts | Kind: Python module | Lines: 122 | SHA-256: e2d46bc0817f3eb3...

Purpose

This Python module contributes to the backend domain contracts subsystem. It defines focused behavior around ChatMessage, SessionChatRequest, RepositoryIndexRequest and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Session. Its extracted size is 122 lines and 528 lexical tokens; the source digest begins e2d46bc0817f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ChatMessage, SessionChatRequest, RepositoryIndexRequest, CodeEditProposalRequest, CommitMessageUpdateRequest, ApprovalDecisionRequest, CreatePullRequestRequest. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/.env.example, backend/app/agent/graph.py. The containing category is Backend domain contracts.

Simplified pseudocode

```
READ backend/app/schemas/session.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/schemas/session.py is a Python module that owns a focused part of backend domain contracts. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/auth_service.py

Category: Backend services | Kind: Python module | Lines: 44 | SHA-256: 8ce2ed80da501d01...

Purpose

This service coordinates the Auth Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Auth Service. Its extracted size is 44 lines and 191 lexical tokens; the source digest begins 8ce2ed80da50.

Important symbols

Important exported or declared symbols/sections identified by static inspection: register_user, login_user, get_me. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/security.py, backend/app/db/mongo.py, backend/app/schemas/auth.py, backend/app/services/serializers.py, backend/app/utils/datetime.py. Potential consumers found through bounded textual/import analysis: docs/IMPLEMENTATION_AUDIT.md, docs/IMPLEMENTATION_GAP_ANALYSIS.md, scripts/generate_engineering_book.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/auth_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/auth_service.py is a Python module that owns a focused part of backend services. Related files: backend/app/core/security.py, backend/app/db/mongo.py, backend/app/schemas/auth.py, backend/app/services/serializers.py, backend/app/utils/datetime.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/background_job_service.py

Category: Backend services | Kind: Python module | Lines: 204 | SHA-256: a833973e27d8bc8b...

Purpose

This service coordinates the Background Job Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Background Job Service. Its extracted size is 204 lines and 745 lexical tokens; the source digest begins a833973e27d8.

Important symbols

Important exported or declared symbols/sections identified by static inspection: BackgroundJobService. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py, backend/app/db/mongo.py, backend/app/utils/datetime.py, backend/app/utils/ids.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/main.py, backend/app/services/incident_service.py, backend/app/services/session_service.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/background_job_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/background_job_service.py is a Python module that owns a focused part of backend services. Related files: backend/app/core/config.py, backend/app/db/mongo.py, backend/app/utils/datetime.py, backend/app/utils/ids.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/candidate_verification_service.py

Category: Backend services | Kind: Python module | Lines: 62 | SHA-256: 5f0313f20d612bb5...

Purpose

This service coordinates the Candidate Verification Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Candidate Verification Service. Its extracted size is 62 lines and 259 lexical tokens; the source digest begins 5f0313f20d61.

Important symbols

Important exported or declared symbols/sections identified by static inspection: initial_candidate_verification. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/schemas/delivery.py, backend/app/utils/datetime.py. Potential consumers found through bounded textual/import analysis: backend/app/services/delivery_service.py, backend/tests/test_candidate_verification.py, scripts/generate_engineering_book.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/candidate_verification_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/candidate_verification_service.py is a Python module that owns a focused part of backend services. Related files: backend/app/schemas/delivery.py, backend/app/utils/datetime.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/ci_applicability.py

Category: Backend services | Kind: Python module | Lines: 79 | SHA-256: 4ee63805e34a13f6...

Purpose

This service coordinates the Ci Applicability domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Ci Applicability. Its extracted size is 79 lines and 235 lexical tokens; the source digest begins 4ee63805e34a.

Important symbols

Important exported or declared symbols/sections identified by static inspection: CIFailureApplicability, CIApplicabilityResult, _WORKING_DIRECTORY, workflow_path_applicability, assess_workflow_path. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/state.py, backend/app/services/ci_investigation_service.py, backend/evals/models.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/ci_applicability.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/ci_applicability.py is a Python module that owns a focused part of backend services. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/ci_investigation_service.py

Category: Backend services | Kind: Python module | Lines: 676 | SHA-256: b721700254d1eb06...

Purpose

This service coordinates the Ci Investigation Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Ci Investigation Service. Its extracted size is 676 lines and 2428 lexical tokens; the source digest begins b721700254d1.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _PATH_HINT, _FAILED_CONCLUSIONS, CIInvestigationService, compare_workflow_history, compare_workflow_versions, _same_workflow, _history_excerpt. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/git/actions.py, backend/app/git/change_provider.py, backend/app/services/ci_applicability.py, backend/app/services/evidence_service.py, backend/app/services/patch_engine.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/services/incident_service.py, backend/app/services/session_service.py, backend/evals/runner.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/ci_investigation_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/ci_investigation_service.py is a Python module that owns a focused part of backend services. Related files: backend/app/git/actions.py, backend/app/git/change_provider.py, backend/app/services/ci_applicability.py, backend/app/services/evidence_service.py, backend/app/services/patch_engine.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/ci_request_intent.py

Category: Backend services | Kind: Python module | Lines: 36 | SHA-256: 0468c2778bdcac11...

Purpose

This service coordinates the Ci Request Intent domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Ci Request Intent. Its extracted size is 36 lines and 172 lexical tokens; the source digest begins 0468c2778bdc.

Important symbols

Important exported or declared symbols/sections identified by static inspection: CIRequestIntent, _FAILURE_WORDS, _CI_WORDS, _RUN, _PR, extract_ci_request_intent. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/state.py, backend/app/services/ci_run_resolution_service.py, backend/app/services/session_service.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/ci_request_intent.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/ci_request_intent.py is a Python module that owns a focused part of backend services. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/ci_run_resolution_service.py

Category: Backend services | Kind: Python module | Lines: 75 | SHA-256: 9a03c2171e212228...

Purpose

This service coordinates the Ci Run Resolution Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Ci Run Resolution Service. Its extracted size is 75 lines and 292 lexical tokens; the source digest begins 9a03c2171e21.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ResolvedCIRun, CIRunResolutionService. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/git/actions.py, backend/app/services/ci_request_intent.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/services/session_service.py, backend/tests/test_ci_run_resolution.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/ci_run_resolution_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/ci_run_resolution_service.py is a Python module that owns a focused part of backend services. Related files: backend/app/git/actions.py, backend/app/services/ci_request_intent.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/context_budget.py

Category: Backend services | Kind: Python module | Lines: 144 | SHA-256: 58ef4c891df91451...

Purpose

This service coordinates the Context Budget domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Context Budget. Its extracted size is 144 lines and 613 lexical tokens; the source digest begins 58ef4c891df9.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ContextBudget, PatchCandidateBudget, _truncate, build_bounded_answer_context, build_bounded_patch_candidates. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/tests/test_context_budget.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/context_budget.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/context_budget.py is a Python module that owns a focused part of backend services. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/delivery_ci_service.py

Category: Backend services | Kind: Python module | Lines: 39 | SHA-256: 9a76cbde7b45fc79...

Purpose

This service coordinates the Delivery Ci Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Delivery Ci Service. Its extracted size is 39 lines and 226 lexical tokens; the source digest begins 9a76cbde7b45.

Important symbols

Important exported or declared symbols/sections identified by static inspection: DeliveryCIVerdict, evaluate_delivery_ci. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/git/actions.py. Potential consumers found through bounded textual/import analysis: backend/app/workers/worker.py, backend/tests/test_delivery_ci_service.py, docs/IMPLEMENTATION_GAP_ANALYSIS.md, scripts/generate_engineering_book.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/delivery_ci_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/delivery_ci_service.py is a Python module that owns a focused part of backend services. Related files: backend/app/git/actions.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/delivery_service.py

Category: Backend services | Kind: Python module | Lines: 412 | SHA-256: 3271836807353254...

Purpose

This service coordinates the Delivery Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Delivery Service. Its extracted size is 412 lines and 1749 lexical tokens; the source digest begins 327183680735.

Important symbols

Important exported or declared symbols/sections identified by static inspection: RISK_ORDER, content_hash, canonical_json, DeliveryService. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/db/mongo.py, backend/app/schemas/delivery.py, backend/app/services/github_service.py, backend/app/services/redaction_service.py, backend/app/services/workspace_service.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/execution/policy.py, backend/app/services/patch_engine.py, backend/app/services/session_service.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/delivery_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/delivery_service.py is a Python module that owns a focused part of backend services. Related files: backend/app/db/mongo.py, backend/app/schemas/delivery.py, backend/app/services/github_service.py, backend/app/services/redaction_service.py, backend/app/services/workspace_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/diagnostic_stream_service.py

Category: Backend services | Kind: Python module | Lines: 239 | SHA-256: 6ff5395e6d8542f6...

Purpose

This service coordinates the Diagnostic Stream Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Diagnostic Stream Service. Its extracted size is 239 lines and 1019 lexical tokens; the source digest begins 6ff5395e6d85.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _MAX_STREAM_LOG_CHARS, _MAX_STREAM_DIFF_CHARS, _MAX_STREAM_SOURCE_CHARS, _MAX_REPOSITORY_PROPOSAL_CONTEXT_CHARS, _analysis, stream_diagnostic, _stream_repository_backed_proposal. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/db/mongo.py, backend/app/git/actions.py, backend/app/schemas/session.py, backend/app/services/redaction_service.py, backend/app/services/session_service.py. Potential consumers found through bounded textual/import analysis: backend/app/api/routes/diagnostic.py, backend/devops/agent.py, backend/tests/test_diagnostic_stream.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/diagnostic_stream_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/diagnostic_stream_service.py is a Python module that owns a focused part of backend services. Related files: backend/app/db/mongo.py, backend/app/git/actions.py, backend/app/schemas/session.py, backend/app/services/redaction_service.py, backend/app/services/session_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/evidence_service.py

Category: Backend services | Kind: Python module | Lines: 96 | SHA-256: c359472ca6ecb0c2...

Purpose

This service coordinates the Evidence Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Evidence Service. Its extracted size is 96 lines and 321 lexical tokens; the source digest begins c359472ca6ec.

Important symbols

Important exported or declared symbols/sections identified by static inspection: EvidenceService. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/db/mongo.py, backend/app/schemas/investigation.py, backend/app/services/redaction_service.py, backend/app/utils/datetime.py, backend/app/utils/ids.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/services/ci_investigation_service.py, backend/app/services/session_service.py, docs/IMPLEMENTATION_GAP_ANALYSIS.md. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/evidence_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/evidence_service.py is a Python module that owns a focused part of backend services. Related files: backend/app/db/mongo.py, backend/app/schemas/investigation.py, backend/app/services/redaction_service.py, backend/app/utils/datetime.py, backend/app/utils/ids.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/flaky_test_service.py

Category: Backend services | Kind: Python module | Lines: 179 | SHA-256: 734ca8d2c243c323...

Purpose

This service coordinates the Flaky Test Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Flaky Test Service. Its extracted size is 179 lines and 572 lexical tokens; the source digest begins 734ca8d2c243.

Important symbols

Important exported or declared symbols/sections identified by static inspection: FlakyTestAssessment, FlakyTestObservation, build_flaky_test_observations, FlakyTestObservationService. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/db/mongo.py, backend/app/utils/datetime.py, backend/app/utils/ids.py. Potential consumers found through bounded textual/import analysis: backend/app/services/incident_service.py, backend/tests/test_flaky_test_service.py, scripts/generate_engineering_book.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/flaky_test_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/flaky_test_service.py is a Python module that owns a focused part of backend services. Related files: backend/app/db/mongo.py, backend/app/utils/datetime.py, backend/app/utils/ids.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/github_service.py

Category: Backend services | Kind: Python module | Lines: 419 | SHA-256: 213c8e7121e3bd91...

Purpose

This service coordinates the Github Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Github Service. Its extracted size is 419 lines and 1694 lexical tokens; the source digest begins 213c8e7121e3.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _GITHUB_REPOSITORY_PART, _ACTIONS_LOG_DOWNLOAD_HOST_SUFFIX, _ACTIONS_LOG_AZURE_HOST, is_safe_actions_log_download_url, external_log_download_headers, normalize_github_repository_url, safe_frontend_redirect. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py, backend/app/core/security.py, backend/app/db/mongo.py, backend/app/utils/datetime.py. Potential consumers found through bounded textual/import analysis: backend/app/api/routes/github.py, backend/app/git/actions.py, backend/app/git/change_provider.py, backend/app/git/pr_status.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/github_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/github_service.py is a Python module that owns a focused part of backend services. Related files: backend/app/core/config.py, backend/app/core/security.py, backend/app/db/mongo.py, backend/app/utils/datetime.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/incident_export_service.py

Category: Backend services | Kind: Python module | Lines: 123 | SHA-256: e0d11be702a2efca...

Purpose

This service coordinates the Incident Export Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Incident Export Service. Its extracted size is 123 lines and 418 lexical tokens; the source digest begins e0d11be702a2.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _INCIDENT_FIELDS, incident_evidence_ids, build_incident_export, incident_export_markdown. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/services/redaction_service.py. Potential consumers found through bounded textual/import analysis: backend/app/api/routes/incidents.py, backend/tests/test_incident_models.py, docs/IMPLEMENTATION_GAP_ANALYSIS.md, scripts/generate_engineering_book.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/incident_export_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/incident_export_service.py is a Python module that owns a focused part of backend services. Related files: backend/app/services/redaction_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/incident_service.py

Category: Backend services | Kind: Python module | Lines: 896 | SHA-256: b51859c7d0e23599...

Purpose

This service coordinates the Incident Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Incident Service. Its extracted size is 896 lines and 3137 lexical tokens; the source digest begins b51859c7d0e2.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _DELIVERY_ID, _BRANCH, _FAILURE_CONCLUSIONS, _GROUP_TIMELINE_MAX_ENTRIES, GitHubWebhookService, build_incident_diagnosis, build_structured_incident_diagnosis. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py, backend/app/db/mongo.py, backend/app/memory/memory_service.py, backend/app/memory/models.py, backend/app/schemas/incident.py. Potential consumers found through bounded textual/import analysis: backend/app/api/routes/github.py, backend/app/api/routes/incidents.py, backend/app/main.py, backend/app/workers/worker.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/incident_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/incident_service.py is a Python module that owns a focused part of backend services. Related files: backend/app/core/config.py, backend/app/db/mongo.py, backend/app/memory/memory_service.py, backend/app/memory/models.py, backend/app/schemas/incident.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/patch_engine.py

Category: Backend services | Kind: Python module | Lines: 337 | SHA-256: ea6d025cd2b0ada7...

Purpose

This service coordinates the Patch Engine domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Patch Engine. Its extracted size is 337 lines and 1363 lexical tokens; the source digest begins ea6d025cd2b0.

Important symbols

Important exported or declared symbols/sections identified by static inspection: PatchSafetyError, RepositoryPathPolicy, PatchEngine. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py, backend/app/schemas/delivery.py, backend/app/schemas/patch.py, backend/app/services/delivery_service.py, backend/app/services/redaction_service.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/services/ci_investigation_service.py, backend/app/services/session_service.py, backend/evals/runner.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/patch_engine.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/patch_engine.py is a Python module that owns a focused part of backend services. Related files: backend/app/core/config.py, backend/app/schemas/delivery.py, backend/app/schemas/patch.py, backend/app/services/delivery_service.py, backend/app/services/redaction_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/product_assistant_service.py

Category: Backend services | Kind: Python module | Lines: 292 | SHA-256: e80a7c39e5dd903e...

Purpose

This service coordinates the Product Assistant Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Product Assistant Service. Its extracted size is 292 lines and 1635 lexical tokens; the source digest begins e80a7c39e5dd.

Important symbols

Important exported or declared symbols/sections identified by static inspection: DEFAULT_QUESTIONS, SUGGESTED_PROMPTS, MAX_GUIDE_HISTORY_CHARS, PRODUCT_GUIDE, SYSTEM_PROMPT, ProductAssistantService. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py, backend/app/db/mongo.py, backend/app/services/redaction_service.py, backend/app/utils/datetime.py, backend/app/utils/ids.py. Potential consumers found through bounded textual/import analysis: backend/app/api/routes/assistant.py, backend/tests/test_product_assistant.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/product_assistant_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/product_assistant_service.py is a Python module that owns a focused part of backend services. Related files: backend/app/core/config.py, backend/app/db/mongo.py, backend/app/services/redaction_service.py, backend/app/utils/datetime.py, backend/app/utils/ids.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/rag_service.py

Category: Backend services | Kind: Python module | Lines: 330 | SHA-256: 58ee57492adf8725...

Purpose

This service coordinates the Rag Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Rag Service. Its extracted size is 330 lines and 1224 lexical tokens; the source digest begins 58ee57492adf.

Important symbols

Important exported or declared symbols/sections identified by static inspection: RagService. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py, backend/app/db/mongo.py, backend/app/services/workspace_service.py, backend/app/utils/chunking.py, backend/app/utils/datetime.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/services/session_service.py, backend/app/workers/worker.py, backend/tests/test_rag_chunking.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/rag_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/rag_service.py is a Python module that owns a focused part of backend services. Related files: backend/app/core/config.py, backend/app/db/mongo.py, backend/app/services/workspace_service.py, backend/app/utils/chunking.py, backend/app/utils/datetime.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/redaction_service.py

Category: Backend services | Kind: Python module | Lines: 30 | SHA-256: 7788640ea18c608a...

Purpose

This service coordinates the Redaction Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Redaction Service. Its extracted size is 30 lines and 171 lexical tokens; the source digest begins 7788640ea18c.

Important symbols

Important exported or declared symbols/sections identified by static inspection: RedactionService. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/api/routes/incidents.py, backend/app/execution/docker_runtime.py, backend/app/execution/local_runtime.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/redaction_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/redaction_service.py is a Python module that owns a focused part of backend services. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/serializers.py

Category: Backend services | Kind: Python module | Lines: 40 | SHA-256: e0811c2f661ec6a0...

Purpose

This service coordinates the Serializers domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Serializers. Its extracted size is 40 lines and 152 lexical tokens; the source digest begins e0811c2f661e.

Important symbols

Important exported or declared symbols/sections identified by static inspection: serialize_doc, public_user. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/api/routes/auth.py, backend/app/services/auth_service.py, backend/app/services/session_service.py, backend/tests/test_run_replay.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/serializers.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/serializers.py is a Python module that owns a focused part of backend services. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/session_service.py

Category: Backend services | Kind: Python module | Lines: 2255 | SHA-256: 1de9d810a8a68f7b...

Purpose

This service coordinates the Session Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Session Service. Its extracted size is 2255 lines and 9314 lexical tokens; the source digest begins 1de9d810a8a6.

Important symbols

Important exported or declared symbols/sections identified by static inspection: CI_REFRESH_INTERVAL, _SAFE_BRANCH, _REPLAY_EVIDENCE_PROJECTION, _MAX_REPLAY_DIFF_BYTES, _dict_items, _string_items, _safe_replay_text. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/agent/graph.py, backend/app/billing/usage_service.py, backend/app/core/config.py, backend/app/db/mongo.py, backend/app/git/actions.py. Potential consumers found through bounded textual/import analysis: backend/app/api/routes/dev.py, backend/app/api/routes/diagnostic.py, backend/app/api/routes/session.py, backend/app/main.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/session_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/session_service.py is a Python module that owns a focused part of backend services. Related files: backend/app/agent/graph.py, backend/app/billing/usage_service.py, backend/app/core/config.py, backend/app/db/mongo.py, backend/app/git/actions.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND SERVICES

FILE REFERENCE:

backend/app/services/workspace_service.py

Category: Backend services | Kind: Python module | Lines: 406 | SHA-256: 78da44c2ecb7c3c0...

Purpose

This service coordinates the Workspace Service domain boundary. It combines typed inputs, persistence or provider access, and safety checks while leaving API authentication and UI presentation to adjacent layers.

Key responsibilities

It exists to provide a coherent Python module boundary for Workspace Service. Its extracted size is 406 lines and 1922 lexical tokens; the source digest begins 78da44c2ecb7.

Important symbols

Important exported or declared symbols/sections identified by static inspection: SAFE_READ_COMMANDS, repo_name_from_url, workspace_path, branch_name_from_prompt, WorkspaceService. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py, backend/app/services/github_service.py. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/tools.py, backend/app/api/routes/memory.py, backend/app/intelligence/graph/service.py. The containing category is Backend services.

Simplified pseudocode

```
READ backend/app/services/workspace_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/services/workspace_service.py is a Python module that owns a focused part of backend services. Related files: backend/app/core/config.py, backend/app/services/github_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/utils/chunking.py

Category: Backend platform | Kind: Python module | Lines: 12 | SHA-256: ecf51d171383beb5...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around chunk_text and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Chunking. Its extracted size is 12 lines and 50 lexical tokens; the source digest begins ecf51d171383.

Important symbols

Important exported or declared symbols/sections identified by static inspection: chunk_text. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/services/rag_service.py, backend/tests/test_rag_chunking.py, docs/IMPLEMENTATION_AUDIT.md. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/utils/chunking.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/utils/chunking.py is a Python module that owns a focused part of backend platform. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/utils/datetime.py

Category: Backend platform | Kind: Python module | Lines: 9 | SHA-256: b61e360db473d74b...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around utc_now, iso_now and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Datetime. Its extracted size is 9 lines and 18 lexical tokens; the source digest begins b61e360db473.

Important symbols

Important exported or declared symbols/sections identified by static inspection: utc_now, iso_now. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/api/deps.py, backend/app/api/routes/incidents.py, backend/app/billing/billing_service.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/utils/datetime.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/utils/datetime.py is a Python module that owns a focused part of backend platform. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/utils/ids.py

Category: Backend platform | Kind: Python module | Lines: 6 | SHA-256: acbf1cdd6b7da9a1...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around new_id and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Ids. Its extracted size is 6 lines and 20 lexical tokens; the source digest begins acbf1cdd6b7d.

Important symbols

Important exported or declared symbols/sections identified by static inspection: new_id. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/.env.example, backend/app/agent/graph.py, backend/app/agent/state.py, backend/app/api/deps.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/utils/ids.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/utils/ids.py is a Python module that owns a focused part of backend platform. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/utils/sse.py

Category: Backend platform | Kind: Python module | Lines: 11 | SHA-256: e8433ba31c14bb3c...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around sse_event, heartbeat and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Sse. Its extracted size is 11 lines and 42 lexical tokens; the source digest begins e8433ba31c14.

Important symbols

Important exported or declared symbols/sections identified by static inspection: sse_event, heartbeat. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, README.md, backend/.env.example, backend/app/agent/graph.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/utils/sse.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/utils/sse.py is a Python module that owns a focused part of backend platform. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/workers/__init__.py

Category: Backend platform | Kind: Python module | Lines: 1 | SHA-256: 595e39e73bcd4084...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around its declared configuration and content structure and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Init . Its extracted size is 1 lines and 7 lexical tokens; the source digest begins 595e39e73bcd.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/tools.py, backend/app/api/middleware.py, backend/app/billing/billing_service.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/workers/__init__.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/workers/__init__.py is a Python module that owns a focused part of backend platform. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND PLATFORM

FILE REFERENCE:

backend/app/workers/worker.py

Category: Backend platform | Kind: Python module | Lines: 262 | SHA-256: 454804451e0a4f3e...

Purpose

This Python module contributes to the backend platform subsystem. It defines focused behavior around DeferredJob, renew_job_lease, run_job and is composed by higher-level application, service, or worker code.

Key responsibilities

It exists to provide a coherent Python module boundary for Worker. Its extracted size is 262 lines and 1026 lexical tokens; the source digest begins 454804451e0a.

Important symbols

Important exported or declared symbols/sections identified by static inspection: DeferredJob, renew_job_lease, run_job, run, main. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py, backend/app/db/mongo.py, backend/app/git/actions.py, backend/app/intelligence/graph/service.py, backend/app/intelligence/graph/store.py. Potential consumers found through bounded textual/import analysis: backend/.env.example, backend/app/agent/graph.py, backend/app/billing/billing_service.py, backend/app/core/agent_run_guard.py. The containing category is Backend platform.

Simplified pseudocode

```
READ backend/app/workers/worker.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The implementation is reviewed in the context of tenant isolation, bounded inputs, centralized redaction, and fail-closed behavior where a durable or authorization-dependent resource is absent.

Brief summary

In one sentence: backend/app/workers/worker.py is a Python module that owns a focused part of backend platform. Related files: backend/app/core/config.py, backend/app/db/mongo.py, backend/app/git/actions.py, backend/app/intelligence/graph/service.py, backend/app/intelligence/graph/store.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

REPOSITORY REFERENCE

FILE REFERENCE:

backend/devops/__init__.py

Category: Repository reference | Kind: Python module | Lines: 1 | SHA-256: 210a295e040df39e...

Purpose

This Python module is part of the repository reference layer. It supplies the repository behavior or operational contract associated with its declared configuration and content structure.

Key responsibilities

It exists to provide a coherent Python module boundary for Init . Its extracted size is 1 lines and 9 lexical tokens; the source digest begins 210a295e040d.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/tools.py, backend/app/api/middleware.py, backend/app/billing/billing_service.py. The containing category is Repository reference.

Simplified pseudocode

```
READ backend/devops/__init__.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: backend/devops/__init__.py is a Python module that owns a focused part of repository reference. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

REPOSITORY REFERENCE

FILE REFERENCE:

backend/devops/agent.py

Category: Repository reference | Kind: Python module | Lines: 18 | SHA-256: ac32b23bb7f60d0e...

Purpose

This Python module is part of the repository reference layer. It supplies the repository behavior or operational contract associated with DiagnosticAgent.

Key responsibilities

It exists to provide a coherent Python module boundary for Agent. Its extracted size is 18 lines and 73 lexical tokens; the source digest begins ac32b23bb7f6.

Important symbols

Important exported or declared symbols/sections identified by static inspection: DiagnosticAgent. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/schemas/session.py, backend/app/services/diagnostic_stream_service.py. Potential consumers found through bounded textual/import analysis: .github/workflows/ci.yml, backend/.env.example, backend/app/__init__.py, backend/app/agent/graph.py. The containing category is Repository reference.

Simplified pseudocode

```
READ backend/devops/agent.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: backend/devops/agent.py is a Python module that owns a focused part of repository reference. Related files: backend/app/schemas/session.py, backend/app/services/diagnostic_stream_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

REPOSITORY REFERENCE

FILE REFERENCE:

backend/devops/server.py

Category: Repository reference | Kind: Python module | Lines: 10 | SHA-256: 2aea6ca0ef031e5b...

Purpose

This Python module is part of the repository reference layer. It supplies the repository behavior or operational contract associated with its declared configuration and content structure.

Key responsibilities

It exists to provide a coherent Python module boundary for Server. Its extracted size is 10 lines and 46 lexical tokens; the source digest begins 2aea6ca0ef03.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/main.py. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/.env.example. The containing category is Repository reference.

Simplified pseudocode

```
READ backend/devops/server.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: backend/devops/server.py is a Python module that owns a focused part of repository reference. Related files: backend/app/main.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

backend/docs/AGENT_SAFETY.md

Category: Architecture documentation | Kind: documentation | Lines: 64 | SHA-256: 0535e58c31ce006b...

Purpose

This documentation file records the repository's current guidance for Agent Safety. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Agent Safety. Its extracted size is 64 lines and 680 lexical tokens; the source digest begins 0535e58c31ce.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Agent Safety, Core Safety Boundaries, Non-Negotiable Invariants, Operational Memory Safety, Model Cost and Availability Bounds, Path Security, Secret Handling. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/IMPLEMENTATION_STATUS.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: backend/docs/AGENT_SAFETY.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

backend/docs/CONTEXT_ENGINEERING.md

Category: Architecture documentation | Kind: documentation | Lines: 135 | SHA-256: ca5b315f63a836a9...

Purpose

This documentation file records the repository's current guidance for Context Engineering. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Context Engineering. Its extracted size is 135 lines and 497 lexical tokens; the source digest begins ca5b315f63a8.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Context Engineering, Context Pack Assembly, Precedence Rule, Evidence vs. Related History, Operational Memory in Context, Retrieval Telemetry, Policy Boundary. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/IMPLEMENTATION_STATUS.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: backend/docs/CONTEXT_ENGINEERING.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

backend/docs/OPERATIONAL_MEMORY.md

Category: Architecture documentation | Kind: documentation | Lines: 250 | SHA-256: f5e31a153e5ff828...

Purpose

This documentation file records the repository's current guidance for Operational Memory. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Operational Memory. Its extracted size is 250 lines and 983 lexical tokens; the source digest begins f5e31a153e5f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Operational Memory, Architectural Principles, Semantic Memory Kinds, Technical Tags (not kinds), Write Policy, Verification Lifecycle, Provenance Model. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/IMPLEMENTATION_STATUS.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: backend/docs/OPERATIONAL_MEMORY.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/__init__.py

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 886bac6f1d23a8fe...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for its declared configuration and content structure without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Init . Its extracted size is 1 lines and 5 lexical tokens; the source digest begins 886bac6f1d23.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/tools.py, backend/app/api/middleware.py, backend/app/billing/billing_service.py. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/__init__.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/__init__.py is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/actions-path.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: e894e7355d705d17...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Actions Path. Its extracted size is 1 lines and 37 lexical tokens; the source digest begins e894e7355d70.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, expected_evidence_paths, expected_tools. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/evals/cases/ci-current-workflow-unavailable.json, backend/evals/cases/ci-historical-fixed.json, backend/evals/cases/ci-historical-workflow.json, backend/evals/cases/ci-huge-log.json. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/actions-path.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/actions-path.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/changed-head.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: aa0b01c648eb4a19...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Changed Head. Its extracted size is 1 lines and 37 lexical tokens; the source digest begins aa0b01c648eb.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, expected_evidence_paths, expected_tools. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/changed-head.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/changed-head.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/ci-current-workflow-unavailable.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 66512b3e3c8f152e...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Ci Current Workflow Unavailable. Its extracted size is 1 lines and 39 lexical tokens; the source digest begins 66512b3e3c8f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, metadata. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/ci-current-workflow-unavailable.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/ci-current-workflow-unavailable.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/ci-historical-fixed.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 3cb5fd826a07abf6...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Ci Historical Fixed. Its extracted size is 1 lines and 40 lexical tokens; the source digest begins 3cb5fd826a07.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, metadata. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/DEMO_SCRIPT.md. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/ci-historical-fixed.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/ci-historical-fixed.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/ci-historical-workflow.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: bbab59fac359389c...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Ci Historical Workflow. Its extracted size is 1 lines and 38 lexical tokens; the source digest begins bbab59fac359.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, metadata. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/ci-historical-workflow.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/ci-historical-workflow.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/ci-huge-log.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: dddd49e14a50a8a2...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Ci Huge Log. Its extracted size is 1 lines and 28 lexical tokens; the source digest begins dddd49e14a50.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, metadata. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/ci-huge-log.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/ci-huge-log.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/ci-logs-unavailable.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: bf4eddafc8678496...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Ci Logs Unavailable. Its extracted size is 1 lines and 30 lexical tokens; the source digest begins bf4eddafc867.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, metadata. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/ci-logs-unavailable.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/ci-logs-unavailable.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/ci-malicious-log.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 808cf29d4ff956f3...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Ci Malicious Log. Its extracted size is 1 lines and 33 lexical tokens; the source digest begins 808cf29d4ff9.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, metadata. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/ci-malicious-log.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/ci-malicious-log.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/ci-missing-dependency.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: d6ee905eb2ff69a3...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Ci Missing Dependency. Its extracted size is 1 lines and 42 lexical tokens; the source digest begins d6ee905eb2ff.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, metadata. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/ci-missing-dependency.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/ci-missing-dependency.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/ci-secret-redaction.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 6dd86d053ffe0765...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Ci Secret Redaction. Its extracted size is 1 lines and 48 lexical tokens; the source digest begins 6dd86d053ffe.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, metadata. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/ci-secret-redaction.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/ci-secret-redaction.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/ci-stale-working-directory.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 6f1e220eb1ae14d6...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Ci Stale Working Directory. Its extracted size is 1 lines and 69 lexical tokens; the source digest begins 6f1e220eb1ae.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, metadata. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/DEMO_SCRIPT.md, docs/FEATURE_PARITY_MATRIX.md. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/ci-stale-working-directory.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/ci-stale-working-directory.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/ci-test-regression.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: e9cfe299c073d231...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Ci Test Regression. Its extracted size is 1 lines and 42 lexical tokens; the source digest begins e9cfe299c073.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, metadata. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/ci-test-regression.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/ci-test-regression.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/ci-unrelated-workflow-change.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 11d86fcd9755055f...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Ci Unrelated Workflow Change. Its extracted size is 1 lines and 45 lexical tokens; the source digest begins 11d86fcd9755.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, metadata. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/ci-unrelated-workflow-change.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/ci-unrelated-workflow-change.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/ci-workflow-command.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 2e99395a55bb67a6...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Ci Workflow Command. Its extracted size is 1 lines and 27 lexical tokens; the source digest begins 2e99395a55bb.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, metadata. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/ci-workflow-command.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/ci-workflow-command.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/compose-reference.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 6fffe7f9f4e5a734...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Compose Reference. Its extracted size is 1 lines and 32 lexical tokens; the source digest begins 6fffe7f9f4e5.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, expected_evidence_paths, expected_tools. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/compose-reference.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/compose-reference.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/compose-typo.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: bb9bec722e15ce4b...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Compose Typo. Its extracted size is 1 lines and 36 lexical tokens; the source digest begins bb9bec722e15.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, expected_evidence_paths, expected_changed_files. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/compose-typo.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/compose-typo.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/docker-binding.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 32b48cafe664e9de...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Docker Binding. Its extracted size is 1 lines and 37 lexical tokens; the source digest begins 32b48cafe664.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, expected_evidence_paths, expected_tools. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/EVALUATION.md. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/docker-binding.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/docker-binding.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/git-status.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 95b7cdc9aee8847e...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Git Status. Its extracted size is 1 lines and 31 lexical tokens; the source digest begins 95b7cdc9aee8.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, expected_evidence_paths, expected_tools. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/api/routes/session.py, backend/tests/test_evals_and_tracing.py, client/src/components/ide-workspace.tsx, client/src/lib/api.ts. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/git-status.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/git-status.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/github-actions-path.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 035a644b54e89a85...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Github Actions Path. Its extracted size is 1 lines and 41 lexical tokens; the source digest begins 035a644b54e8.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, expected_evidence_paths, expected_changed_files. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/github-actions-path.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/github-actions-path.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/insufficient-evidence.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 1f09ccd7669dadbd...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Insufficient Evidence. Its extracted size is 1 lines and 28 lexical tokens; the source digest begins 1f09ccd7669d.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, expected_evidence_paths, expected_tools. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/insufficient-evidence.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/insufficient-evidence.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/memory-recall.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: e17358712c1f64c8...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Memory Recall. Its extracted size is 1 lines and 53 lexical tokens; the source digest begins e17358712c1f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, expected_evidence_paths, expected_changed_files. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/memory-recall.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/memory-recall.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/missing-env.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 2a4dc2c72b81c796...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Missing Env. Its extracted size is 1 lines and 32 lexical tokens; the source digest begins 2a4dc2c72b81.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, expected_evidence_paths, expected_tools. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/missing-env.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/missing-env.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/path-traversal-edit.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 931c4190107f8639...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Path Traversal Edit. Its extracted size is 1 lines and 31 lexical tokens; the source digest begins 931c4190107f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, expected_evidence_paths, should_reject. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/evals/runner.py. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/path-traversal-edit.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/path-traversal-edit.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/prompt-injection.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 3210bc142a4fd4ff...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Prompt Injection. Its extracted size is 1 lines and 33 lexical tokens; the source digest begins 3210bc142a4f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, expected_evidence_paths, expected_tools. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/tests/test_evals_and_tracing.py, docs/LIVE_EVALUATION.md. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/prompt-injection.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/prompt-injection.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/push-request.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 2ce040cda7ba88af...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Push Request. Its extracted size is 1 lines and 30 lexical tokens; the source digest begins 2ce040cda7ba.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, expected_evidence_paths, expected_tools. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/push-request.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/push-request.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/python-bug.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: d5bc67fdd5218fbb...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Python Bug. Its extracted size is 1 lines and 38 lexical tokens; the source digest begins d5bc67fdd521.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, expected_evidence_paths, expected_changed_files. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/python-bug.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/python-bug.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/stale-readme.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 1797a0e1bf792e21...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Stale Readme. Its extracted size is 1 lines and 36 lexical tokens; the source digest begins 1797a0e1bf79.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, expected_evidence_paths, expected_tools. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/stale-readme.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/stale-readme.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/cases/ts-api-mismatch.json

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: c9787b86d5a9f5f9...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for id, name, category without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Ts Api Mismatch. Its extracted size is 1 lines and 37 lexical tokens; the source digest begins c9787b86d5a9.

Important symbols

Important exported or declared symbols/sections identified by static inspection: id, name, category, user_request, repository_fixture, expected_evidence_paths, expected_changed_files. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/cases/ts-api-mismatch.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/cases/ts-api-mismatch.json is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/actions-path/.github/workflows/test.yml

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: a00906d808892c18...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for working-directory without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Test. Its extracted size is 1 lines and 3 lexical tokens; the source digest begins a00906d80889.

Important symbols

Important exported or declared symbols/sections identified by static inspection: working-directory. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .github/workflows/ci.yml, .github/workflows/deployment-readiness.yml, .github/workflows/live-evals.yml. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/actions-path/.github/workflows/test.yml from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/actions-path/.github/workflows/test.yml is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/actions-path/backend/main.py

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 4f4c4ec95d1f4117...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for its declared configuration and content structure without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Main. Its extracted size is 1 lines and 2 lexical tokens; the source digest begins 4f4c4ec95d1f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/taste.md, .github/workflows/ci.yml, .github/workflows/deployment-readiness.yml, README.md. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/actions-path/backend/main.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/actions-path/backend/main.py is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/changed-head/app.py

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: eb42a118c0b1b170...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for HOST without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for App. Its extracted size is 1 lines and 5 lexical tokens; the source digest begins eb42a118c0b1.

Important symbols

Important exported or declared symbols/sections identified by static inspection: HOST. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/Dockerfile. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/changed-head/app.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/changed-head/app.py is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/compose-reference/docker-compose.yml

Category: Deterministic evaluations | Kind: evaluation | Lines: 4 | SHA-256: 88885f446708300e...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for services without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Docker Compose. Its extracted size is 4 lines and 7 lexical tokens; the source digest begins 88885f446708.

Important symbols

Important exported or declared symbols/sections identified by static inspection: services. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/services/patch_engine.py, backend/app/services/workspace_service.py, backend/evals/cases/compose-reference.json. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/compose-reference/docker-compose.yml from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/compose-reference/docker-compose.yml is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/compose-typo/docker-compose.yml

Category: Deterministic evaluations | Kind: evaluation | Lines: 8 | SHA-256: 359d795d651585af...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for version, services without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Docker Compose. Its extracted size is 8 lines and 12 lexical tokens; the source digest begins 359d795d6515.

Important symbols

Important exported or declared symbols/sections identified by static inspection: version, services. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/services/patch_engine.py, backend/app/services/workspace_service.py, backend/evals/cases/compose-reference.json. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/compose-typo/docker-compose.yml from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/compose-typo/docker-compose.yml is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/docker-binding/app.py

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: eb42a118c0b1b170...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for HOST without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for App. Its extracted size is 1 lines and 5 lexical tokens; the source digest begins eb42a118c0b1.

Important symbols

Important exported or declared symbols/sections identified by static inspection: HOST. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/Dockerfile. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/docker-binding/app.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/docker-binding/app.py is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/git-status/app.py

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 8011d775a8b101dc...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for its declared configuration and content structure without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for App. Its extracted size is 1 lines and 3 lexical tokens; the source digest begins 8011d775a8b1.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/Dockerfile. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/git-status/app.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/git-status/app.py is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/github-actions-path/.github/workflows/ci.yml

Category: Deterministic evaluations | Kind: evaluation | Lines: 10 | SHA-256: 9eeb9af97c2bb23d...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for name, on, jobs without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Ci. Its extracted size is 10 lines and 22 lexical tokens; the source digest begins 9eeb9af97c2b.

Important symbols

Important exported or declared symbols/sections identified by static inspection: name, on, jobs. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, .github/workflows/ci.yml, .github/workflows/deployment-readiness.yml. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/github-actions-path/.github/workflows/ci.yml from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/github-actions-path/.github/workflows/ci.yml is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/injection/README.md

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 6e913b9130023500...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for its declared configuration and content structure without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Readme. Its extracted size is 1 lines and 12 lexical tokens; the source digest begins 6e913b913002.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/docs/CONTEXT_ENGINEERING.md, backend/evals/cases/prompt-injection.json, backend/evals/cases/stale-readme.json, backend/tests/test_load_harness_safety.py. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/injection/README.md from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/injection/README.md is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/insufficient/a.py

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 3f83ff3c3cf0e556...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for its declared configuration and content structure without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for A. Its extracted size is 1 lines and 4 lexical tokens; the source digest begins 3f83ff3c3cf0.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, .github/workflows/ci.yml, .github/workflows/deployment-readiness.yml. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/insufficient/a.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/insufficient/a.py is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/memory-recall/api.py

Category: Deterministic evaluations | Kind: evaluation | Lines: 12 | SHA-256: ca38898db20642e8...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for PORT, SERVICE without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Api. Its extracted size is 12 lines and 64 lexical tokens; the source digest begins ca38898db206.

Important symbols

Important exported or declared symbols/sections identified by static inspection: PORT, SERVICE. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/taste.md, backend/.env.example, backend/app/agent/graph.py, backend/app/api/deps.py. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/memory-recall/api.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/memory-recall/api.py is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/memory-recall/docker-compose.yml

Category: Deterministic evaluations | Kind: evaluation | Lines: 6 | SHA-256: 08c1b0167604272a...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for services without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Docker Compose. Its extracted size is 6 lines and 23 lexical tokens; the source digest begins 08c1b0167604.

Important symbols

Important exported or declared symbols/sections identified by static inspection: services. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/services/patch_engine.py, backend/app/services/workspace_service.py, backend/evals/cases/compose-reference.json. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/memory-recall/docker-compose.yml from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/memory-recall/docker-compose.yml is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/missing-env/app.py

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: b4b309136032129c...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for MONGO_URI without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for App. Its extracted size is 1 lines and 2 lexical tokens; the source digest begins b4b309136032.

Important symbols

Important exported or declared symbols/sections identified by static inspection: MONGO_URI. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/Dockerfile. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/missing-env/app.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/missing-env/app.py is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/missing-env/render.yaml

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: a163d50c2e1cc570...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for services without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Render. Its extracted size is 1 lines and 1 lexical tokens; the source digest begins a163d50c2e1c.

Important symbols

Important exported or declared symbols/sections identified by static inspection: services. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .github/workflows/deployment-readiness.yml, backend/app/agent/graph.py, backend/app/core/access_logging.py, backend/app/services/product_assistant_service.py. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/missing-env/render.yaml from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/missing-env/render.yaml is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/path-traversal-edit/app.py

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 5d8c3e5b3f6fd736...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for API_KEY without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for App. Its extracted size is 1 lines and 2 lexical tokens; the source digest begins 5d8c3e5b3f6f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: API_KEY. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/Dockerfile. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/path-traversal-edit/app.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/path-traversal-edit/app.py is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/push-request/app.py

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: d54c2d9f7fbcc627...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for its declared configuration and content structure without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for App. Its extracted size is 1 lines and 4 lexical tokens; the source digest begins d54c2d9f7fbc.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/Dockerfile. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/push-request/app.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/push-request/app.py is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/python-bug/app.py

Category: Deterministic evaluations | Kind: evaluation | Lines: 4 | SHA-256: e2573787f5e275e3...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for check_port without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for App. Its extracted size is 4 lines and 10 lexical tokens; the source digest begins e2573787f5e2.

Important symbols

Important exported or declared symbols/sections identified by static inspection: check_port. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/Dockerfile. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/python-bug/app.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/python-bug/app.py is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/stale-readme/README.md

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 4265f34523bd398d...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for its declared configuration and content structure without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Readme. Its extracted size is 1 lines and 6 lexical tokens; the source digest begins 4265f34523bd.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/docs/CONTEXT_ENGINEERING.md, backend/evals/cases/prompt-injection.json, backend/evals/cases/stale-readme.json, backend/tests/test_load_harness_safety.py. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/stale-readme/README.md from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/stale-readme/README.md is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/stale-readme/backend/main.py

Category: Deterministic evaluations | Kind: evaluation | Lines: 1 | SHA-256: 040c2b679258ec6f...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for PORT without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Main. Its extracted size is 1 lines and 2 lexical tokens; the source digest begins 040c2b679258.

Important symbols

Important exported or declared symbols/sections identified by static inspection: PORT. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/taste.md, .github/workflows/ci.yml, .github/workflows/deployment-readiness.yml, README.md. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/stale-readme/backend/main.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/stale-readme/backend/main.py is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/fixtures/ts-api-mismatch/src/api.ts

Category: Deterministic evaluations | Kind: evaluation | Lines: 4 | SHA-256: 05166bb3fb459a78...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for getSession, res without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Api. Its extracted size is 4 lines and 13 lexical tokens; the source digest begins 05166bb3fb45.

Important symbols

Important exported or declared symbols/sections identified by static inspection: getSession, res. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/taste.md, backend/.env.example, backend/app/agent/graph.py, backend/app/api/deps.py. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/fixtures/ts-api-mismatch/src/api.ts from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/fixtures/ts-api-mismatch/src/api.ts is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/graders.py

Category: Deterministic evaluations | Kind: evaluation | Lines: 23 | SHA-256: 27a23b52ae4d110e...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for evidence_recall, citation_precision, unsupported_claim_count without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Graders. Its extracted size is 23 lines and 115 lexical tokens; the source digest begins 27a23b52ae4d.

Important symbols

Important exported or declared symbols/sections identified by static inspection: evidence_recall, citation_precision, unsupported_claim_count, tool_selection. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/schemas/investigation.py. Potential consumers found through bounded textual/import analysis: backend/evals/runner.py, backend/tests/test_evals_and_tracing.py. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/graders.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/graders.py is a evaluation that owns a focused part of deterministic evaluations. Related files: backend/app/schemas/investigation.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/models.py

Category: Deterministic evaluations | Kind: evaluation | Lines: 73 | SHA-256: 41c35128f006a6cc...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for EvalCase, EvalResult, LiveDeliveryTestResult without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Models. Its extracted size is 73 lines and 263 lexical tokens; the source digest begins 41c35128f006.

Important symbols

Important exported or declared symbols/sections identified by static inspection: EvalCase, EvalResult, LiveDeliveryTestResult, LiveEvalReport. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/app/agent/graph.py. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/models.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/models.py is a evaluation that owns a focused part of deterministic evaluations. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DETERMINISTIC EVALUATIONS

FILE REFERENCE:

backend/evals/runner.py

Category: Deterministic evaluations | Kind: evaluation | Lines: 374 | SHA-256: dd7d6edff4ad5b04...

Purpose

This evaluation artifact defines deterministic inputs or grading behavior for deterministic evaluations. It provides repeatable evidence for ROOT, load_cases, run_case without representing live-provider execution.

Key responsibilities

It exists to provide a coherent evaluation boundary for Runner. Its extracted size is 374 lines and 1499 lexical tokens; the source digest begins dd7d6edff4ad.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ROOT, load_cases, run_case, run_ci_case, run_live_github_test, summarize, main. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/agent/policy.py, backend/app/core/config.py, backend/app/git/actions.py, backend/app/schemas/patch.py, backend/app/services/ci_applicability.py. Potential consumers found through bounded textual/import analysis: .github/workflows/ci.yml, .github/workflows/deployment-readiness.yml, .github/workflows/live-evals.yml, README.md. The containing category is Deterministic evaluations.

Simplified pseudocode

```
READ backend/evals/runner.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/evals/runner.py is a evaluation that owns a focused part of deterministic evaluations. Related files: backend/app/agent/policy.py, backend/app/core/config.py, backend/app/git/actions.py, backend/app/schemas/patch.py, backend/app/services/ci_applicability.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

PROJECT CONFIGURATION

FILE REFERENCE:

backend/pyproject.toml

Category: Project configuration | Kind: configuration | Lines: 19 | SHA-256: aa8f264ff1f6f5ca...

Purpose

This configuration is part of the project configuration layer. It supplies the repository behavior or operational contract associated with its declared configuration and content structure.

Key responsibilities

It exists to provide a coherent configuration boundary for Pyproject. Its extracted size is 19 lines and 58 lexical tokens; the source digest begins aa8f264ff1f6.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/intelligence/graph/builder.py, backend/app/services/ci_investigation_service.py, backend/app/services/delivery_service.py. The containing category is Project configuration.

Simplified pseudocode

```
READ backend/pyproject.toml from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: backend/pyproject.toml is a configuration that owns a focused part of project configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

REPOSITORY REFERENCE

FILE REFERENCE:

backend/requirements.txt

Category: Repository reference | Kind: configuration | Lines: 29 | SHA-256: ba695c9873a4acd5...

Purpose

This configuration is part of the repository reference layer. It supplies the repository behavior or operational contract associated with its declared configuration and content structure.

Key responsibilities

It exists to provide a coherent configuration boundary for Requirements. Its extracted size is 29 lines and 169 lexical tokens; the source digest begins ba695c9873a4.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .github/workflows/ci.yml, .github/workflows/deployment-readiness.yml, .github/workflows/live-evals.yml, README.md. The containing category is Repository reference.

Simplified pseudocode

```
READ backend/requirements.txt from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: backend/requirements.txt is a configuration that owns a focused part of repository reference. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_agent_policy.py

Category: Backend tests | Kind: test | Lines: 21 | SHA-256: 8017493359f1f229...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_read_only_status_is_safe, test_mutating_prompt_requires_approval, test_docker_build_check_is_read_only_diagnostic.

Key responsibilities

It exists to provide a coherent test boundary for Test Agent Policy. Its extracted size is 21 lines and 78 lexical tokens; the source digest begins 8017493359f1.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_read_only_status_is_safe, test_mutating_prompt_requires_approval, test_docker_build_check_is_read_only_diagnostic. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/agent/policy.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_agent_policy.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_agent_policy.py is a test that owns a focused part of backend tests. Related files: backend/app/agent/policy.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_agent_run_guard.py

Category: Backend tests | Kind: test | Lines: 40 | SHA-256: 7c6e6e6d35669aba...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_agent_run_guard_allows_only_one_concurrent_run_for_a_tenant, test_agent_run_guard_isolated_between_tenants, test_in_memory_lease_expires_after_its_ttl.

Key responsibilities

It exists to provide a coherent test boundary for Test Agent Run Guard. Its extracted size is 40 lines and 143 lexical tokens; the source digest begins 7c6e6e6d3566.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_agent_run_guard_allows_only_one_concurrent_run_for_a_tenant, test_agent_run_guard_isolated_between_tenants, test_in_memory_lease_expires_after_its_ttl. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/agent_run_guard.py, backend/app/core/config.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_agent_run_guard.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_agent_run_guard.py is a test that owns a focused part of backend tests. Related files: backend/app/core/agent_run_guard.py, backend/app/core/config.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_assistant_usage_guard.py

Category: Backend tests | Kind: test | Lines: 54 | SHA-256: d7af3aef0b8897de...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_model_backed_guide_request_uses_the_browser_idempotency_key.

Key responsibilities

It exists to provide a coherent test boundary for Test Assistant Usage Guard. Its extracted size is 54 lines and 193 lexical tokens; the source digest begins d7af3aef0b88.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_model_backed_guide_request_uses_the_browser_idempotency_key. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/agent_run_guard.py, backend/app/schemas/assistant.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_assistant_usage_guard.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_assistant_usage_guard.py is a test that owns a focused part of backend tests. Related files: backend/app/core/agent_run_guard.py, backend/app/schemas/assistant.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_auth_failure_safety.py

Category: Backend tests | Kind: test | Lines: 155 | SHA-256: 673bd09ac7d5e901...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_invalid_jwt_does_not_echo_verification_details, test_jwks_outage_is_retryable_and_does_not_echo_provider_detail, test_clerk_token_is_bound_to_the_configured_issuer_and_authorized_party.

Key responsibilities

It exists to provide a coherent test boundary for Test Auth Failure Safety. Its extracted size is 155 lines and 558 lexical tokens; the source digest begins 673bd09ac7d5.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_invalid_jwt_does_not_echo_verification_details, test_jwks_outage_is_retryable_and_does_not_echo_provider_detail, test_clerk_token_is_bound_to_the_configured_issuer_and_authorized_party, test_clerk_token_with_untrusted_authorized_party_is_rejected, test_concurrent_first_login_reuses_the_unique_clerk_identity, test_oversized_bearer_token_is_rejected_before_jwt_processing, test_clerk_subject_must_be_a_bounded_string. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_auth_failure_safety.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_auth_failure_safety.py is a test that owns a focused part of backend tests. Related files: backend/app/core/config.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_background_jobs.py

Category: Backend tests | Kind: test | Lines: 149 | SHA-256: 098d521288a4ac69...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include FakeJobsCollection, test_reindex_jobs_are_deduplicated_and_contain_no_repository_input, FakeSessionsCollection.

Key responsibilities

It exists to provide a coherent test boundary for Test Background Jobs. Its extracted size is 149 lines and 557 lexical tokens; the source digest begins 098d521288a4.

Important symbols

Important exported or declared symbols/sections identified by static inspection: FakeJobsCollection, test_reindex_jobs_are_deduplicated_and_contain_no_repository_input, FakeSessionsCollection, test_queued_reindex_returns_immediately_without_cloning_in_api_process, FakeClaimCollection, test_background_job_claim_uses_a_reclaimable_lease, test_stale_worker_cannot_renew_a_reclaimed_job. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py, backend/app/services/background_job_service.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_background_jobs.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_background_jobs.py is a test that owns a focused part of backend tests. Related files: backend/app/core/config.py, backend/app/services/background_job_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_billing_lifecycle.py

Category: Backend tests | Kind: test | Lines: 125 | SHA-256: 6c911e2546b57410...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_duplicate_checkout_returns_the_existing_hosted_checkout, test_old_subscription_cancellation_cannot_downgrade_newer_provider_subscription, test_stale_provider_event_cannot_downgrade_a_newer_subscription_state.

Key responsibilities

It exists to provide a coherent test boundary for Test Billing Lifecycle. Its extracted size is 125 lines and 396 lexical tokens; the source digest begins 6c911e2546b5.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_duplicate_checkout_returns_the_existing_hosted_checkout, test_old_subscription_cancellation_cannot_downgrade_newer_provider_subscription, test_stale_provider_event_cannot_downgrade_a_newer_subscription_state. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/billing/billing_service.py, backend/app/billing/models.py, backend/app/core/config.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_billing_lifecycle.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_billing_lifecycle.py is a test that owns a focused part of backend tests. Related files: backend/app/billing/billing_service.py, backend/app/billing/models.py, backend/app/core/config.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_candidate_verification.py

Category: Backend tests | Kind: test | Lines: 87 | SHA-256: b3b51de2b6c5376f...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_candidate_verification_is_immutable_and_never_claims_remote_execution, test_static_validation_is_not_presented_as_remote_verification, test_public_candidate_verification_is_allowlisted.

Key responsibilities

It exists to provide a coherent test boundary for Test Candidate Verification. Its extracted size is 87 lines and 249 lexical tokens; the source digest begins b3b51de2b6c5.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_candidate_verification_is_immutable_and_never_claims_remote_execution, test_static_validation_is_not_presented_as_remote_verification, test_public_candidate_verification_is_allowlisted. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/schemas/delivery.py, backend/app/services/candidate_verification_service.py, backend/app/services/delivery_service.py, backend/app/services/session_service.py. Potential consumers found through bounded textual/import analysis: backend/tests/test_system_capabilities.py. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_candidate_verification.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_candidate_verification.py is a test that owns a focused part of backend tests. Related files: backend/app/schemas/delivery.py, backend/app/services/candidate_verification_service.py, backend/app/services/delivery_service.py, backend/app/services/session_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_change_impact.py

Category: Backend tests | Kind: test | Lines: 152 | SHA-256: 65a013ed21c64e0b...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include _snapshot, _plan, test_impact_analysis_finds_reverse_dependents_and_related_tests.

Key responsibilities

It exists to provide a coherent test boundary for Test Change Impact. Its extracted size is 152 lines and 613 lexical tokens; the source digest begins 65a013ed21c6.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _snapshot, _plan, test_impact_analysis_finds_reverse_dependents_and_related_tests, test_impact_analysis_refuses_to_mix_plan_and_graph_revisions, test_impact_analysis_reports_unmapped_files_without_inventing_dependents, _Collection, test_impact_store_is_idempotent_and_tenant_scoped. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/intelligence/graph/builder.py, backend/app/intelligence/impact/models.py, backend/app/intelligence/impact/service.py, backend/app/intelligence/impact/store.py, backend/app/schemas/delivery.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_change_impact.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_change_impact.py is a test that owns a focused part of backend tests. Related files: backend/app/intelligence/graph/builder.py, backend/app/intelligence/impact/models.py, backend/app/intelligence/impact/service.py, backend/app/intelligence/impact/store.py, backend/app/schemas/delivery.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_ci_applicability.py

Category: Backend tests | Kind: test | Lines: 37 | SHA-256: 163821f389d316a4...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_historical_workflow_fix_is_not_current_defect, test_same_workflow_is_still_current, test_unrelated_workflow_edit_does_not_mark_path_failure_fixed.

Key responsibilities

It exists to provide a coherent test boundary for Test Ci Applicability. Its extracted size is 37 lines and 90 lexical tokens; the source digest begins 163821f389d3.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_historical_workflow_fix_is_not_current_defect, test_same_workflow_is_still_current, test_unrelated_workflow_edit_does_not_mark_path_failure_fixed, test_missing_current_state_fails_safe. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/services/ci_applicability.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_ci_applicability.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_ci_applicability.py is a test that owns a focused part of backend tests. Related files: backend/app/services/ci_applicability.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_ci_investigation_service.py

Category: Backend tests | Kind: test | Lines: 385 | SHA-256: bb9c0f17b77a52f1...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include FakeActions, FakeEvidence, FakeWorkspace.

Key responsibilities

It exists to provide a coherent test boundary for Test Ci Investigation Service. Its extracted size is 385 lines and 1160 lexical tokens; the source digest begins bb9c0f17b77a.

Important symbols

Important exported or declared symbols/sections identified by static inspection: FakeActions, FakeEvidence, FakeWorkspace, FakeChanges, test_ci_service_persists_separate_historical_current_and_changed_evidence, test_ci_service_preserves_failed_test_identity_as_untrusted_evidence, test_ci_service_includes_existing_manifest_for_dependency_failures. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/agent/graph.py, backend/app/git/actions.py, backend/app/git/change_provider.py, backend/app/services/ci_investigation_service.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_ci_investigation_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_ci_investigation_service.py is a test that owns a focused part of backend tests. Related files: backend/app/agent/graph.py, backend/app/git/actions.py, backend/app/git/change_provider.py, backend/app/services/ci_investigation_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_ci_request_intent.py

Category: Backend tests | Kind: test | Lines: 11 | SHA-256: a4ccf7b71b77c96d...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_ci_failure_request_is_detected_with_explicit_run, test_workflow_explanation_is_not_a_live_ci_request.

Key responsibilities

It exists to provide a coherent test boundary for Test Ci Request Intent. Its extracted size is 11 lines and 34 lexical tokens; the source digest begins a4ccf7b71b77.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_ci_failure_request_is_detected_with_explicit_run, test_workflow_explanation_is_not_a_live_ci_request. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/services/ci_request_intent.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_ci_request_intent.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_ci_request_intent.py is a test that owns a focused part of backend tests. Related files: backend/app/services/ci_request_intent.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_ci_run_resolution.py

Category: Backend tests | Kind: test | Lines: 98 | SHA-256: 78cbf26f4d8c445e...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include Actions, run, test_explicit_failed_run_is_repository_scoped.

Key responsibilities

It exists to provide a coherent test boundary for Test Ci Run Resolution. Its extracted size is 98 lines and 400 lexical tokens; the source digest begins 78cbf26f4d8c.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Actions, run, test_explicit_failed_run_is_repository_scoped, test_latest_failed_and_branch_filter, test_pr_run_resolution_uses_matching_repository_head_sha, test_expired_github_actions_access_is_a_partial_investigation, test_code_workbench_forces_proposal_not_log_selected_diagnostic. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/agent/graph.py, backend/app/git/actions.py, backend/app/services/ci_request_intent.py, backend/app/services/ci_run_resolution_service.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_ci_run_resolution.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_ci_run_resolution.py is a test that owns a focused part of backend tests. Related files: backend/app/agent/graph.py, backend/app/git/actions.py, backend/app/services/ci_request_intent.py, backend/app/services/ci_run_resolution_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_context_budget.py

Category: Backend tests | Kind: test | Lines: 53 | SHA-256: 00e910f40012fd6f...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_context_budget_keeps_ranked_sources_and_evidence_ids_within_cap, test_context_budget_never_exceeds_source_cap_or_discards_provenance_silently, test_patch_candidate_budget_preserves_paths_and_hashes_within_limit.

Key responsibilities

It exists to provide a coherent test boundary for Test Context Budget. Its extracted size is 53 lines and 174 lexical tokens; the source digest begins 00e910f40012.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_context_budget_keeps_ranked_sources_and_evidence_ids_within_cap, test_context_budget_never_exceeds_source_cap_or_discards_provenance_silently, test_patch_candidate_budget_preserves_paths_and_hashes_within_limit. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/services/context_budget.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_context_budget.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_context_budget.py is a test that owns a focused part of backend tests. Related files: backend/app/services/context_budget.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_delivery_ci_service.py

Category: Backend tests | Kind: test | Lines: 48 | SHA-256: 299a77cd60b58247...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include run, test_delivery_ci_requires_exact_commit_sha_before_reporting_pass, test_delivery_ci_reports_running_and_failed_without_claiming_success.

Key responsibilities

It exists to provide a coherent test boundary for Test Delivery Ci Service. Its extracted size is 48 lines and 151 lexical tokens; the source digest begins 299a77cd60b5.

Important symbols

Important exported or declared symbols/sections identified by static inspection: run, test_delivery_ci_requires_exact_commit_sha_before_reporting_pass, test_delivery_ci_reports_running_and_failed_without_claiming_success, test_delivery_ci_reports_pass_only_for_completed_successful_matching_runs. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/git/actions.py, backend/app/services/delivery_ci_service.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_delivery_ci_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_delivery_ci_service.py is a test that owns a focused part of backend tests. Related files: backend/app/git/actions.py, backend/app/services/delivery_ci_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_delivery_policy.py

Category: Backend tests | Kind: test | Lines: 275 | SHA-256: 265c9afd5cede37b...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_registered_tool_policy_is_deterministic, test_structured_edit_produces_exact_hashes_and_diff, test_dependency_manifest_is_not_misclassified_as_documentation.

Key responsibilities

It exists to provide a coherent test boundary for Test Delivery Policy. Its extracted size is 275 lines and 934 lexical tokens; the source digest begins 265c9afd5ced.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_registered_tool_policy_is_deterministic, test_structured_edit_produces_exact_hashes_and_diff, test_dependency_manifest_is_not_misclassified_as_documentation, test_patch_engine_rejects_path_traversal_and_sensitive_files, test_approval_hash_is_stable_and_changes_with_diff, test_commit_message_is_part_of_the_delivery_approval_binding, test_changed_head_invalidates_approval. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/schemas/delivery.py, backend/app/schemas/patch.py, backend/app/schemas/session.py, backend/app/services/delivery_service.py, backend/app/services/patch_engine.py. Potential consumers found through bounded textual/import analysis: docs/PRODUCTION_READINESS_AUDIT.md. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_delivery_policy.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_delivery_policy.py is a test that owns a focused part of backend tests. Related files: backend/app/schemas/delivery.py, backend/app/schemas/patch.py, backend/app/schemas/session.py, backend/app/services/delivery_service.py, backend/app/services/patch_engine.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_deployment_blueprints.py

Category: Backend tests | Kind: test | Lines: 124 | SHA-256: 1ba2473cafc06a57...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include ROOT, _PRODUCTION_SECRETS, _blueprint.

Key responsibilities

It exists to provide a coherent test boundary for Test Deployment Blueprints. Its extracted size is 124 lines and 566 lexical tokens; the source digest begins 1ba2473cafc0.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ROOT, _PRODUCTION_SECRETS, _blueprint, _resolved_service_environment, test_render_blueprints_use_a_strict_ready_check_and_validate_each_role, test_free_blueprint_is_isolated_and_never_claims_an_always_on_worker, test_scaled_render_blueprint_keeps_long_running_work_off_the_public_api. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_deployment_blueprints.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_deployment_blueprints.py is a test that owns a focused part of backend tests. Related files: backend/app/core/config.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_diagnostic_stream.py

Category: Backend tests | Kind: test | Lines: 213 | SHA-256: c8b992248b5cdce0...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_streamed_diagnostic_redacts_untrusted_log_and_never_mutates, test_diagnostic_payload_accepts_snake_case_transport_fields, test_repository_backed_proposal_maps_only_existing_delivery_plan.

Key responsibilities

It exists to provide a coherent test boundary for Test Diagnostic Stream. Its extracted size is 213 lines and 740 lexical tokens; the source digest begins c8b992248b5c.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_streamed_diagnostic_redacts_untrusted_log_and_never_mutates, test_diagnostic_payload_accepts_snake_case_transport_fields, test_repository_backed_proposal_maps_only_existing_delivery_plan, test_repository_backed_proposal_forwards_the_evidence_selected_file, test_repository_backed_proposal_surfaces_expired_github_recovery, test_repository_backed_proposal_bounds_untrusted_context_before_graph, test_generated_proposal_uses_the_same_quota_and_tenant_lease. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/schemas/session.py, backend/app/services/diagnostic_stream_service.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_diagnostic_stream.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_diagnostic_stream.py is a test that owns a focused part of backend tests. Related files: backend/app/schemas/session.py, backend/app/services/diagnostic_stream_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_engineering_graph.py

Category: Backend tests | Kind: test | Lines: 172 | SHA-256: 9b1fa84cbfdec7af...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include build_snapshot, test_python_ast_graph_extracts_symbols_imports_and_fastapi_routes, test_workflow_commands_are_never_persisted_as_graph_attributes.

Key responsibilities

It exists to provide a coherent test boundary for Test Engineering Graph. Its extracted size is 172 lines and 836 lexical tokens; the source digest begins 9b1fa84cbfde.

Important symbols

Important exported or declared symbols/sections identified by static inspection: build_snapshot, test_python_ast_graph_extracts_symbols_imports_and_fastapi_routes, test_workflow_commands_are_never_persisted_as_graph_attributes, test_same_logical_symbol_gets_distinct_revision_version, test_neighborhood_is_bidirectional_bounded_and_deterministic, test_invalid_python_is_reported_as_a_coverage_limitation_not_an_exception, FakeSnapshotCollection. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/intelligence/graph/builder.py, backend/app/intelligence/graph/models.py, backend/app/intelligence/graph/service.py, backend/app/intelligence/graph/store.py, backend/app/schemas/intelligence.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_engineering_graph.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_engineering_graph.py is a test that owns a focused part of backend tests. Related files: backend/app/intelligence/graph/builder.py, backend/app/intelligence/graph/models.py, backend/app/intelligence/graph/service.py, backend/app/intelligence/graph/store.py, backend/app/schemas/intelligence.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_evals_and_tracing.py

Category: Backend tests | Kind: test | Lines: 44 | SHA-256: 79b231aa9b612ca3...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_eval_case_parsing_and_deterministic_runner, test_evidence_and_tool_graders, test_trace_span_is_nested_safe_and_redacted.

Key responsibilities

It exists to provide a coherent test boundary for Test Evals And Tracing. Its extracted size is 44 lines and 200 lexical tokens; the source digest begins 79b231aa9b61.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_eval_case_parsing_and_deterministic_runner, test_evidence_and_tool_graders, test_trace_span_is_nested_safe_and_redacted, test_failure_retry_taxonomy. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/failures.py, backend/app/observability/recorder.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_evals_and_tracing.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_evals_and_tracing.py is a test that owns a focused part of backend tests. Related files: backend/app/core/failures.py, backend/app/observability/recorder.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_execution_sandbox.py

Category: Backend tests | Kind: test | Lines: 154 | SHA-256: abf055b1ccc6586e...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_direct_docker_sandbox_default_matches_selected_python_runtime, test_environment_sanitization_strips_secrets, test_workspace_integrity_rejects_modified_candidate.

Key responsibilities

It exists to provide a coherent test boundary for Test Execution Sandbox. Its extracted size is 154 lines and 498 lexical tokens; the source digest begins abf055b1ccc6.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_direct_docker_sandbox_default_matches_selected_python_runtime, test_environment_sanitization_strips_secrets, test_workspace_integrity_rejects_modified_candidate, test_workspace_integrity_rejects_sensitive_side_effects, test_runtime_manager_executes_registered_validator, test_runtime_manager_rejects_unregistered_validator, test_disabled_runtime_never_falls_back_to_local_repository_execution. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/execution/docker_runtime.py, backend/app/execution/policy.py, backend/app/execution/runtime.py, backend/app/services/delivery_service.py. Potential consumers found through bounded textual/import analysis: docs/PRODUCTION_READINESS_AUDIT.md. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_execution_sandbox.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_execution_sandbox.py is a test that owns a focused part of backend tests. Related files: backend/app/execution/docker_runtime.py, backend/app/execution/policy.py, backend/app/execution/runtime.py, backend/app/services/delivery_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_flaky_test_service.py

Category: Backend tests | Kind: test | Lines: 142 | SHA-256: 2d4f50f32c24ca4d...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include _context, test_mixed_same_sha_outcomes_produce_a_limited_flaky_observation, test_repeated_failures_and_small_samples_are_distinguished.

Key responsibilities

It exists to provide a coherent test boundary for Test Flaky Test Service. Its extracted size is 142 lines and 410 lexical tokens; the source digest begins 2d4f50f32c24.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _context, test_mixed_same_sha_outcomes_produce_a_limited_flaky_observation, test_repeated_failures_and_small_samples_are_distinguished, test_flaky_observation_list_is_tenant_scoped, test_observation_persistence_keeps_only_bounded_safe_fields. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/services/flaky_test_service.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_flaky_test_service.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_flaky_test_service.py is a test that owns a focused part of backend tests. Related files: backend/app/services/flaky_test_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_frontend_deployment_security.py

Category: Backend tests | Kind: test | Lines: 21 | SHA-256: 5dc76f7d453f9748...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_vercel_configuration_has_browser_security_headers_and_hashed_asset_caching.

Key responsibilities

It exists to provide a coherent test boundary for Test Frontend Deployment Security. Its extracted size is 21 lines and 111 lexical tokens; the source digest begins 5dc76f7d453f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_vercel_configuration_has_browser_security_headers_and_hashed_asset_caching. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_frontend_deployment_security.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_frontend_deployment_security.py is a test that owns a focused part of backend tests. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_github_actions.py

Category: Backend tests | Kind: test | Lines: 52 | SHA-256: 03aafb730c6fa766...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_ci_log_excerpts_are_bounded_redacted_and_categorized, test_ci_log_is_evidence_not_execution_instruction, test_ci_log_prefers_the_specific_failed_test_window_over_generic_progress.

Key responsibilities

It exists to provide a coherent test boundary for Test Github Actions. Its extracted size is 52 lines and 206 lexical tokens; the source digest begins 03aafb730c6f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_ci_log_excerpts_are_bounded_redacted_and_categorized, test_ci_log_is_evidence_not_execution_instruction, test_ci_log_prefers_the_specific_failed_test_window_over_generic_progress. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/git/actions.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_github_actions.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_github_actions.py is a test that owns a focused part of backend tests. Related files: backend/app/git/actions.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_github_repository_boundary.py

Category: Backend tests | Kind: test | Lines: 177 | SHA-256: 7507f0813687538e...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_repository_url_is_canonicalized_without_credentials_or_arbitrary_hosts, test_oauth_redirect_cannot_leave_the_configured_frontend, test_development_oauth_redirect_preserves_the_loopback_origin_that_started_it.

Key responsibilities

It exists to provide a coherent test boundary for Test Github Repository Boundary. Its extracted size is 177 lines and 658 lexical tokens; the source digest begins 7507f0813687.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_repository_url_is_canonicalized_without_credentials_or_arbitrary_hosts, test_oauth_redirect_cannot_leave_the_configured_frontend, test_development_oauth_redirect_preserves_the_loopback_origin_that_started_it, test_production_oauth_redirect_does_not_allow_the_alternate_loopback_origin, test_actions_log_redirect_headers_never_forward_github_oauth_credentials, test_actions_log_redirect_is_limited_to_github_actions_download_hosts, test_repository_resolution_requires_github_api_access. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_github_repository_boundary.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_github_repository_boundary.py is a test that owns a focused part of backend tests. Related files: backend/app/core/config.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_github_webhook_intake.py

Category: Backend tests | Kind: test | Lines: 269 | SHA-256: cb63adbb9823cf62...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include _Cursor, _Events, _Automation.

Key responsibilities

It exists to provide a coherent test boundary for Test Github Webhook Intake. Its extracted size is 269 lines and 836 lexical tokens; the source digest begins cb63adbb9823.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _Cursor, _Events, _Automation, _Incidents, _payload, _signature, webhook_db. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py, backend/app/services/incident_service.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_github_webhook_intake.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_github_webhook_intake.py is a test that owns a focused part of backend tests. Related files: backend/app/core/config.py, backend/app/services/incident_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_graph_indexing_mode.py

Category: Backend tests | Kind: test | Lines: 81 | SHA-256: e471697c678b01f9...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_queued_indexing_never_runs_inside_the_agent_api_path.

Key responsibilities

It exists to provide a coherent test boundary for Test Graph Indexing Mode. Its extracted size is 81 lines and 279 lexical tokens; the source digest begins e471697c678b.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_queued_indexing_never_runs_inside_the_agent_api_path. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/agent/graph.py, backend/app/core/config.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_graph_indexing_mode.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_graph_indexing_mode.py is a test that owns a focused part of backend tests. Related files: backend/app/agent/graph.py, backend/app/core/config.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_http_protection.py

Category: Backend tests | Kind: test | Lines: 398 | SHA-256: fdae78f5f7a7f3bd...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include FERNET_KEY, test_access_logs_redact_oauth_callback_query_values, test_access_log_redaction_keeps_query_free_paths_unchanged.

Key responsibilities

It exists to provide a coherent test boundary for Test Http Protection. Its extracted size is 398 lines and 1235 lexical tokens; the source digest begins fdae78f5f7a7.

Important symbols

Important exported or declared symbols/sections identified by static inspection: FERNET_KEY, test_access_logs_redact_oauth_callback_query_values, test_access_log_redaction_keeps_query_free_paths_unchanged, test_in_memory_rate_limiter_enforces_a_bounded_sliding_window, test_in_memory_rate_limiter_evicts_oldest_unique_keys_at_capacity, test_rate_limit_key_does_not_include_raw_ip_address, test_trusted_client_ip_accepts_only_a_single_valid_edge_address. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/api/middleware.py, backend/app/core/access_logging.py, backend/app/core/config.py, backend/app/core/rate_limit.py, backend/app/schemas/session.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_http_protection.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_http_protection.py is a test that owns a focused part of backend tests. Related files: backend/app/api/middleware.py, backend/app/core/access_logging.py, backend/app/core/config.py, backend/app/core/rate_limit.py, backend/app/schemas/session.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_incident_models.py

Category: Backend tests | Kind: test | Lines: 448 | SHA-256: 67f7d7e9cf97a57d...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include _context, test_incident_diagnosis_never_claims_correlation_is_causation, test_incident_diagnosis_is_limited_when_ci_logs_are_unavailable.

Key responsibilities

It exists to provide a coherent test boundary for Test Incident Models. Its extracted size is 448 lines and 1452 lexical tokens; the source digest begins 67f7d7e9cf97.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _context, test_incident_diagnosis_never_claims_correlation_is_causation, test_incident_diagnosis_is_limited_when_ci_logs_are_unavailable, test_structured_incident_diagnosis_is_evidence_scoped, test_incident_group_key_requires_matching_repository_workflow_commit_and_failures, test_incident_dispatch_retries_an_ungrouped_collected_record, test_flaky_signal_requires_a_test_identity_and_mixed_same_sha_history. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/api/routes/incidents.py, backend/app/git/actions.py, backend/app/schemas/incident.py, backend/app/services/incident_export_service.py, backend/app/services/incident_service.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_incident_models.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_incident_models.py is a test that owns a focused part of backend tests. Related files: backend/app/api/routes/incidents.py, backend/app/git/actions.py, backend/app/schemas/incident.py, backend/app/services/incident_export_service.py, backend/app/services/incident_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_live_eval_truthfulness.py

Category: Backend tests | Kind: test | Lines: 26 | SHA-256: 9b5a4c8e5ea85cf7...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_allowlisted_delivery_scaffold_is_not_reported_as_live_success, test_non_allowlisted_delivery_target_is_aborted, test_live_report_defaults_to_no_external_checks.

Key responsibilities

It exists to provide a coherent test boundary for Test Live Eval Truthfulness. Its extracted size is 26 lines and 89 lexical tokens; the source digest begins 9b5a4c8e5ea8.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_allowlisted_delivery_scaffold_is_not_reported_as_live_success, test_non_allowlisted_delivery_target_is_aborted, test_live_report_defaults_to_no_external_checks. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_live_eval_truthfulness.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_live_eval_truthfulness.py is a test that owns a focused part of backend tests. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_load_harness_safety.py

Category: Backend tests | Kind: test | Lines: 28 | SHA-256: b6097f7159f2b37f...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include ROOT, LOAD_DIR, test_load_harness_only_targets_non_billable_read_surfaces.

Key responsibilities

It exists to provide a coherent test boundary for Test Load Harness Safety. Its extracted size is 28 lines and 121 lexical tokens; the source digest begins b6097f7159f2.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ROOT, LOAD_DIR, test_load_harness_only_targets_non_billable_read_surfaces, test_load_harness_requires_staging_https_and_disposable_authentication. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_load_harness_safety.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_load_harness_safety.py is a test that owns a focused part of backend tests. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_main.py

Category: Backend tests | Kind: test | Lines: 127 | SHA-256: 73c20668429a765e...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_lifespan_keeps_api_available_when_mongo_is_unavailable, test_lifespan_fails_closed_if_required_database_indexes_cannot_be_created, test_unhandled_error_response_does_not_expose_internal_details.

Key responsibilities

It exists to provide a coherent test boundary for Test Main. Its extracted size is 127 lines and 415 lexical tokens; the source digest begins 73c20668429a.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_lifespan_keeps_api_available_when_mongo_is_unavailable, test_lifespan_fails_closed_if_required_database_indexes_cannot_be_created, test_unhandled_error_response_does_not_expose_internal_details, test_database_outage_returns_a_safe_retryable_response, test_health_reports_a_degraded_database_dependency, test_readiness_refuses_customer_traffic_when_database_is_unavailable, test_readiness_accepts_traffic_only_after_database_connection. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/db/mongo.py, backend/app/services/session_service.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_main.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_main.py is a test that owns a focused part of backend tests. Related files: backend/app/db/mongo.py, backend/app/services/session_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_memory_freshness.py

Category: Backend tests | Kind: test | Lines: 166 | SHA-256: ac7fd43859dfda6c...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include _git, git_repo, _evaluate.

Key responsibilities

It exists to provide a coherent test boundary for Test Memory Freshness. Its extracted size is 166 lines and 645 lexical tokens; the source digest begins ac7fd43859df.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _git, git_repo, _evaluate, test_irrelevant_change_keeps_memory_current, test_relevant_change_requires_revalidation, test_deleted_repository_fact_is_invalidated, test_rename_is_retained_as_history. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/git/pr_status.py, backend/app/memory/freshness.py, backend/app/memory/memory_service.py, backend/app/memory/models.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_memory_freshness.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_memory_freshness.py is a test that owns a focused part of backend tests. Related files: backend/app/git/pr_status.py, backend/app/memory/freshness.py, backend/app/memory/memory_service.py, backend/app/memory/models.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_memory_routes.py

Category: Backend tests | Kind: test | Lines: 24 | SHA-256: 2425ff2f4dbb9dd9...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_memory_routes_register_specific_staleness_before_greedy_repository_path, test_memory_list_uses_the_authenticated_internal_user_id.

Key responsibilities

It exists to provide a coherent test boundary for Test Memory Routes. Its extracted size is 24 lines and 82 lexical tokens; the source digest begins 2425ff2f4dbb.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_memory_routes_register_specific_staleness_before_greedy_repository_path, test_memory_list_uses_the_authenticated_internal_user_id. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_memory_routes.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_memory_routes.py is a test that owns a focused part of backend tests. Related files: backend/app/core/config.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_model_usage_events.py

Category: Backend tests | Kind: test | Lines: 23 | SHA-256: 54d40753f05ab52e...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_provider_usage_event_keeps_only_provider_reported_counts, test_provider_usage_event_does_not_invent_missing_usage.

Key responsibilities

It exists to provide a coherent test boundary for Test Model Usage Events. Its extracted size is 23 lines and 54 lexical tokens; the source digest begins 54d40753f05a.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_provider_usage_event_keeps_only_provider_reported_counts, test_provider_usage_event_does_not_invent_missing_usage. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/agent/graph.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_model_usage_events.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_model_usage_events.py is a test that owns a focused part of backend tests. Related files: backend/app/agent/graph.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_mongo_pooling.py

Category: Backend tests | Kind: test | Lines: 40 | SHA-256: 9b66b1493b3b8857...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_mongo_connection_uses_a_bounded_pool.

Key responsibilities

It exists to provide a coherent test boundary for Test Mongo Pooling. Its extracted size is 40 lines and 97 lexical tokens; the source digest begins 9b66b1493b3b.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_mongo_connection_uses_a_bounded_pool. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_mongo_pooling.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_mongo_pooling.py is a test that owns a focused part of backend tests. Related files: backend/app/core/config.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_oauth_state_security.py

Category: Backend tests | Kind: test | Lines: 127 | SHA-256: 6248d2a8c943095f...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_production_fernet_keys_encrypt_natively_and_can_decrypt_a_legacy_token, test_signed_oauth_state_expires_even_when_its_signature_is_valid, test_oauth_connect_persists_only_a_hashed_one_time_challenge.

Key responsibilities

It exists to provide a coherent test boundary for Test Oauth State Security. Its extracted size is 127 lines and 488 lexical tokens; the source digest begins 6248d2a8c943.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_production_fernet_keys_encrypt_natively_and_can_decrypt_a_legacy_token, test_signed_oauth_state_expires_even_when_its_signature_is_valid, test_oauth_connect_persists_only_a_hashed_one_time_challenge, test_oauth_callback_consumes_the_challenge_before_linking_an_account. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_oauth_state_security.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_oauth_state_security.py is a test that owns a focused part of backend tests. Related files: backend/app/core/config.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_operational_memory.py

Category: Backend tests | Kind: test | Lines: 623 | SHA-256: b6d79ae0297b8125...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include _now, _make_entry, TestSchemas.

Key responsibilities

It exists to provide a coherent test boundary for Test Operational Memory. Its extracted size is 623 lines and 1645 lexical tokens; the source digest begins b6d79ae0297b.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _now, _make_entry, TestSchemas, TestProvenanceGate, TestConfidenceThreshold, TestRedaction, TestDeduplication. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/memory/memory_service.py, backend/app/memory/models.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_operational_memory.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_operational_memory.py is a test that owns a focused part of backend tests. Related files: backend/app/memory/memory_service.py, backend/app/memory/models.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_operational_memory_redesign.py

Category: Backend tests | Kind: test | Lines: 1072 | SHA-256: cdc7bd0a25e170ba...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include _now, _candidate, _make_doc.

Key responsibilities

It exists to provide a coherent test boundary for Test Operational Memory Redesign. Its extracted size is 1072 lines and 3150 lexical tokens; the source digest begins cdc7bd0a25e1.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _now, _candidate, _make_doc, _AsyncIter, _cursor_from, TestWritePolicyGate, TestDeliveredVsMerged. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/memory/memory_service.py, backend/app/memory/models.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_operational_memory_redesign.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_operational_memory_redesign.py is a test that owns a focused part of backend tests. Related files: backend/app/memory/memory_service.py, backend/app/memory/models.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_patch_engine.py

Category: Backend tests | Kind: test | Lines: 268 | SHA-256: 835154f52389421c...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_path_policy_rejects_malicious_paths, test_path_policy_allows_env_example, test_patch_engine_rejects_unevidenced_file_edit.

Key responsibilities

It exists to provide a coherent test boundary for Test Patch Engine. Its extracted size is 268 lines and 731 lexical tokens; the source digest begins 835154f52389.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_path_policy_rejects_malicious_paths, test_path_policy_allows_env_example, test_patch_engine_rejects_unevidenced_file_edit, test_patch_engine_rejects_suspicious_command_injection, test_patch_engine_rejects_stale_file_hash, test_patch_engine_rejects_binary_files, test_workflow_change_has_deterministic_high_risk. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/schemas/patch.py, backend/app/services/delivery_service.py, backend/app/services/patch_engine.py. Potential consumers found through bounded textual/import analysis: backend/tests/test_delivery_policy.py. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_patch_engine.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_patch_engine.py is a test that owns a focused part of backend tests. Related files: backend/app/schemas/patch.py, backend/app/services/delivery_service.py, backend/app/services/patch_engine.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_product_assistant.py

Category: Backend tests | Kind: test | Lines: 130 | SHA-256: b1b4a76aac66807a...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_offline_guide_explains_github_setup, test_offline_guide_explains_approval_boundary, test_offline_guide_explains_lexical_rag_baseline.

Key responsibilities

It exists to provide a coherent test boundary for Test Product Assistant. Its extracted size is 130 lines and 411 lexical tokens; the source digest begins b1b4a76aac66.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_offline_guide_explains_github_setup, test_offline_guide_explains_approval_boundary, test_offline_guide_explains_lexical_rag_baseline, test_default_guide_answers_indexing_without_model, test_default_guide_uses_strict_readiness_for_deployment, test_product_guide_redacts_secrets_before_history_persistence, test_default_guide_preserves_approval_boundary. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py, backend/app/schemas/assistant.py, backend/app/services/product_assistant_service.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_product_assistant.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_product_assistant.py is a test that owns a focused part of backend tests. Related files: backend/app/core/config.py, backend/app/schemas/assistant.py, backend/app/services/product_assistant_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_rag_chunking.py

Category: Backend tests | Kind: test | Lines: 217 | SHA-256: ddb52bae7b9fb340...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_chunk_text_empty_input, test_chunk_text_overlaps_long_input, test_rag_rows_always_include_lexical_retrieval_fields.

Key responsibilities

It exists to provide a coherent test boundary for Test Rag Chunking. Its extracted size is 217 lines and 795 lexical tokens; the source digest begins ddb52bae7b9f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_chunk_text_empty_input, test_chunk_text_overlaps_long_input, test_rag_rows_always_include_lexical_retrieval_fields, test_lexical_rag_scores_natural_language_terms_and_has_safe_fallback, test_index_workspace_persists_lexical_chunks_without_vector_search, test_repository_index_uses_aggregate_file_and_byte_limits, test_repository_index_stops_at_the_chunk_ceiling. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/services/rag_service.py, backend/app/services/workspace_service.py, backend/app/utils/chunking.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_rag_chunking.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_rag_chunking.py is a test that owns a focused part of backend tests. Related files: backend/app/services/rag_service.py, backend/app/services/workspace_service.py, backend/app/utils/chunking.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_razorpay_security.py

Category: Backend tests | Kind: test | Lines: 41 | SHA-256: 02eb436f6dd6d9e1...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_razorpay_webhook_signature_requires_exact_raw_body, test_billing_client_selects_provider_plan_only_from_server_config, test_hosted_checkout_url_cannot_be_an_open_redirect.

Key responsibilities

It exists to provide a coherent test boundary for Test Razorpay Security. Its extracted size is 41 lines and 165 lexical tokens; the source digest begins 02eb436f6dd6.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_razorpay_webhook_signature_requires_exact_raw_body, test_billing_client_selects_provider_plan_only_from_server_config, test_hosted_checkout_url_cannot_be_an_open_redirect, test_hosted_checkout_url_accepts_only_https_razorpay_domains. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/billing/models.py, backend/app/billing/razorpay.py, backend/app/core/config.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_razorpay_security.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_razorpay_security.py is a test that owns a focused part of backend tests. Related files: backend/app/billing/models.py, backend/app/billing/razorpay.py, backend/app/core/config.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_redaction_and_investigation.py

Category: Backend tests | Kind: test | Lines: 338 | SHA-256: 7242fe14933829e2...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_redaction_removes_common_credentials_from_evidence, test_redaction_removes_cloud_tokens_and_non_mongo_connection_strings, test_evidence_requires_stable_provenance_fields.

Key responsibilities

It exists to provide a coherent test boundary for Test Redaction And Investigation. Its extracted size is 338 lines and 1047 lexical tokens; the source digest begins 7242fe149338.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_redaction_removes_common_credentials_from_evidence, test_redaction_removes_cloud_tokens_and_non_mongo_connection_strings, test_evidence_requires_stable_provenance_fields, test_empty_rag_can_fall_back_to_bounded_direct_repository_reads, test_session_hydration_returns_only_the_users_latest_command_center, test_session_hydration_uses_the_canonical_pending_delivery_approval, test_request_delivery_approval_reuses_a_plan_bound_pending_record. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/agent/graph.py, backend/app/schemas/investigation.py, backend/app/services/redaction_service.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_redaction_and_investigation.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_redaction_and_investigation.py is a test that owns a focused part of backend tests. Related files: backend/app/agent/graph.py, backend/app/schemas/investigation.py, backend/app/services/redaction_service.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_render_blueprint.py

Category: Backend tests | Kind: test | Lines: 71 | SHA-256: 5bb8404246b26fc9...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include _environment, test_render_workers_stay_on_the_bounded_512_mb_tier_and_keep_oauth_secrets_out_of_worker_roles, test_mvp_blueprint_keeps_the_100_to_500_user_default_to_one_api_service.

Key responsibilities

It exists to provide a coherent test boundary for Test Render Blueprint. Its extracted size is 71 lines and 356 lexical tokens; the source digest begins 5bb8404246b2.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _environment, test_render_workers_stay_on_the_bounded_512_mb_tier_and_keep_oauth_secrets_out_of_worker_roles, test_mvp_blueprint_keeps_the_100_to_500_user_default_to_one_api_service, test_free_blueprint_uses_distinct_free_service_names. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/tests/test_deployment_blueprints.py. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_render_blueprint.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_render_blueprint.py is a test that owns a focused part of backend tests. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_run_replay.py

Category: Backend tests | Kind: test | Lines: 764 | SHA-256: 137abf26e33fcd54...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include _Cursor, _Collection, _ReplayDb.

Key responsibilities

It exists to provide a coherent test boundary for Test Run Replay. Its extracted size is 764 lines and 2696 lexical tokens; the source digest begins 137abf26e33f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: _Cursor, _Collection, _ReplayDb, _replay_test_app, test_run_replay_reconstructs_snapshot_trace_and_delivery_without_source_bytes, test_trace_less_persisted_run_is_replayable_with_an_honest_limitation, test_replay_degrades_a_legacy_trace_without_an_identifier. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/api/deps.py, backend/app/schemas/replay.py, backend/app/services/serializers.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_run_replay.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_run_replay.py is a test that owns a focused part of backend tests. Related files: backend/app/api/deps.py, backend/app/schemas/replay.py, backend/app/services/serializers.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_runtime_requirements.py

Category: Backend tests | Kind: test | Lines: 9 | SHA-256: 9de5557fadcab726...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_clerk_jwks_runtime_dependency_is_pinned.

Key responsibilities

It exists to provide a coherent test boundary for Test Runtime Requirements. Its extracted size is 9 lines and 38 lexical tokens; the source digest begins 9de5557fadca.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_clerk_jwks_runtime_dependency_is_pinned. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/PRODUCTION_READINESS_AUDIT.md. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_runtime_requirements.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_runtime_requirements.py is a test that owns a focused part of backend tests. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_session_uniqueness.py

Category: Backend tests | Kind: test | Lines: 69 | SHA-256: de60eea1d2564cc2...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_concurrent_session_insert_returns_existing_tenant_session, test_session_index_setup_contains_tenant_unique_constraint.

Key responsibilities

It exists to provide a coherent test boundary for Test Session Uniqueness. Its extracted size is 69 lines and 229 lexical tokens; the source digest begins de60eea1d256.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_concurrent_session_insert_returns_existing_tenant_session, test_session_index_setup_contains_tenant_unique_constraint. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/schemas/session.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_session_uniqueness.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_session_uniqueness.py is a test that owns a focused part of backend tests. Related files: backend/app/schemas/session.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_startup_resilience.py

Category: Backend tests | Kind: test | Lines: 23 | SHA-256: 0d28a6f8e7dc7263...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_mongo_recovery_reinstalls_indexes_before_reporting_success.

Key responsibilities

It exists to provide a coherent test boundary for Test Startup Resilience. Its extracted size is 23 lines and 65 lexical tokens; the source digest begins 0d28a6f8e7dc.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_mongo_recovery_reinstalls_indexes_before_reporting_success. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_startup_resilience.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_startup_resilience.py is a test that owns a focused part of backend tests. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_system_capabilities.py

Category: Backend tests | Kind: test | Lines: 53 | SHA-256: e867d6fa85a27f93...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_disabled_execution_is_reported_as_unavailable, test_missing_docker_sandbox_is_not_advertised_as_local_execution, test_restricted_local_execution_is_only_advertised_outside_production.

Key responsibilities

It exists to provide a coherent test boundary for Test System Capabilities. Its extracted size is 53 lines and 186 lexical tokens; the source digest begins e867d6fa85a2.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_disabled_execution_is_reported_as_unavailable, test_missing_docker_sandbox_is_not_advertised_as_local_execution, test_restricted_local_execution_is_only_advertised_outside_production, test_candidate_verification_never_advertises_unconfigured_remote_ci. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/core/config.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_system_capabilities.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_system_capabilities.py is a test that owns a focused part of backend tests. Related files: backend/app/core/config.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_usage_entitlements.py

Category: Backend tests | Kind: test | Lines: 150 | SHA-256: 5cecadcdc0d918b6...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_inactive_paid_subscription_cannot_unlock_paid_agent_quota, test_unknown_or_missing_tier_fails_to_the_free_entitlement, test_agent_run_limits_come_from_server_configuration.

Key responsibilities

It exists to provide a coherent test boundary for Test Usage Entitlements. Its extracted size is 150 lines and 460 lexical tokens; the source digest begins 5cecadcdc0d9.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_inactive_paid_subscription_cannot_unlock_paid_agent_quota, test_unknown_or_missing_tier_fails_to_the_free_entitlement, test_agent_run_limits_come_from_server_configuration, test_period_key_is_utc_month_stable, test_agent_request_idempotency_key_is_hashed_before_persistence, test_provider_usage_normalization_rejects_invalid_or_negative_counts, test_duplicate_agent_request_does_not_consume_a_second_entitlement. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: backend/app/billing/models.py, backend/app/billing/usage_service.py, backend/app/core/config.py. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_usage_entitlements.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_usage_entitlements.py is a test that owns a focused part of backend tests. Related files: backend/app/billing/models.py, backend/app/billing/usage_service.py, backend/app/core/config.py. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

BACKEND TESTS

FILE REFERENCE:

backend/tests/test_worker_shutdown.py

Category: Backend tests | Kind: test | Lines: 36 | SHA-256: 55b450d783babe68...

Purpose

This test module protects the behavior of the backend tests area by arranging controlled inputs, exercising the public or service boundary, and asserting its safety or correctness contract. Its visible symbols include test_worker_drain_does_not_claim_a_new_job_after_shutdown.

Key responsibilities

It exists to provide a coherent test boundary for Test Worker Shutdown. Its extracted size is 36 lines and 101 lexical tokens; the source digest begins 55b450d783ba.

Important symbols

Important exported or declared symbols/sections identified by static inspection: test_worker_drain_does_not_claim_a_new_job_after_shutdown. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Backend tests.

Simplified pseudocode

```
READ backend/tests/test_worker_shutdown.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates when the verification suite exercises this subsystem. The file protects a behavior contract rather than serving an end-user request directly.

Engineering details

The file is a deterministic regression guard. Its result is evidence about checked behavior, not proof that a live provider, deployment, or production integration was exercised.

Brief summary

In one sentence: backend/tests/test_worker_shutdown.py is a test that owns a focused part of backend tests. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

PROJECT CONFIGURATION

FILE REFERENCE:

client/.env.example

Category: Project configuration | Kind: configuration | Lines: 2 | SHA-256: d6e342057c2679d1...

Purpose

This configuration is part of the project configuration layer. It supplies the repository behavior or operational contract associated with its declared configuration and content structure.

Key responsibilities

It exists to provide a coherent configuration boundary for .Env. Its extracted size is 2 lines and 15 lexical tokens; the source digest begins d6e342057c26.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, README.md, backend/README.md. The containing category is Project configuration.

Simplified pseudocode

```
READ client/.env.example from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/.env.example is a configuration that owns a focused part of project configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DEPLOYMENT CONFIGURATION

FILE REFERENCE:

client/Dockerfile

Category: Deployment configuration | Kind: configuration | Lines: 13 | SHA-256: 0f95d287950d2a46...

Purpose

This configuration is part of the deployment configuration layer. It supplies the repository behavior or operational contract associated with its declared configuration and content structure.

Key responsibilities

It exists to provide a coherent configuration boundary for Dockerfile. Its extracted size is 13 lines and 49 lexical tokens; the source digest begins 0f95d287950d.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/execution/docker_runtime.py, backend/app/services/workspace_service.py, backend/tests/test_agent_policy.py, backend/tests/test_redaction_and_investigation.py. The containing category is Deployment configuration.

Simplified pseudocode

```
READ client/Dockerfile from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/Dockerfile is a configuration that owns a focused part of deployment configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

REPOSITORY REFERENCE

FILE REFERENCE:

client/README.md

Category: Repository reference | Kind: documentation | Lines: 73 | SHA-256: 8dc9e0ee2281acb3...

Purpose

This documentation file records the repository's current guidance for Readme. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Readme. Its extracted size is 73 lines and 289 lexical tokens; the source digest begins 8dc9e0ee2281.

Important symbols

Important exported or declared symbols/sections identified by static inspection: React + TypeScript + Vite, React Compiler, Expanding the ESLint configuration. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/docs/CONTEXT_ENGINEERING.md, backend/evals/cases/prompt-injection.json, backend/evals/cases/stale-readme.json, backend/tests/test_load_harness_safety.py. The containing category is Repository reference.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/README.md is a documentation that owns a focused part of repository reference. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

PROJECT CONFIGURATION

FILE REFERENCE:

client/components.json

Category: Project configuration | Kind: JSON configuration | Lines: 27 | SHA-256: 76322fc650a931dd...

Purpose

This JSON configuration is part of the project configuration layer. It supplies the repository behavior or operational contract associated with $schema, style, rsc.

Key responsibilities

It exists to provide a coherent JSON configuration boundary for Components. Its extracted size is 27 lines and 58 lexical tokens; the source digest begins 76322fc650a9.

Important symbols

Important exported or declared symbols/sections identified by static inspection: $schema, style, rsc, tsx, tailwind, iconLibrary, rtl. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/eslint.config.js, client/src/components/ai-elements/chain-of-thought.tsx, client/src/components/ai-elements/commit.tsx, client/src/components/ai-elements/conversation.tsx. The containing category is Project configuration.

Simplified pseudocode

```
READ client/components.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/components.json is a JSON configuration that owns a focused part of project configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

REPOSITORY REFERENCE

FILE REFERENCE:

client/eslint.config.js

Category: Repository reference | Kind: configuration | Lines: 35 | SHA-256: 761b08b1e3c3cdf3...

Purpose

This configuration is part of the repository reference layer. It supplies the repository behavior or operational contract associated with its declared configuration and content structure.

Key responsibilities

It exists to provide a coherent configuration boundary for Eslint.Config. Its extracted size is 35 lines and 95 lexical tokens; the source digest begins 761b08b1e3c3.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/README.md. The containing category is Repository reference.

Simplified pseudocode

```
READ client/eslint.config.js from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/eslint.config.js is a configuration that owns a focused part of repository reference. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

REPOSITORY REFERENCE

FILE REFERENCE:

client/index.html

Category: Repository reference | Kind: configuration | Lines: 19 | SHA-256: 71466d8431bfe3f6...

Purpose

This configuration is part of the repository reference layer. It supplies the repository behavior or operational contract associated with its declared configuration and content structure.

Key responsibilities

It exists to provide a coherent configuration boundary for Index. Its extracted size is 19 lines and 96 lexical tokens; the source digest begins 71466d8431bf.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/.env.example, backend/README.md. The containing category is Repository reference.

Simplified pseudocode

```
READ client/index.html from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/index.html is a configuration that owns a focused part of repository reference. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

REPOSITORY REFERENCE

FILE REFERENCE:

client/nginx.conf

Category: Repository reference | Kind: configuration | Lines: 23 | SHA-256: 2a151ab4e495cb3b...

Purpose

This configuration is part of the repository reference layer. It supplies the repository behavior or operational contract associated with its declared configuration and content structure.

Key responsibilities

It exists to provide a coherent configuration boundary for Nginx. Its extracted size is 23 lines and 151 lexical tokens; the source digest begins 2a151ab4e495.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/Dockerfile. The containing category is Repository reference.

Simplified pseudocode

```
READ client/nginx.conf from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/nginx.conf is a configuration that owns a focused part of repository reference. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

PROJECT CONFIGURATION

FILE REFERENCE:

client/package.json

Category: Project configuration | Kind: JSON configuration | Lines: 75 | SHA-256: 75a324b0d74e3dfa...

Purpose

This JSON configuration is part of the project configuration layer. It supplies the repository behavior or operational contract associated with name, private, version.

Key responsibilities

It exists to provide a coherent JSON configuration boundary for Package. Its extracted size is 75 lines and 315 lexical tokens; the source digest begins 75a324b0d74e.

Important symbols

Important exported or declared symbols/sections identified by static inspection: name, private, version, type, engines, scripts, dependencies. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, .github/workflows/deployment-readiness.yml, README.md. The containing category is Project configuration.

Simplified pseudocode

```
READ client/package.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/package.json is a JSON configuration that owns a focused part of project configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

REPOSITORY REFERENCE

FILE REFERENCE:

client/scripts/check-auth-redirect.mjs

Category: Repository reference | Kind: automation script | Lines: 59 | SHA-256: 3b1ff70a8eb553a9...

Purpose

This automation script is part of the repository reference layer. It supplies the repository behavior or operational contract associated with server, cases, test.

Key responsibilities

It exists to provide a coherent automation script boundary for Check Auth Redirect. Its extracted size is 59 lines and 167 lexical tokens; the source digest begins 3b1ff70a8eb5.

Important symbols

Important exported or declared symbols/sections identified by static inspection: server, cases, test, actual. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/package.json. The containing category is Repository reference.

Simplified pseudocode

```
READ client/scripts/check-auth-redirect.mjs from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/scripts/check-auth-redirect.mjs is a automation script that owns a focused part of repository reference. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

REPOSITORY REFERENCE

FILE REFERENCE:

client/scripts/check-bundle-budget.mjs

Category: Repository reference | Kind: automation script | Lines: 37 | SHA-256: 1f62f2522e445e31...

Purpose

This automation script is part of the repository reference layer. It supplies the repository behavior or operational contract associated with assetsDirectory, maximumChunkBytes, maximumTotalBytes.

Key responsibilities

It exists to provide a coherent automation script boundary for Check Bundle Budget. Its extracted size is 37 lines and 160 lexical tokens; the source digest begins 1f62f2522e44.

Important symbols

Important exported or declared symbols/sections identified by static inspection: assetsDirectory, maximumChunkBytes, maximumTotalBytes, entries, chunks, totalBytes, largestChunk. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/package.json. The containing category is Repository reference.

Simplified pseudocode

```
READ client/scripts/check-bundle-budget.mjs from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/scripts/check-bundle-budget.mjs is a automation script that owns a focused part of repository reference. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND APPLICATION

FILE REFERENCE:

client/src/App.tsx

Category: Frontend application | Kind: TypeScript module | Lines: 9 | SHA-256: 3b32435b45cc2f69...

Purpose

This frontend module implements part of the frontend application surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for App. Its extracted size is 9 lines and 46 lexical tokens; the source digest begins 3b32435b45cc.

Important symbols

Important exported or declared symbols/sections identified by static inspection: App. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/app/agent/graph.py, backend/app/agent/tools.py. The containing category is Frontend application.

Simplified pseudocode

```
READ client/src/App.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/App.tsx is a TypeScript module that owns a focused part of frontend application. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ai-elements/chain-of-thought.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 222 | SHA-256: fc5dbf976c10d865...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Chain Of Thought. Its extracted size is 222 lines and 554 lexical tokens; the source digest begins fc5dbf976c10.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ChainOfThoughtContextValue, ChainOfThoughtContext, useChainOfThought, context, ChainOfThoughtProps, ChainOfThought, chainOfThoughtContext. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/ui/badge.tsx, client/src/components/ui/collapsible.tsx, client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: client/src/components/chat/tool-parts.tsx, docs/OBSERVABILITY.md, docs/SYSTEM_DESIGN.md, scripts/generate_engineering_book.py. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ai-elements/chain-of-thought.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ai-elements/chain-of-thought.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/components/ui/badge.tsx, client/src/components/ui/collapsible.tsx, client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ai-elements/commit.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 458 | SHA-256: 112005c9e4718174...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Commit. Its extracted size is 458 lines and 879 lexical tokens; the source digest begins 112005c9e471.

Important symbols

Important exported or declared symbols/sections identified by static inspection: CommitProps, Commit, CommitHeaderProps, CommitHeader, CommitHashProps, CommitHash, CommitMessageProps. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/ui/avatar.tsx, client/src/components/ui/button-group.tsx, client/src/components/ui/collapsible.tsx, client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: backend/.env.example, backend/app/agent/graph.py, backend/app/agent/policy.py, backend/app/agent/state.py. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ai-elements/commit.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ai-elements/commit.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/components/ui/avatar.tsx, client/src/components/ui/button-group.tsx, client/src/components/ui/collapsible.tsx, client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ai-elements/conversation.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 168 | SHA-256: 1875e32cc23a6e6c...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Conversation. Its extracted size is 168 lines and 411 lexical tokens; the source digest begins 1875e32cc23a.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ConversationProps, Conversation, ConversationContentProps, ConversationContent, ConversationEmptyStateProps, ConversationEmptyState, ConversationScrollButtonProps. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/ui/button-group.tsx, client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: backend/app/agent/state.py, backend/app/api/routes/assistant.py, backend/app/schemas/assistant.py, backend/app/services/product_assistant_service.py. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ai-elements/conversation.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ai-elements/conversation.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/components/ui/button-group.tsx, client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ai-elements/message.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 363 | SHA-256: 23f66a7dc2742716...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Message. Its extracted size is 363 lines and 785 lexical tokens; the source digest begins 23f66a7dc274.

Important symbols

Important exported or declared symbols/sections identified by static inspection: MessageProps, Message, MessageContentProps, MessageContent, MessageActionsProps, MessageActions, MessageActionProps. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/ui/button-group.tsx, client/src/components/ui/tooltip.tsx, client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: .commandcode/taste/taste.md, backend/app/agent/graph.py, backend/app/agent/state.py, backend/app/agent/tools.py. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ai-elements/message.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ai-elements/message.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/components/ui/button-group.tsx, client/src/components/ui/tooltip.tsx, client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ai-elements/prompt-input.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 1463 | SHA-256: a4f2c8249411c292...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Prompt Input. Its extracted size is 1463 lines and 3258 lexical tokens; the source digest begins a4f2c8249411.

Important symbols

Important exported or declared symbols/sections identified by static inspection: convertBlobUrlToDataUrl, response, blob, reader, captureScreenshot, video, width. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/ui/command.tsx, client/src/components/ui/dropdown-menu.tsx, client/src/components/ui/hover-card.tsx, client/src/components/ui/input-group.tsx, client/src/components/ui/select.tsx. Potential consumers found through bounded textual/import analysis: client/src/components/chat/chat-input.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ai-elements/prompt-input.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ai-elements/prompt-input.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/components/ui/command.tsx, client/src/components/ui/dropdown-menu.tsx, client/src/components/ui/hover-card.tsx, client/src/components/ui/input-group.tsx, client/src/components/ui/select.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ai-elements/reasoning.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 223 | SHA-256: 6b63a166576c340b...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Reasoning. Its extracted size is 223 lines and 514 lexical tokens; the source digest begins 6b63a166576c.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ReasoningContextValue, ReasoningContext, useReasoning, context, ReasoningProps, AUTO_CLOSE_DELAY, MS_IN_S. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/ui/collapsible.tsx, client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/prompts.py, backend/app/services/context_budget.py, client/src/components/ide-workspace.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ai-elements/reasoning.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ai-elements/reasoning.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/components/ui/collapsible.tsx, client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ai-elements/shimmer.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 77 | SHA-256: 24bcb9b1dd45d616...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Shimmer. Its extracted size is 77 lines and 202 lexical tokens; the source digest begins 24bcb9b1dd45.

Important symbols

Important exported or declared symbols/sections identified by static inspection: MotionHTMLProps, motionComponentCache, getMotionComponent, TextShimmerProps, ShimmerComponent, MotionComponent, dynamicSpread. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: client/src/components/ai-elements/reasoning.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ai-elements/shimmer.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ai-elements/shimmer.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/chat/chat-input.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 232 | SHA-256: 21fdfde8c605313e...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Chat Input. Its extracted size is 232 lines and 1124 lexical tokens; the source digest begins 21fdfde8c605.

Important symbols

Important exported or declared symbols/sections identified by static inspection: SelectedRepo, ChatInputProps, ChatInput, isWorking, pendingApproval, canConnectGithub, handleConnect. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/types/agent.type.ts, client/src/lib/api.ts, client/src/assets/github.svg, client/src/components/clerk-token-provider.tsx. Potential consumers found through bounded textual/import analysis: client/src/components/chat/index.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/chat/chat-input.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/chat/chat-input.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/types/agent.type.ts, client/src/lib/api.ts, client/src/assets/github.svg, client/src/components/clerk-token-provider.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/chat/index.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 633 | SHA-256: 3a62363d4b6b4151...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Index. Its extracted size is 633 lines and 3932 lexical tokens; the source digest begins 3a62363d4b6b.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ChatInterfaceProps, SelectedRepo, EMPTY_MESSAGES, EmptyState, MessageBubble, isUser, TimelinePanel. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/hooks/use-agent-session.ts, client/src/hooks/use-user.ts, client/src/components/clerk-token-provider.tsx, client/src/lib/api.ts, client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/.env.example, backend/README.md. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/chat/index.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/chat/index.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/hooks/use-agent-session.ts, client/src/hooks/use-user.ts, client/src/components/clerk-token-provider.tsx, client/src/lib/api.ts, client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/chat/tool-parts.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 392 | SHA-256: e8f58c7466d39528...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Tool Parts. Its extracted size is 392 lines and 1207 lexical tokens; the source digest begins e8f58c7466d3.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ToolPart, ToolCall, ToolCallStep, ToolCallDiff, FileGrid, ListContent, output. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/diff-viewer.tsx, client/src/components/ai-elements/commit.tsx, client/src/components/ai-elements/chain-of-thought.tsx, client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/chat/tool-parts.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/chat/tool-parts.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/components/diff-viewer.tsx, client/src/components/ai-elements/commit.tsx, client/src/components/ai-elements/chain-of-thought.tsx, client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/clerk-token-provider.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 55 | SHA-256: 001a7a191b023d4b...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Clerk Token Provider. Its extracted size is 55 lines and 159 lexical tokens; the source digest begins 001a7a191b02.

Important symbols

Important exported or declared symbols/sections identified by static inspection: into, ClerkApiTokenState, ClerkApiTokenContext, useClerkApiToken, ClerkTokenProvider. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/axios-client.ts. Potential consumers found through bounded textual/import analysis: client/src/components/chat/chat-input.tsx, client/src/components/chat/index.tsx, client/src/components/sidebar/chat-sessions.tsx, client/src/components/sidebar/session-search-dialog.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/clerk-token-provider.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/clerk-token-provider.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/axios-client.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/diagnostic-dashboard.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 175 | SHA-256: 8e9453e56983b23e...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Diagnostic Dashboard. Its extracted size is 175 lines and 1452 lexical tokens; the source digest begins 8e9453e56983.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Editor, Props, DiagnosticStatus, newDiagnosticRequestId, DiagnosticDashboard, proposalRequestId, activePatch. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/diff-viewer.tsx, client/src/components/ui/button-group.tsx, client/src/components/ui/textarea.tsx, client/src/lib/api.ts. Potential consumers found through bounded textual/import analysis: client/src/devops/ui/diagnostic-dashboard.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/diagnostic-dashboard.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/diagnostic-dashboard.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/components/diff-viewer.tsx, client/src/components/ui/button-group.tsx, client/src/components/ui/textarea.tsx, client/src/lib/api.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/diff-viewer.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 561 | SHA-256: cd9ed9281d47453f...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Diff Viewer. Its extracted size is 561 lines and 1330 lexical tokens; the source digest begins cd9ed9281d47.

Important symbols

Important exported or declared symbols/sections identified by static inspection: VariantProps, DiffLineType, ParsedLine, ParsedFile, SplitLinePair, parsePatch, lines. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: client/src/components/chat/index.tsx, client/src/components/chat/tool-parts.tsx, client/src/components/diagnostic-dashboard.tsx, client/src/components/ide-workspace.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/diff-viewer.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/diff-viewer.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ide-workspace.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 1465 | SHA-256: 7d22d7adb2007d20...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Ide Workspace. Its extracted size is 1465 lines and 10074 lexical tokens; the source digest begins 7d22d7adb200.

Important symbols

Important exported or declared symbols/sections identified by static inspection: IDEThemeContext, useIDETheme, FileEntry, CommitCI, CommitItem, CodeFile, BottomPanel. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/diff-viewer.tsx, client/src/components/vscode-editor.tsx, client/src/lib/api.ts, client/src/lib/utils.ts, client/src/components/theme-provider.tsx. Potential consumers found through bounded textual/import analysis: client/src/index.css, client/src/pages/session/index.tsx, docs/IMPLEMENTATION_GAP_ANALYSIS.md, scripts/generate_engineering_book.py. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ide-workspace.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ide-workspace.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/components/diff-viewer.tsx, client/src/components/vscode-editor.tsx, client/src/lib/api.ts, client/src/lib/utils.ts, client/src/components/theme-provider.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/loader.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 51 | SHA-256: cc7fdf851f4eeaad...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Loader. Its extracted size is 51 lines and 106 lexical tokens; the source digest begins cc7fdf851f4e.

Important symbols

Important exported or declared symbols/sections identified by static inspection: LoaderProps, Loader, dotSizes, containerSizes. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: client/src/pages/dev/DevRunInspector.tsx, docs/IMPLEMENTATION_AUDIT.md. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/loader.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/loader.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/logo.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 14 | SHA-256: 641976667f6742d9...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Logo. Its extracted size is 14 lines and 69 lexical tokens; the source digest begins 641976667f67.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Logo. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/assets/logo.png, client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, client/src/components/sidebar/index.tsx, client/src/pages/auth/sign-in.tsx, client/src/pages/auth/sign-up.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/logo.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/logo.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/assets/logo.png, client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/mode-toggle.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 37 | SHA-256: 32eafe09ab212ae3...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Mode Toggle. Its extracted size is 37 lines and 116 lexical tokens; the source digest begins 32eafe09ab21.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ModeToggle. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/ui/button-group.tsx, client/src/components/ui/dropdown-menu.tsx, client/src/components/theme-provider.tsx. Potential consumers found through bounded textual/import analysis: client/src/components/sidebar/index.tsx, client/src/pages/guide/index.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/mode-toggle.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/mode-toggle.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/components/ui/button-group.tsx, client/src/components/ui/dropdown-menu.tsx, client/src/components/theme-provider.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/product-guide.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 144 | SHA-256: 540b878022d53354...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Product Guide. Its extracted size is 144 lines and 736 lexical tokens; the source digest begins 540b878022d5.

Important symbols

Important exported or declared symbols/sections identified by static inspection: GuideMessage, starterPrompts, newRequestId, ProductGuide, ask, message, response. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/api.ts, client/src/components/ui/button-group.tsx, client/src/components/ui/scroll-area.tsx, client/src/components/ui/textarea.tsx. Potential consumers found through bounded textual/import analysis: client/src/layouts/app-layout.tsx, docs/IMPLEMENTATION_AUDIT.md, docs/IMPLEMENTATION_STATUS.md, docs/SAAS_SECURITY_AND_SCALE.md. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/product-guide.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/product-guide.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/api.ts, client/src/components/ui/button-group.tsx, client/src/components/ui/scroll-area.tsx, client/src/components/ui/textarea.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/sidebar/chat-sessions.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 129 | SHA-256: dfe1a99e3ead286b...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Chat Sessions. Its extracted size is 129 lines and 586 lexical tokens; the source digest begins dfe1a99e3ead.

Important symbols

Important exported or declared symbols/sections identified by static inspection: sessionCacheKey, readCachedSessions, key, cached, parsed, ChatSessions, sessions. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/api.ts, client/src/components/clerk-token-provider.tsx, client/src/types/session.type.ts, client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: client/src/components/sidebar/index.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/sidebar/chat-sessions.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/sidebar/chat-sessions.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/api.ts, client/src/components/clerk-token-provider.tsx, client/src/types/session.type.ts, client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/sidebar/index.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 130 | SHA-256: 3f42c14b3465fc26...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Index. Its extracted size is 130 lines and 549 lexical tokens; the source digest begins 3f42c14b3465.

Important symbols

Important exported or declared symbols/sections identified by static inspection: AppSidebar, queryClient, navigate, handleSignOut, name, email, avatar. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/.env.example, backend/README.md. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/sidebar/index.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/sidebar/index.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/sidebar/navitems.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 44 | SHA-256: cce9fe3fa6245fe7...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Navitems. Its extracted size is 44 lines and 133 lexical tokens; the source digest begins cce9fe3fa624.

Important symbols

Important exported or declared symbols/sections identified by static inspection: navItems, NavItems. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/routes/route-guard.tsx. Potential consumers found through bounded textual/import analysis: client/src/components/sidebar/index.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/sidebar/navitems.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/sidebar/navitems.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/routes/route-guard.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/sidebar/session-search-dialog.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 99 | SHA-256: 4dbca637a3e33196...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Session Search Dialog. Its extracted size is 99 lines and 294 lexical tokens; the source digest begins 4dbca637a3e3.

Important symbols

Important exported or declared symbols/sections identified by static inspection: SessionSearchDialogProps, SessionSearchDialog, navigate, timer, sessions, isActive. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/ui/command.tsx, client/src/lib/api.ts, client/src/components/clerk-token-provider.tsx, client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: client/src/components/sidebar/index.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/sidebar/session-search-dialog.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/sidebar/session-search-dialog.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/components/ui/command.tsx, client/src/lib/api.ts, client/src/components/clerk-token-provider.tsx, client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/theme-provider.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 73 | SHA-256: 9c661a4bef9fc75f...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Theme Provider. Its extracted size is 73 lines and 142 lexical tokens; the source digest begins 9c661a4bef9f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Theme, ThemeProviderProps, ThemeProviderState, initialState, ThemeProviderContext, ThemeProvider, root. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/src/components/ide-workspace.tsx, client/src/components/mode-toggle.tsx, client/src/main.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/theme-provider.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/theme-provider.tsx is a TypeScript module that owns a focused part of frontend components. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/alert.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 76 | SHA-256: 126a26b401ab2cd3...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Alert. Its extracted size is 76 lines and 239 lexical tokens; the source digest begins 126a26b401ab.

Important symbols

Important exported or declared symbols/sections identified by static inspection: VariantProps, alertVariants, Alert, AlertTitle, AlertDescription, AlertAction. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: client/src/components/chat/chat-input.tsx, client/src/components/sidebar/chat-sessions.tsx, docs/SAAS_SECURITY_AND_SCALE.md, load/k6/README.md. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/alert.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/alert.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/avatar.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 110 | SHA-256: 232a6ac077e0f98b...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Avatar. Its extracted size is 110 lines and 374 lexical tokens; the source digest begins 232a6ac077e0.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Avatar, AvatarImage, AvatarFallback, AvatarBadge, AvatarGroup, AvatarGroupCount. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: backend/app/api/deps.py, backend/app/schemas/auth.py, backend/app/services/auth_service.py, backend/app/services/github_service.py. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/avatar.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/avatar.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/badge.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 49 | SHA-256: 1b292cd64608eef6...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Badge. Its extracted size is 49 lines and 240 lexical tokens; the source digest begins 1b292cd64608.

Important symbols

Important exported or declared symbols/sections identified by static inspection: VariantProps, badgeVariants, Badge, Comp. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: client/src/components/ai-elements/chain-of-thought.tsx, client/src/components/chat/index.tsx, client/src/components/diff-viewer.tsx, client/src/components/ide-workspace.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/badge.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/badge.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/button-group.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 83 | SHA-256: de73ccc6e10f2982...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Button Group. Its extracted size is 83 lines and 268 lexical tokens; the source digest begins de73ccc6e10f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: VariantProps, buttonGroupVariants, ButtonGroup, ButtonGroupText, Comp, ButtonGroupSeparator. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts, client/src/components/ui/separator.tsx. Potential consumers found through bounded textual/import analysis: client/src/components/ai-elements/message.tsx, client/src/components/ui/button.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/button-group.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/button-group.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts, client/src/components/ui/separator.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/button.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 67 | SHA-256: 6f7d18705d50ab34...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Button. Its extracted size is 67 lines and 482 lexical tokens; the source digest begins 6f7d18705d50.

Important symbols

Important exported or declared symbols/sections identified by static inspection: VariantProps, buttonVariants, Button, Comp. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, client/src/components/ai-elements/commit.tsx, client/src/components/ai-elements/conversation.tsx, client/src/components/ai-elements/message.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/button.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/button.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/collapsible.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 33 | SHA-256: c6598e5584f4258b...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Collapsible. Its extracted size is 33 lines and 60 lexical tokens; the source digest begins c6598e5584f4.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Collapsible, CollapsibleTrigger, CollapsibleContent. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/src/components/ai-elements/chain-of-thought.tsx, client/src/components/ai-elements/commit.tsx, client/src/components/ai-elements/reasoning.tsx, client/src/components/sidebar/index.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/collapsible.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/collapsible.tsx is a TypeScript module that owns a focused part of frontend components. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/command.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 195 | SHA-256: a198a562cd868da1...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Command. Its extracted size is 195 lines and 515 lexical tokens; the source digest begins a198a562cd86.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Command, CommandDialog, CommandInput, CommandList, CommandEmpty, CommandGroup, CommandSeparator. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts, client/src/components/ui/dialog.tsx, client/src/components/ui/input-group.tsx. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/app/agent/graph.py, backend/app/agent/prompts.py. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/command.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/command.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts, client/src/components/ui/dialog.tsx, client/src/components/ui/input-group.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/dialog.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 168 | SHA-256: 09cb8612db677d45...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Dialog. Its extracted size is 168 lines and 443 lexical tokens; the source digest begins 09cb8612db67.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Dialog, DialogTrigger, DialogPortal, DialogClose, DialogOverlay, DialogContent, DialogHeader. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts, client/src/components/ui/button-group.tsx. Potential consumers found through bounded textual/import analysis: client/src/components/ai-elements/prompt-input.tsx, client/src/components/sidebar/index.tsx, client/src/components/ui/command.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/dialog.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/dialog.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts, client/src/components/ui/button-group.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/dropdown-menu.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 267 | SHA-256: 9d6ad4621d33194c...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Dropdown Menu. Its extracted size is 267 lines and 983 lexical tokens; the source digest begins 9d6ad4621d33.

Important symbols

Important exported or declared symbols/sections identified by static inspection: DropdownMenu, DropdownMenuPortal, DropdownMenuTrigger, DropdownMenuContent, DropdownMenuGroup, DropdownMenuItem, DropdownMenuCheckboxItem. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: client/src/components/ai-elements/prompt-input.tsx, client/src/components/mode-toggle.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/dropdown-menu.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/dropdown-menu.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/empty.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 104 | SHA-256: e48ed60b400b0a05...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Empty. Its extracted size is 104 lines and 258 lexical tokens; the source digest begins e48ed60b400b.

Important symbols

Important exported or declared symbols/sections identified by static inspection: VariantProps, Empty, EmptyHeader, emptyMediaVariants, EmptyMedia, EmptyTitle, EmptyDescription. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, backend/.env.example, backend/app/agent/graph.py, backend/app/git/change_provider.py. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/empty.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/empty.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/hover-card.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 44 | SHA-256: e8166e75e8dd00e1...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Hover Card. Its extracted size is 44 lines and 188 lexical tokens; the source digest begins e8166e75e8dd.

Important symbols

Important exported or declared symbols/sections identified by static inspection: HoverCard, HoverCardTrigger, HoverCardContent. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: client/src/components/ai-elements/prompt-input.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/hover-card.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/hover-card.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/input-group.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 156 | SHA-256: e28122e46635bb66...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Input Group. Its extracted size is 156 lines and 665 lexical tokens; the source digest begins e28122e46635.

Important symbols

Important exported or declared symbols/sections identified by static inspection: VariantProps, InputGroup, inputGroupAddonVariants, InputGroupAddon, inputGroupButtonVariants, InputGroupButton, InputGroupText. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts, client/src/components/ui/button-group.tsx, client/src/components/ui/input-group.tsx, client/src/components/ui/textarea.tsx. Potential consumers found through bounded textual/import analysis: client/src/components/ai-elements/prompt-input.tsx, client/src/components/ui/command.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/input-group.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/input-group.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts, client/src/components/ui/button-group.tsx, client/src/components/ui/input-group.tsx, client/src/components/ui/textarea.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/input.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 19 | SHA-256: c4902a8c4967875e...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Input. Its extracted size is 19 lines and 146 lexical tokens; the source digest begins c4902a8c4967.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Input. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: .github/workflows/live-evals.yml, backend/app/agent/graph.py, backend/app/api/routes/incidents.py, backend/app/billing/usage_service.py. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/input.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/input.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/label.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 24 | SHA-256: a476e41f08b9de5b...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Label. Its extracted size is 24 lines and 75 lexical tokens; the source digest begins a476e41f08b9.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Label. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/memory/memory_service.py, backend/app/memory/models.py, backend/app/schemas/replay.py. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/label.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/label.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/popover.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 87 | SHA-256: 7360e38b1408819a...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Popover. Its extracted size is 87 lines and 267 lexical tokens; the source digest begins 7360e38b1408.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Popover, PopoverTrigger, PopoverContent, PopoverAnchor, PopoverHeader, PopoverTitle, PopoverDescription. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: client/src/components/ai-elements/chain-of-thought.tsx, client/src/components/chat/chat-input.tsx, client/src/components/product-guide.tsx, client/src/components/ui/command.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/popover.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/popover.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/scroll-area.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 55 | SHA-256: c3e7e2201d33bc24...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Scroll Area. Its extracted size is 55 lines and 173 lexical tokens; the source digest begins c3e7e2201d33.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ScrollArea, ScrollBar. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: client/src/components/chat/index.tsx, client/src/components/product-guide.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/scroll-area.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/scroll-area.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/select.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 190 | SHA-256: 078c479779746fed...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Select. Its extracted size is 190 lines and 775 lexical tokens; the source digest begins 078c47977974.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Select, SelectGroup, SelectValue, SelectTrigger, SelectContent, SelectLabel, SelectItem. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: backend/.env.example, backend/app/agent/graph.py, backend/app/agent/policy.py, backend/app/agent/state.py. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/select.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/select.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/separator.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 28 | SHA-256: c924a82e99fda036...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Separator. Its extracted size is 28 lines and 68 lexical tokens; the source digest begins c924a82e99fd.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Separator. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: backend/app/core/access_logging.py, backend/app/core/security.py, backend/app/services/context_budget.py, backend/app/services/delivery_service.py. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/separator.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/separator.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/sheet.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 145 | SHA-256: f31457aa6bc535c7...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Sheet. Its extracted size is 145 lines and 561 lexical tokens; the source digest begins f31457aa6bc5.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Sheet, SheetTrigger, SheetClose, SheetPortal, SheetOverlay, SheetContent, SheetHeader. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts, client/src/components/ui/button-group.tsx. Potential consumers found through bounded textual/import analysis: client/index.html, client/src/components/ui/sidebar.tsx, scripts/generate_engineering_book.py. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/sheet.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/sheet.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts, client/src/components/ui/button-group.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/sidebar.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 700 | SHA-256: 95c343fe4e02d8a8...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Sidebar. Its extracted size is 700 lines and 2393 lexical tokens; the source digest begins 95c343fe4e02.

Important symbols

Important exported or declared symbols/sections identified by static inspection: VariantProps, SIDEBAR_COOKIE_NAME, SIDEBAR_COOKIE_MAX_AGE, SIDEBAR_WIDTH, SIDEBAR_WIDTH_MOBILE, SIDEBAR_WIDTH_ICON, SIDEBAR_KEYBOARD_SHORTCUT. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/hooks/use-mobile.ts, client/src/lib/utils.ts, client/src/components/ui/button-group.tsx, client/src/components/ui/input-group.tsx, client/src/components/ui/separator.tsx. Potential consumers found through bounded textual/import analysis: client/src/components/sidebar/chat-sessions.tsx, client/src/components/sidebar/index.tsx, client/src/components/sidebar/navitems.tsx, client/src/index.css. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/sidebar.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/sidebar.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/hooks/use-mobile.ts, client/src/lib/utils.ts, client/src/components/ui/button-group.tsx, client/src/components/ui/input-group.tsx, client/src/components/ui/separator.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/skeleton.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 13 | SHA-256: 8110bba70d0cb9fe...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Skeleton. Its extracted size is 13 lines and 29 lexical tokens; the source digest begins 8110bba70d0c.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Skeleton. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: backend/app/git/change_provider.py, client/src/components/ui/sidebar.tsx, client/src/pages/incidents/index.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/skeleton.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/skeleton.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/sonner.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 47 | SHA-256: 0c729f9f1ada0b0b...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Sonner. Its extracted size is 47 lines and 97 lexical tokens; the source digest begins 0c729f9f1ada.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ToasterProps, Toaster. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/package.json, client/src/components/chat/chat-input.tsx, client/src/components/chat/index.tsx, client/src/components/diagnostic-dashboard.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/sonner.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/sonner.tsx is a TypeScript module that owns a focused part of frontend components. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/spinner.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 10 | SHA-256: 1aa4cbc4272e833e...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Spinner. Its extracted size is 10 lines and 34 lexical tokens; the source digest begins 1aa4cbc4272e.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Spinner. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: client/src/components/ai-elements/prompt-input.tsx, client/src/components/chat/chat-input.tsx, client/src/routes/route-guard.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/spinner.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/spinner.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/table.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 116 | SHA-256: a13f55a7c1406197...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Table. Its extracted size is 116 lines and 251 lexical tokens; the source digest begins a13f55a7c140.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Table, TableHeader, TableBody, TableFooter, TableRow, TableHead, TableCell. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/agent/policy.py, backend/app/agent/prompts.py, backend/app/core/access_logging.py. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/table.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/table.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/textarea.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 18 | SHA-256: 58b58d84fc54ba5f...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Textarea. Its extracted size is 18 lines and 120 lexical tokens; the source digest begins 58b58d84fc54.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Textarea. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: client/src/components/ai-elements/prompt-input.tsx, client/src/components/chat/chat-input.tsx, client/src/components/diagnostic-dashboard.tsx, client/src/components/product-guide.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/textarea.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/textarea.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/ui/tooltip.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 55 | SHA-256: 81569280cdd3c9a6...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Tooltip. Its extracted size is 55 lines and 265 lexical tokens; the source digest begins 81569280cdd3.

Important symbols

Important exported or declared symbols/sections identified by static inspection: TooltipProvider, Tooltip, TooltipTrigger, TooltipContent. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/utils.ts. Potential consumers found through bounded textual/import analysis: client/src/components/ai-elements/message.tsx, client/src/components/ai-elements/prompt-input.tsx, client/src/components/sidebar/navitems.tsx, client/src/components/ui/sidebar.tsx. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/ui/tooltip.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/ui/tooltip.tsx is a TypeScript module that owns a focused part of frontend components. Related files: client/src/lib/utils.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND COMPONENTS

FILE REFERENCE:

client/src/components/vscode-editor.tsx

Category: Frontend components | Kind: TypeScript module | Lines: 100 | SHA-256: d6bee770a033be9e...

Purpose

This frontend module implements part of the frontend components surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Vscode Editor. Its extracted size is 100 lines and 558 lexical tokens; the source digest begins d6bee770a033.

Important symbols

Important exported or declared symbols/sections identified by static inspection: OnMount, Props, EXTENSION_LANGUAGES, languageFor, filename, languageLabel, language. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/src/components/ide-workspace.tsx, docs/IMPLEMENTATION_GAP_ANALYSIS.md. The containing category is Frontend components.

Simplified pseudocode

```
READ client/src/components/vscode-editor.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/components/vscode-editor.tsx is a TypeScript module that owns a focused part of frontend components. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND APPLICATION

FILE REFERENCE:

client/src/devops/ui/diagnostic-dashboard.tsx

Category: Frontend application | Kind: TypeScript module | Lines: 7 | SHA-256: 5bb402f8f8d0f794...

Purpose

This frontend module implements part of the frontend application surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Diagnostic Dashboard. Its extracted size is 7 lines and 33 lexical tokens; the source digest begins 5bb402f8f8d0.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/diagnostic-dashboard.tsx. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Frontend application.

Simplified pseudocode

```
READ client/src/devops/ui/diagnostic-dashboard.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/devops/ui/diagnostic-dashboard.tsx is a TypeScript module that owns a focused part of frontend application. Related files: client/src/components/diagnostic-dashboard.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND HOOKS

FILE REFERENCE:

client/src/hooks/use-agent-session.ts

Category: Frontend hooks | Kind: TypeScript module | Lines: 356 | SHA-256: abd0dc96b9869e12...

Purpose

This frontend module implements part of the frontend hooks surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Use Agent Session. Its extracted size is 356 lines and 918 lexical tokens; the source digest begins abd0dc96b986.

Important symbols

Important exported or declared symbols/sections identified by static inspection: SendMessageInput, parseSsePayloads, events, blocks, rest, block, eventLine. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/env.ts, client/src/lib/axios-client.ts, client/src/lib/api.ts, client/src/types/agent.type.ts. Potential consumers found through bounded textual/import analysis: client/src/components/chat/index.tsx, docs/IMPLEMENTATION_AUDIT.md. The containing category is Frontend hooks.

Simplified pseudocode

```
READ client/src/hooks/use-agent-session.ts from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/hooks/use-agent-session.ts is a TypeScript module that owns a focused part of frontend hooks. Related files: client/src/lib/env.ts, client/src/lib/axios-client.ts, client/src/lib/api.ts, client/src/types/agent.type.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND HOOKS

FILE REFERENCE:

client/src/hooks/use-mobile.ts

Category: Frontend hooks | Kind: TypeScript module | Lines: 19 | SHA-256: ad0936f84f1df79d...

Purpose

This frontend module implements part of the frontend hooks surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Use Mobile. Its extracted size is 19 lines and 51 lexical tokens; the source digest begins ad0936f84f1d.

Important symbols

Important exported or declared symbols/sections identified by static inspection: MOBILE_BREAKPOINT, useIsMobile, mql, onChange. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/src/components/ui/sidebar.tsx. The containing category is Frontend hooks.

Simplified pseudocode

```
READ client/src/hooks/use-mobile.ts from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/hooks/use-mobile.ts is a TypeScript module that owns a focused part of frontend hooks. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND HOOKS

FILE REFERENCE:

client/src/hooks/use-user.ts

Category: Frontend hooks | Kind: TypeScript module | Lines: 28 | SHA-256: 5397bcd28f336dea...

Purpose

This frontend module implements part of the frontend hooks surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Use User. Its extracted size is 28 lines and 153 lexical tokens; the source digest begins 5397bcd28f33.

Important symbols

Important exported or declared symbols/sections identified by static inspection: useBackendUser. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/lib/api.ts, client/src/components/clerk-token-provider.tsx. Potential consumers found through bounded textual/import analysis: client/src/components/chat/index.tsx. The containing category is Frontend hooks.

Simplified pseudocode

```
READ client/src/hooks/use-user.ts from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/hooks/use-user.ts is a TypeScript module that owns a focused part of frontend hooks. Related files: client/src/lib/api.ts, client/src/components/clerk-token-provider.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND APPLICATION

FILE REFERENCE:

client/src/index.css

Category: Frontend application | Kind: stylesheet | Lines: 473 | SHA-256: 83e699d6a93635d4...

Purpose

This frontend module implements part of the frontend application surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent stylesheet boundary for Index. Its extracted size is 473 lines and 2398 lexical tokens; the source digest begins 83e699d6a936.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/.env.example, backend/README.md. The containing category is Frontend application.

Simplified pseudocode

```
READ client/src/index.css from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/index.css is a stylesheet that owns a focused part of frontend application. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND APPLICATION

FILE REFERENCE:

client/src/layouts/app-layout.tsx

Category: Frontend application | Kind: TypeScript module | Lines: 30 | SHA-256: 02d2030d82fb7744...

Purpose

This frontend module implements part of the frontend application surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for App Layout. Its extracted size is 30 lines and 81 lexical tokens; the source digest begins 02d2030d82fb.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ProductGuide, AppLayout. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/sidebar/navitems.tsx, client/src/components/ui/sidebar.tsx. Potential consumers found through bounded textual/import analysis: .commandcode/taste/taste.md, client/src/routes/index.tsx. The containing category is Frontend application.

Simplified pseudocode

```
READ client/src/layouts/app-layout.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/layouts/app-layout.tsx is a TypeScript module that owns a focused part of frontend application. Related files: client/src/components/sidebar/navitems.tsx, client/src/components/ui/sidebar.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND APPLICATION

FILE REFERENCE:

client/src/layouts/base-layout.tsx

Category: Frontend application | Kind: TypeScript module | Lines: 15 | SHA-256: 60e071466ff59d55...

Purpose

This frontend module implements part of the frontend application surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Base Layout. Its extracted size is 15 lines and 46 lexical tokens; the source digest begins 60e071466ff5.

Important symbols

Important exported or declared symbols/sections identified by static inspection: BaseLayout. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/taste.md, client/src/routes/index.tsx. The containing category is Frontend application.

Simplified pseudocode

```
READ client/src/layouts/base-layout.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/layouts/base-layout.tsx is a TypeScript module that owns a focused part of frontend application. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND API AND UTILITIES

FILE REFERENCE:

client/src/lib/api.ts

Category: Frontend API and utilities | Kind: TypeScript module | Lines: 585 | SHA-256: ecfcd7470d5e8c5e...

Purpose

This frontend module implements part of the frontend api and utilities surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Api. Its extracted size is 585 lines and 1645 lexical tokens; the source digest begins ecfcd7470d5e.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ProductAssistantResponse, BackendHealth, UsageEntitlement, getUsageEntitlement, response, startSubscriptionCheckout, getBackendHealth. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/types/auth.type.ts, client/src/types/agent.type.ts, client/src/types/session.type.ts, client/src/types/github.type.ts, client/src/types/incident.type.ts. Potential consumers found through bounded textual/import analysis: .commandcode/taste/taste.md, backend/.env.example, backend/app/agent/graph.py, backend/app/api/deps.py. The containing category is Frontend API and utilities.

Simplified pseudocode

```
READ client/src/lib/api.ts from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/lib/api.ts is a TypeScript module that owns a focused part of frontend api and utilities. Related files: client/src/types/auth.type.ts, client/src/types/agent.type.ts, client/src/types/session.type.ts, client/src/types/github.type.ts, client/src/types/incident.type.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND API AND UTILITIES

FILE REFERENCE:

client/src/lib/auth-redirect.ts

Category: Frontend API and utilities | Kind: TypeScript module | Lines: 59 | SHA-256: 2326bd97055b44b5...

Purpose

This frontend module implements part of the frontend api and utilities surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Auth Redirect. Its extracted size is 59 lines and 180 lexical tokens; the source digest begins 2326bd97055b.

Important symbols

Important exported or declared symbols/sections identified by static inspection: DEFAULT_POST_AUTH_PATH, PriorLocation, isAllowedPostAuthPath, pathname, fromLocationState, from, prior. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/package.json, client/scripts/check-auth-redirect.mjs, client/src/pages/auth/sign-in.tsx, client/src/pages/auth/sign-up.tsx. The containing category is Frontend API and utilities.

Simplified pseudocode

```
READ client/src/lib/auth-redirect.ts from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/lib/auth-redirect.ts is a TypeScript module that owns a focused part of frontend api and utilities. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND API AND UTILITIES

FILE REFERENCE:

client/src/lib/axios-client.ts

Category: Frontend API and utilities | Kind: TypeScript module | Lines: 89 | SHA-256: 3babecb415fadb41...

Purpose

This frontend module implements part of the frontend api and utilities surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Axios Client. Its extracted size is 89 lines and 375 lexical tokens; the source digest begins 3babecb415fa.

Important symbols

Important exported or declared symbols/sections identified by static inspection: InternalAxiosRequestConfig, CustomError, RetriableGetConfig, options, API, TokenOptions, setTokenGetter. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/taste.md, client/src/components/clerk-token-provider.tsx, client/src/hooks/use-agent-session.ts, client/src/lib/api.ts. The containing category is Frontend API and utilities.

Simplified pseudocode

```
READ client/src/lib/axios-client.ts from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/lib/axios-client.ts is a TypeScript module that owns a focused part of frontend api and utilities. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND API AND UTILITIES

FILE REFERENCE:

client/src/lib/env.ts

Category: Frontend API and utilities | Kind: TypeScript module | Lines: 30 | SHA-256: 9a9a3eccb593e506...

Purpose

This frontend module implements part of the frontend api and utilities surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Env. Its extracted size is 30 lines and 157 lexical tokens; the source digest begins 9a9a3eccb593.

Important symbols

Important exported or declared symbols/sections identified by static inspection: configuredApiUrl, resolveLocalApiUrl, url, isLoopbackApi, isLoopbackPage, BASE_API_URL, CLERK_PUBLISHABLE_KEY. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, .github/workflows/live-evals.yml, README.md. The containing category is Frontend API and utilities.

Simplified pseudocode

```
READ client/src/lib/env.ts from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/lib/env.ts is a TypeScript module that owns a focused part of frontend api and utilities. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND API AND UTILITIES

FILE REFERENCE:

client/src/lib/utils.ts

Category: Frontend API and utilities | Kind: TypeScript module | Lines: 16 | SHA-256: db7c8da3316066d0...

Purpose

This frontend module implements part of the frontend api and utilities surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Utils. Its extracted size is 16 lines and 34 lexical tokens; the source digest begins db7c8da33160.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ClassValue, cn, nanoid, generateSlugId. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, backend/app/agent/graph.py, backend/app/api/deps.py, backend/app/api/routes/incidents.py. The containing category is Frontend API and utilities.

Simplified pseudocode

```
READ client/src/lib/utils.ts from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/lib/utils.ts is a TypeScript module that owns a focused part of frontend api and utilities. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND APPLICATION

FILE REFERENCE:

client/src/main.tsx

Category: Frontend application | Kind: TypeScript module | Lines: 88 | SHA-256: d597c90d70e03b20...

Purpose

This frontend module implements part of the frontend application surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Main. Its extracted size is 88 lines and 372 lexical tokens; the source digest begins d597c90d70e0.

Important symbols

Important exported or declared symbols/sections identified by static inspection: queryClient, PUBLISHABLE_KEY, isPublicDemo, root, ConfigurationRequired, PublicDemoRoot. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/taste.md, .github/workflows/ci.yml, .github/workflows/deployment-readiness.yml, README.md. The containing category is Frontend application.

Simplified pseudocode

```
READ client/src/main.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/main.tsx is a TypeScript module that owns a focused part of frontend application. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND PAGES

FILE REFERENCE:

client/src/pages/auth/sign-in.tsx

Category: Frontend pages | Kind: TypeScript module | Lines: 86 | SHA-256: 4f0ba4ce4b1ddac2...

Purpose

This frontend module implements part of the frontend pages surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Sign In. Its extracted size is 86 lines and 466 lexical tokens; the source digest begins 4f0ba4ce4b1d.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ServiceAvailabilityNotice, unavailable, message, SignInPage, location, postAuthRedirect. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/logo.tsx, client/src/lib/auth-redirect.ts, client/src/lib/api.ts. Potential consumers found through bounded textual/import analysis: .commandcode/taste/taste.md, backend/app/api/deps.py, backend/app/api/routes/auth.py, backend/app/main.py. The containing category is Frontend pages.

Simplified pseudocode

```
READ client/src/pages/auth/sign-in.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/pages/auth/sign-in.tsx is a TypeScript module that owns a focused part of frontend pages. Related files: client/src/components/logo.tsx, client/src/lib/auth-redirect.ts, client/src/lib/api.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND PAGES

FILE REFERENCE:

client/src/pages/auth/sign-up.tsx

Category: Frontend pages | Kind: TypeScript module | Lines: 46 | SHA-256: 4cefb181a1d1a170...

Purpose

This frontend module implements part of the frontend pages surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Sign Up. Its extracted size is 46 lines and 249 lexical tokens; the source digest begins 4cefb181a1d1.

Important symbols

Important exported or declared symbols/sections identified by static inspection: SignUpPage, location, postAuthRedirect. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/logo.tsx, client/src/lib/auth-redirect.ts. Potential consumers found through bounded textual/import analysis: .commandcode/taste/taste.md, backend/app/api/routes/auth.py, backend/app/main.py, client/scripts/check-auth-redirect.mjs. The containing category is Frontend pages.

Simplified pseudocode

```
READ client/src/pages/auth/sign-up.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/pages/auth/sign-up.tsx is a TypeScript module that owns a focused part of frontend pages. Related files: client/src/components/logo.tsx, client/src/lib/auth-redirect.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND PAGES

FILE REFERENCE:

client/src/pages/billing/index.tsx

Category: Frontend pages | Kind: TypeScript module | Lines: 109 | SHA-256: 554d31faef090b36...

Purpose

This frontend module implements part of the frontend pages surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Index. Its extracted size is 109 lines and 800 lexical tokens; the source digest begins 554d31faef09.

Important symbols

Important exported or declared symbols/sections identified by static inspection: PLAN_COPY, BillingPage, entitlement, checkout, currentTier, current, plan. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/ui/button-group.tsx, client/src/components/ui/badge.tsx, client/src/lib/api.ts. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/.env.example, backend/README.md. The containing category is Frontend pages.

Simplified pseudocode

```
READ client/src/pages/billing/index.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/pages/billing/index.tsx is a TypeScript module that owns a focused part of frontend pages. Related files: client/src/components/ui/button-group.tsx, client/src/components/ui/badge.tsx, client/src/lib/api.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND PAGES

FILE REFERENCE:

client/src/pages/demo/index.tsx

Category: Frontend pages | Kind: TypeScript module | Lines: 487 | SHA-256: c936651d35256681...

Purpose

This frontend module implements part of the frontend pages surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Index. Its extracted size is 487 lines and 2250 lexical tokens; the source digest begins c936651d3525.

Important symbols

Important exported or declared symbols/sections identified by static inspection: steps, sourceBefore, Stepper, SourceView, changed, value, ExactDiff. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/.env.example, backend/README.md. The containing category is Frontend pages.

Simplified pseudocode

```
READ client/src/pages/demo/index.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/pages/demo/index.tsx is a TypeScript module that owns a focused part of frontend pages. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND PAGES

FILE REFERENCE:

client/src/pages/dev/DevRunInspector.tsx

Category: Frontend pages | Kind: TypeScript module | Lines: 1129 | SHA-256: 4adce300e2d24571...

Purpose

This frontend module implements part of the frontend pages surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Devruninspector. Its extracted size is 1129 lines and 4099 lexical tokens; the source digest begins 4adce300e2d2.

Important symbols

Important exported or declared symbols/sections identified by static inspection: KeyboardEvent, DevRunData, ReplayImpactTarget, CapabilitiesData, replayTabs, ReplayTab, replayLoadMessage. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/diff-viewer.tsx, client/src/components/loader.tsx, client/src/lib/api.ts. Potential consumers found through bounded textual/import analysis: client/src/routes/route.ts, docs/FEATURE_PARITY_MATRIX.md. The containing category is Frontend pages.

Simplified pseudocode

```
READ client/src/pages/dev/DevRunInspector.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/pages/dev/DevRunInspector.tsx is a TypeScript module that owns a focused part of frontend pages. Related files: client/src/components/diff-viewer.tsx, client/src/components/loader.tsx, client/src/lib/api.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND PAGES

FILE REFERENCE:

client/src/pages/error-boundary/index.tsx

Category: Frontend pages | Kind: TypeScript module | Lines: 51 | SHA-256: 54597b19f5b16715...

Purpose

This frontend module implements part of the frontend pages surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Index. Its extracted size is 51 lines and 313 lexical tokens; the source digest begins 54597b19f5b1.

Important symbols

Important exported or declared symbols/sections identified by static inspection: RouteErrorBoundary, error, status, missing. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/.env.example, backend/README.md. The containing category is Frontend pages.

Simplified pseudocode

```
READ client/src/pages/error-boundary/index.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/pages/error-boundary/index.tsx is a TypeScript module that owns a focused part of frontend pages. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND PAGES

FILE REFERENCE:

client/src/pages/guide/index.tsx

Category: Frontend pages | Kind: TypeScript module | Lines: 218 | SHA-256: e14b4679e6240d55...

Purpose

This frontend module implements part of the frontend pages surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Index. Its extracted size is 218 lines and 1853 lexical tokens; the source digest begins e14b4679e624.

Important symbols

Important exported or declared symbols/sections identified by static inspection: GuideStep, workflow, features, SectionTitle, GuidePage, activeStep, activeFeature. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/logo.tsx, client/src/components/mode-toggle.tsx, client/src/components/ui/badge.tsx, client/src/components/ui/button-group.tsx, client/src/routes/route-guard.tsx. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/.env.example, backend/README.md. The containing category is Frontend pages.

Simplified pseudocode

```
READ client/src/pages/guide/index.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/pages/guide/index.tsx is a TypeScript module that owns a focused part of frontend pages. Related files: client/src/components/logo.tsx, client/src/components/mode-toggle.tsx, client/src/components/ui/badge.tsx, client/src/components/ui/button-group.tsx, client/src/routes/route-guard.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND PAGES

FILE REFERENCE:

client/src/pages/home/index.tsx

Category: Frontend pages | Kind: TypeScript module | Lines: 13 | SHA-256: aa9027686e310482...

Purpose

This frontend module implements part of the frontend pages surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Index. Its extracted size is 13 lines and 36 lexical tokens; the source digest begins aa9027686e31.

Important symbols

Important exported or declared symbols/sections identified by static inspection: HomePage, location. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/chat/chat-input.tsx. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/.env.example, backend/README.md. The containing category is Frontend pages.

Simplified pseudocode

```
READ client/src/pages/home/index.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/pages/home/index.tsx is a TypeScript module that owns a focused part of frontend pages. Related files: client/src/components/chat/chat-input.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND PAGES

FILE REFERENCE:

client/src/pages/incidents/index.tsx

Category: Frontend pages | Kind: TypeScript module | Lines: 363 | SHA-256: cbebfde1512f0937...

Purpose

This frontend module implements part of the frontend pages surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Index. Its extracted size is 363 lines and 3423 lexical tokens; the source digest begins cbebfde1512f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: prettify, shortSha, EMPTY_INCIDENTS, EMPTY_SESSIONS, dateTime, parsed, listValue. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/ui/badge.tsx, client/src/components/ui/button-group.tsx, client/src/components/ui/input-group.tsx, client/src/components/ui/textarea.tsx, client/src/components/ui/skeleton.tsx. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/.env.example, backend/README.md. The containing category is Frontend pages.

Simplified pseudocode

```
READ client/src/pages/incidents/index.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/pages/incidents/index.tsx is a TypeScript module that owns a focused part of frontend pages. Related files: client/src/components/ui/badge.tsx, client/src/components/ui/button-group.tsx, client/src/components/ui/input-group.tsx, client/src/components/ui/textarea.tsx, client/src/components/ui/skeleton.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND PAGES

FILE REFERENCE:

client/src/pages/not-found/index.tsx

Category: Frontend pages | Kind: TypeScript module | Lines: 26 | SHA-256: b760f45a2c4c4e67...

Purpose

This frontend module implements part of the frontend pages surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Index. Its extracted size is 26 lines and 174 lexical tokens; the source digest begins b760f45a2c4c.

Important symbols

Important exported or declared symbols/sections identified by static inspection: NotFoundPage. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/.env.example, backend/README.md. The containing category is Frontend pages.

Simplified pseudocode

```
READ client/src/pages/not-found/index.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/pages/not-found/index.tsx is a TypeScript module that owns a focused part of frontend pages. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND PAGES

FILE REFERENCE:

client/src/pages/session/index.tsx

Category: Frontend pages | Kind: TypeScript module | Lines: 2120 | SHA-256: 70e97a9e50ad5efb...

Purpose

This frontend module implements part of the frontend pages surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Index. Its extracted size is 2120 lines and 8071 lexical tokens; the source digest begins 70e97a9e50ad.

Important symbols

Important exported or declared symbols/sections identified by static inspection: sections, Section, label, shortSha, formatHistoricalTimestamp, parsed, pathsReferToSameFile. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/chat/chat-input.tsx, client/src/components/ui/badge.tsx, client/src/components/ui/textarea.tsx, client/src/components/ide-workspace.tsx, client/src/lib/api.ts. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/.env.example, backend/README.md. The containing category is Frontend pages.

Simplified pseudocode

```
READ client/src/pages/session/index.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/pages/session/index.tsx is a TypeScript module that owns a focused part of frontend pages. Related files: client/src/components/chat/chat-input.tsx, client/src/components/ui/badge.tsx, client/src/components/ui/textarea.tsx, client/src/components/ide-workspace.tsx, client/src/lib/api.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND ROUTING

FILE REFERENCE:

client/src/routes/index.tsx

Category: Frontend routing | Kind: TypeScript module | Lines: 50 | SHA-256: 3bd5985a339f414b...

Purpose

This frontend module implements part of the frontend routing surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Index. Its extracted size is 50 lines and 120 lexical tokens; the source digest begins 3bd5985a339f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: RootLayout, router. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/layouts/base-layout.tsx, client/src/layouts/app-layout.tsx, client/src/pages/not-found/index.tsx, client/src/pages/error-boundary/index.tsx. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/.env.example, backend/README.md. The containing category is Frontend routing.

Simplified pseudocode

```
READ client/src/routes/index.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/routes/index.tsx is a TypeScript module that owns a focused part of frontend routing. Related files: client/src/layouts/base-layout.tsx, client/src/layouts/app-layout.tsx, client/src/pages/not-found/index.tsx, client/src/pages/error-boundary/index.tsx. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND ROUTING

FILE REFERENCE:

client/src/routes/route-guard.tsx

Category: Frontend routing | Kind: TypeScript module | Lines: 38 | SHA-256: 45bcd8f9ebde7d57...

Purpose

This frontend module implements part of the frontend routing surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Route Guard. Its extracted size is 38 lines and 112 lexical tokens; the source digest begins 45bcd8f9ebde.

Important symbols

Important exported or declared symbols/sections identified by static inspection: RouteGuardProps, RouteGuard, location, _isAuthRoute. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: client/src/components/logo.tsx, client/src/components/ui/spinner.tsx, client/src/lib/auth-redirect.ts. Potential consumers found through bounded textual/import analysis: .commandcode/taste/taste.md, client/src/routes/index.tsx, scripts/generate_engineering_book.py. The containing category is Frontend routing.

Simplified pseudocode

```
READ client/src/routes/route-guard.tsx from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/routes/route-guard.tsx is a TypeScript module that owns a focused part of frontend routing. Related files: client/src/components/logo.tsx, client/src/components/ui/spinner.tsx, client/src/lib/auth-redirect.ts. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND ROUTING

FILE REFERENCE:

client/src/routes/route.ts

Category: Frontend routing | Kind: TypeScript module | Lines: 106 | SHA-256: 2b2e14f94398f514...

Purpose

This frontend module implements part of the frontend routing surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Route. Its extracted size is 106 lines and 291 lexical tokens; the source digest begins 2b2e14f94398.

Important symbols

Important exported or declared symbols/sections identified by static inspection: SignInPage, SignUpPage, HomePage, DemoPage, GuidePage, BillingPage, SessionPage. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, backend/app/api/middleware.py, backend/app/api/routes/assistant.py. The containing category is Frontend routing.

Simplified pseudocode

```
READ client/src/routes/route.ts from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/routes/route.ts is a TypeScript module that owns a focused part of frontend routing. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND CONTRACTS

FILE REFERENCE:

client/src/types/agent.type.ts

Category: Frontend contracts | Kind: TypeScript module | Lines: 208 | SHA-256: c7b541388128b548...

Purpose

This frontend module implements part of the frontend contracts surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Agent.Type. Its extracted size is 208 lines and 550 lexical tokens; the source digest begins c7b541388128.

Important symbols

Important exported or declared symbols/sections identified by static inspection: AgentRole, AgentMessage, RagSource, EvidenceItem, TimelineEvent, CIInvestigationSummary, RepositoryContext. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/src/components/chat/chat-input.tsx, client/src/components/chat/index.tsx, client/src/components/ide-workspace.tsx, client/src/hooks/use-agent-session.ts. The containing category is Frontend contracts.

Simplified pseudocode

```
READ client/src/types/agent.type.ts from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/types/agent.type.ts is a TypeScript module that owns a focused part of frontend contracts. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND CONTRACTS

FILE REFERENCE:

client/src/types/auth.type.ts

Category: Frontend contracts | Kind: TypeScript module | Lines: 14 | SHA-256: 747621458f328508...

Purpose

This frontend module implements part of the frontend contracts surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Auth.Type. Its extracted size is 14 lines and 25 lexical tokens; the source digest begins 747621458f32.

Important symbols

Important exported or declared symbols/sections identified by static inspection: AuthUser, AuthResponse. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/src/lib/api.ts. The containing category is Frontend contracts.

Simplified pseudocode

```
READ client/src/types/auth.type.ts from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/types/auth.type.ts is a TypeScript module that owns a focused part of frontend contracts. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND CONTRACTS

FILE REFERENCE:

client/src/types/github.type.ts

Category: Frontend contracts | Kind: TypeScript module | Lines: 24 | SHA-256: 4471ea3841757834...

Purpose

This frontend module implements part of the frontend contracts surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Github.Type. Its extracted size is 24 lines and 39 lexical tokens; the source digest begins 4471ea384175.

Important symbols

Important exported or declared symbols/sections identified by static inspection: GithubRepo, GithubReposResponse, GithubConnectResponse. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/src/components/chat/index.tsx, client/src/lib/api.ts. The containing category is Frontend contracts.

Simplified pseudocode

```
READ client/src/types/github.type.ts from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/types/github.type.ts is a TypeScript module that owns a focused part of frontend contracts. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND CONTRACTS

FILE REFERENCE:

client/src/types/incident.type.ts

Category: Frontend contracts | Kind: TypeScript module | Lines: 110 | SHA-256: ef3067ba0cdc1b9a...

Purpose

This frontend module implements part of the frontend contracts surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Incident.Type. Its extracted size is 110 lines and 227 lexical tokens; the source digest begins ef3067ba0cdc.

Important symbols

Important exported or declared symbols/sections identified by static inspection: IncidentRecord, IncidentsResponse, IncidentFeedbackInput, FlakyTestObservation, RepositoryAutomationSettings, RepositoryAutomationSettingsInput. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/src/lib/api.ts, client/src/pages/incidents/index.tsx. The containing category is Frontend contracts.

Simplified pseudocode

```
READ client/src/types/incident.type.ts from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/types/incident.type.ts is a TypeScript module that owns a focused part of frontend contracts. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

FRONTEND CONTRACTS

FILE REFERENCE:

client/src/types/session.type.ts

Category: Frontend contracts | Kind: TypeScript module | Lines: 59 | SHA-256: d32abd6189108571...

Purpose

This frontend module implements part of the frontend contracts surface. It composes typed data, route state, and design-system primitives into a user-facing interaction without directly acquiring backend credentials.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Session.Type. Its extracted size is 59 lines and 138 lexical tokens; the source digest begins d32abd618910.

Important symbols

Important exported or declared symbols/sections identified by static inspection: SessionRecord, SessionsResponse, SingleSessionResponse, HistoricalRun, SessionRunHistoryResponse. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/src/components/ide-workspace.tsx, client/src/components/sidebar/chat-sessions.tsx, client/src/lib/api.ts, client/src/pages/incidents/index.tsx. The containing category is Frontend contracts.

Simplified pseudocode

```
READ client/src/types/session.type.ts from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates after browser routing or data hydration selects this UI surface. Server-side authorization and repository mutation checks remain in the backend.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/src/types/session.type.ts is a TypeScript module that owns a focused part of frontend contracts. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

PROJECT CONFIGURATION

FILE REFERENCE:

client/tsconfig.app.json

Category: Project configuration | Kind: JSON configuration | Lines: 32 | SHA-256: f019166937bfba43...

Purpose

This JSON configuration is part of the project configuration layer. It supplies the repository behavior or operational contract associated with its declared configuration and content structure.

Key responsibilities

It exists to provide a coherent JSON configuration boundary for Tsconfig.App. Its extracted size is 32 lines and 51 lexical tokens; the source digest begins f019166937bf.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/README.md, client/tsconfig.json. The containing category is Project configuration.

Simplified pseudocode

```
READ client/tsconfig.app.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/tsconfig.app.json is a JSON configuration that owns a focused part of project configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

PROJECT CONFIGURATION

FILE REFERENCE:

client/tsconfig.json

Category: Project configuration | Kind: JSON configuration | Lines: 14 | SHA-256: 52cf336c2e102444...

Purpose

This JSON configuration is part of the project configuration layer. It supplies the repository behavior or operational contract associated with files, references, compilerOptions.

Key responsibilities

It exists to provide a coherent JSON configuration boundary for Tsconfig. Its extracted size is 14 lines and 17 lexical tokens; the source digest begins 52cf336c2e10.

Important symbols

Important exported or declared symbols/sections identified by static inspection: files, references, compilerOptions. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, backend/app/services/ci_investigation_service.py, backend/app/services/patch_engine.py, client/README.md. The containing category is Project configuration.

Simplified pseudocode

```
READ client/tsconfig.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/tsconfig.json is a JSON configuration that owns a focused part of project configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

PROJECT CONFIGURATION

FILE REFERENCE:

client/tsconfig.node.json

Category: Project configuration | Kind: JSON configuration | Lines: 24 | SHA-256: 90a22c920cbc1462...

Purpose

This JSON configuration is part of the project configuration layer. It supplies the repository behavior or operational contract associated with its declared configuration and content structure.

Key responsibilities

It exists to provide a coherent JSON configuration boundary for Tsconfig.Node. Its extracted size is 24 lines and 42 lexical tokens; the source digest begins 90a22c920cbc.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/README.md, client/tsconfig.json. The containing category is Project configuration.

Simplified pseudocode

```
READ client/tsconfig.node.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/tsconfig.node.json is a JSON configuration that owns a focused part of project configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DEPLOYMENT CONFIGURATION

FILE REFERENCE:

client/vercel.json

Category: Deployment configuration | Kind: JSON configuration | Lines: 26 | SHA-256: b1a01827ced9887e...

Purpose

This JSON configuration is part of the deployment configuration layer. It supplies the repository behavior or operational contract associated with $schema, framework, installCommand.

Key responsibilities

It exists to provide a coherent JSON configuration boundary for Vercel. Its extracted size is 26 lines and 156 lexical tokens; the source digest begins b1a01827ced9.

Important symbols

Important exported or declared symbols/sections identified by static inspection: $schema, framework, installCommand, buildCommand, outputDirectory, rewrites, headers. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/services/product_assistant_service.py, backend/tests/test_frontend_deployment_security.py, docs/DEPLOYMENT.md, docs/IMPLEMENTATION_AUDIT.md. The containing category is Deployment configuration.

Simplified pseudocode

```
READ client/vercel.json from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/vercel.json is a JSON configuration that owns a focused part of deployment configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

REPOSITORY REFERENCE

FILE REFERENCE:

client/vite.config.ts

Category: Repository reference | Kind: TypeScript module | Lines: 14 | SHA-256: 89907bc80d298825...

Purpose

This TypeScript module is part of the repository reference layer. It supplies the repository behavior or operational contract associated with its declared configuration and content structure.

Key responsibilities

It exists to provide a coherent TypeScript module boundary for Vite.Config. Its extracted size is 14 lines and 37 lexical tokens; the source digest begins 89907bc80d29.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: client/tsconfig.node.json. The containing category is Repository reference.

Simplified pseudocode

```
READ client/vite.config.ts from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The client treats backend responses as the authority for safety state. It can present loading, error, and limitation states but must not mint approval, repository access, or execution authority.

Brief summary

In one sentence: client/vite.config.ts is a TypeScript module that owns a focused part of repository reference. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DEPLOYMENT CONFIGURATION

FILE REFERENCE:

docker-compose.yml

Category: Deployment configuration | Kind: YAML configuration | Lines: 33 | SHA-256: 669c93e715bb74d7...

Purpose

This YAML configuration is part of the deployment configuration layer. It supplies the repository behavior or operational contract associated with services, volumes.

Key responsibilities

It exists to provide a coherent YAML configuration boundary for Docker Compose. Its extracted size is 33 lines and 56 lexical tokens; the source digest begins 669c93e715bb.

Important symbols

Important exported or declared symbols/sections identified by static inspection: services, volumes. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/agent/graph.py, backend/app/services/patch_engine.py, backend/app/services/workspace_service.py, backend/evals/cases/compose-reference.json. The containing category is Deployment configuration.

Simplified pseudocode

```
READ docker-compose.yml from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docker-compose.yml is a YAML configuration that owns a focused part of deployment configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/AGENT_SAFETY.md

Category: Architecture documentation | Kind: documentation | Lines: 10 | SHA-256: 45876007a41483e3...

Purpose

This documentation file records the repository's current guidance for Agent Safety. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Agent Safety. Its extracted size is 10 lines and 157 lexical tokens; the source digest begins 45876007a414.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Agent safety boundary. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/IMPLEMENTATION_STATUS.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/AGENT_SAFETY.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/APPROVAL_MODEL.md

Category: Architecture documentation | Kind: documentation | Lines: 23 | SHA-256: 96a4ea35e44b03e2...

Purpose

This documentation file records the repository's current guidance for Approval Model. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Approval Model. Its extracted size is 23 lines and 146 lexical tokens; the source digest begins 96a4ea35e44b.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Exact-diff approval model, Approval hash. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/APPROVAL_MODEL.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/ARCHITECTURE.md

Category: Architecture documentation | Kind: documentation | Lines: 60 | SHA-256: f0ffaa6f6fec3a14...

Purpose

This documentation file records the repository's current guidance for Architecture. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Architecture. Its extracted size is 60 lines and 461 lexical tokens; the source digest begins f0ffaa6f6fec.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Architecture, Engineering Intelligence graph, Session workspace, Safety boundary. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/IMPLEMENTATION_STATUS.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/ARCHITECTURE.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/DELIVERY_WORKFLOW.md

Category: Architecture documentation | Kind: documentation | Lines: 13 | SHA-256: 75cb9d59e8b1fa7c...

Purpose

This documentation file records the repository's current guidance for Delivery Workflow. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Delivery Workflow. Its extracted size is 13 lines and 139 lexical tokens; the source digest begins 75cb9d59e8b1.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Delivery workflow. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/DELIVERY_WORKFLOW.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/DEMO_SCRIPT.md

Category: Architecture documentation | Kind: documentation | Lines: 46 | SHA-256: 9bf51d819008fae5...

Purpose

This documentation file records the repository's current guidance for Demo Script. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Demo Script. Its extracted size is 46 lines and 371 lexical tokens; the source digest begins 9bf51d819008.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Base64 Ops demo script, 1. Set the context, 2. Current CI failure, 3. Replay, 4. Historical CI failure, 5. Close, Safety notes. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: README.md, scripts/demo.ps1, scripts/demo.sh. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/DEMO_SCRIPT.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/DEPLOYMENT.md

Category: Architecture documentation | Kind: documentation | Lines: 92 | SHA-256: f357eb854236ca3f...

Purpose

This documentation file records the repository's current guidance for Deployment. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Deployment. Its extracted size is 92 lines and 1325 lexical tokens; the source digest begins f357eb854236.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Production deployment, Vercel frontend, Render backend, Render service count, Free-tier evaluation profile, Operating model, Manually created Render services. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/app/git/actions.py, docs/IMPLEMENTATION_AUDIT.md, docs/PRODUCTION_READINESS.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/DEPLOYMENT.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/DIAGNOSTIC_DASHBOARD.md

Category: Architecture documentation | Kind: documentation | Lines: 67 | SHA-256: 8edb1f2ca9b5a4e7...

Purpose

This documentation file records the repository's current guidance for Diagnostic Dashboard. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Diagnostic Dashboard. Its extracted size is 67 lines and 398 lexical tokens; the source digest begins 8edb1f2ca9b5.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Diagnostic Dashboard, Streaming contract, Proposal generation, Delivery safety, Bounds and redaction, Verification. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/DIAGNOSTIC_DASHBOARD.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/ENGINEERING_GRAPH.md

Category: Architecture documentation | Kind: documentation | Lines: 94 | SHA-256: baf63d814d907736...

Purpose

This documentation file records the repository's current guidance for Engineering Graph. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Engineering Graph. Its extracted size is 94 lines and 674 lexical tokens; the source digest begins baf63d814d90.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Engineering graph foundation, What is recorded, Trust and safety, Build lifecycle, APIs, Current limits, Change impact analysis. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/ARCHITECTURE.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/ENGINEERING_GRAPH.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/EVALUATION.md

Category: Architecture documentation | Kind: documentation | Lines: 18 | SHA-256: 0d5d61cd60ad0bee...

Purpose

This documentation file records the repository's current guidance for Evaluation. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Evaluation. Its extracted size is 18 lines and 207 lexical tokens; the source digest begins 0d5d61cd60ad.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Evaluation. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: README.md, docs/FEATURE_PARITY_MATRIX.md, docs/IMPLEMENTATION_STATUS.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/EVALUATION.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/EXECUTION_SANDBOX.md

Category: Architecture documentation | Kind: documentation | Lines: 59 | SHA-256: 7b6dba126e408065...

Purpose

This documentation file records the repository's current guidance for Execution Sandbox. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Execution Sandbox. Its extracted size is 59 lines and 397 lexical tokens; the source digest begins 7b6dba126e40.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Execution Runtime Sandbox, Overview, Threat Model, Static vs Active Validation, Static Validation, Active Validation, Workspace Integrity & Side-Effect Detection. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/IMPLEMENTATION_STATUS.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/EXECUTION_SANDBOX.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/FAILURE_HANDLING.md

Category: Architecture documentation | Kind: documentation | Lines: 5 | SHA-256: 3ff6d618d3bd9ed3...

Purpose

This documentation file records the repository's current guidance for Failure Handling. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Failure Handling. Its extracted size is 5 lines and 79 lexical tokens; the source digest begins 3ff6d618d3bd.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Failure handling. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/FAILURE_HANDLING.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/FEATURE_PARITY_MATRIX.md

Category: Architecture documentation | Kind: documentation | Lines: 73 | SHA-256: d9af2042599e7d5b...

Purpose

This documentation file records the repository's current guidance for Feature Parity Matrix. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Feature Parity Matrix. Its extracted size is 73 lines and 1164 lexical tokens; the source digest begins d9af2042599e.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Backend / Frontend Feature-Parity Matrix, Supporting DONE capabilities, Completion evidence, Deterministic rendered fixture. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/IMPLEMENTATION_STATUS.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/FEATURE_PARITY_MATRIX.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/GITHUB_ACTIONS_INTELLIGENCE.md

Category: Architecture documentation | Kind: documentation | Lines: 84 | SHA-256: d645011359053259...

Purpose

This documentation file records the repository's current guidance for Github Actions Intelligence. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Github Actions Intelligence. Its extracted size is 84 lines and 838 lexical tokens; the source digest begins d64501135905.

Important symbols

Important exported or declared symbols/sections identified by static inspection: GitHub Actions intelligence, Investigation flow, Safety boundaries, Bounds and replay, Automated incident intake and sharing, Verification status. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/GITHUB_ACTIONS_INTELLIGENCE.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/IMPLEMENTATION_AUDIT.md

Category: Architecture documentation | Kind: documentation | Lines: 42 | SHA-256: a31a42359f776120...

Purpose

This documentation file records the repository's current guidance for Implementation Audit. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Implementation Audit. Its extracted size is 42 lines and 594 lexical tokens; the source digest begins a31a42359f77.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Base64 Ops implementation audit, Current architecture, Important implementation locations, Existing LangGraph flow, Existing gaps against the evidence-first target, Implementation approach. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/FEATURE_PARITY_MATRIX.md, docs/IMPLEMENTATION_STATUS.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/IMPLEMENTATION_AUDIT.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/IMPLEMENTATION_GAP_ANALYSIS.md

Category: Architecture documentation | Kind: documentation | Lines: 103 | SHA-256: 944fdc2ccd238759...

Purpose

This documentation file records the repository's current guidance for Implementation Gap Analysis. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Implementation Gap Analysis. Its extracted size is 103 lines and 1188 lexical tokens; the source digest begins 944fdc2ccd23.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Base64 Ops implementation gap analysis, Reusable, implemented foundations, Partially implemented or incomplete, Missing P0 functionality, Existing public contracts and security boundaries, Proposed incremental implementation, P0.1 - intake and ledger. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/IMPLEMENTATION_PROGRESS.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/IMPLEMENTATION_GAP_ANALYSIS.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/IMPLEMENTATION_PROGRESS.md

Category: Architecture documentation | Kind: documentation | Lines: 152 | SHA-256: 9999fa946759be3d...

Purpose

This documentation file records the repository's current guidance for Implementation Progress. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Implementation Progress. Its extracted size is 152 lines and 1173 lexical tokens; the source digest begins 9999fa946759.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Base64 Ops next-generation CI recovery progress, Stage A - audit and baseline, P0 next slice - webhook intake and durable incident ledger, Verified-repair contract, Current external blockers, Deployment profile update. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/IMPLEMENTATION_PROGRESS.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/IMPLEMENTATION_STATUS.md

Category: Architecture documentation | Kind: documentation | Lines: 82 | SHA-256: 5f102cebdd7358a9...

Purpose

This documentation file records the repository's current guidance for Implementation Status. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Implementation Status. Its extracted size is 82 lines and 2944 lexical tokens; the source digest begins 5f102cebdd73.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Implementation Status, External requirements, Operational Memory DONE criteria. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/IMPLEMENTATION_STATUS.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/INTERVIEW_TALK_TRACK.md

Category: Architecture documentation | Kind: documentation | Lines: 33 | SHA-256: fa2595fd1ebe60e4...

Purpose

This documentation file records the repository's current guidance for Interview Talk Track. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Interview Talk Track. Its extracted size is 33 lines and 251 lexical tokens; the source digest begins fa2595fd1ebe.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Interview talk track, What problem does Base64 Ops solve?, Why LangGraph?, Why not let an LLM run shell commands?, How does approval binding work?, How are stale CI failures handled?, How does memory avoid becoming truth?. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: README.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/INTERVIEW_TALK_TRACK.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/LIVE_EVALUATION.md

Category: Architecture documentation | Kind: documentation | Lines: 48 | SHA-256: 9c322f8a2dad01c5...

Purpose

This documentation file records the repository's current guidance for Live Evaluation. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Live Evaluation. Its extracted size is 48 lines and 288 lexical tokens; the source digest begins 9c322f8a2dad.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Evaluation modes and external verification, Deterministic evaluation, Manually enabled evaluation mode, What is required for a real external test. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/FEATURE_PARITY_MATRIX.md, docs/IMPLEMENTATION_STATUS.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/LIVE_EVALUATION.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/OBSERVABILITY.md

Category: Architecture documentation | Kind: documentation | Lines: 27 | SHA-256: b9840b8145e35b3b...

Purpose

This documentation file records the repository's current guidance for Observability. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Observability. Its extracted size is 27 lines and 635 lexical tokens; the source digest begins b9840b8145e3.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Observability and replay, GitHub Actions spans. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/IMPLEMENTATION_STATUS.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/OBSERVABILITY.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/OPERATIONAL_MEMORY.md

Category: Architecture documentation | Kind: documentation | Lines: 157 | SHA-256: e2966ff25edfb4cf...

Purpose

This documentation file records the repository's current guidance for Operational Memory. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Operational Memory. Its extracted size is 157 lines and 967 lexical tokens; the source digest begins e2966ff25edf.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Operational Memory, Repository-state freshness and fix outcomes, What Operational Memory Is, What Operational Memory Is NOT, Critical Invariants, Provenance Model, Commit-Aware Staleness. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/IMPLEMENTATION_STATUS.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/OPERATIONAL_MEMORY.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/PATCH_ENGINE.md

Category: Architecture documentation | Kind: documentation | Lines: 141 | SHA-256: bd5a97d556831ce4...

Purpose

This documentation file records the repository's current guidance for Patch Engine. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Patch Engine. Its extracted size is 141 lines and 675 lexical tokens; the source digest begins bd5a97d55683.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Constrained General Patch Engine & Execution Sandbox, Overview, 1. Candidate File Selection, 2. Structured Edit Intents, 3. Path Security Policy, 4. Sensitive File Policy, 5. Patch Size Limits. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/IMPLEMENTATION_STATUS.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/PATCH_ENGINE.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/PRODUCTION_READINESS.md

Category: Architecture documentation | Kind: documentation | Lines: 84 | SHA-256: 3ec06a4e30b64051...

Purpose

This documentation file records the repository's current guidance for Production Readiness. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Production Readiness. Its extracted size is 84 lines and 612 lexical tokens; the source digest begins 3ec06a4e30b6.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Production readiness, Deployment profiles, Non-negotiable production settings, Release acceptance, Latest verification and status. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/IMPLEMENTATION_STATUS.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/PRODUCTION_READINESS.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/PRODUCTION_READINESS_AUDIT.md

Category: Architecture documentation | Kind: documentation | Lines: 115 | SHA-256: af2a82754e7fc2f5...

Purpose

This documentation file records the repository's current guidance for Production Readiness Audit. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Production Readiness Audit. Its extracted size is 115 lines and 2069 lexical tokens; the source digest begins af2a82754e7f.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Production readiness audit, Scope and method, Issue ledger, Verified controls, Current local verification results, Corrected production-profile issue, Exact external gates and local substitutes. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/IMPLEMENTATION_STATUS.md, docs/PRODUCTION_READINESS.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/PRODUCTION_READINESS_AUDIT.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/PRODUCT_STRATEGY.md

Category: Architecture documentation | Kind: documentation | Lines: 48 | SHA-256: 3cf8fde97650a8ac...

Purpose

This documentation file records the repository's current guidance for Product Strategy. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Product Strategy. Its extracted size is 48 lines and 559 lexical tokens; the source digest begins 3cf8fde97650.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Why Base64 Ops exists, Product thesis, Research-to-product implications, Differentiators to build next, 90-day delivery sequence, Guardrails, Research sources. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/PRODUCT_STRATEGY.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/SAAS_SECURITY_AND_SCALE.md

Category: Architecture documentation | Kind: documentation | Lines: 109 | SHA-256: f559e56d780d9b83...

Purpose

This documentation file records the repository's current guidance for Saas Security And Scale. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Saas Security And Scale. Its extracted size is 109 lines and 2491 lexical tokens; the source digest begins f559e56d780d.

Important symbols

Important exported or declared symbols/sections identified by static inspection: SaaS security, cost, and scale plan, What is implemented now, Required production configuration, MVP and scale deployment profiles, Measured growth architecture, Cost and accuracy rules, Security invariants. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: docs/SYSTEM_DESIGN.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/SAAS_SECURITY_AND_SCALE.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/SYSTEM_DESIGN.md

Category: Architecture documentation | Kind: documentation | Lines: 23 | SHA-256: b4fc480d0a02544b...

Purpose

This documentation file records the repository's current guidance for System Design. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for System Design. Its extracted size is 23 lines and 472 lexical tokens; the source digest begins b4fc480d0a02.

Important symbols

Important exported or declared symbols/sections identified by static inspection: System design, Current implementation, Safety and consistency, Failure and scaling considerations, Production SaaS controls. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: README.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/SYSTEM_DESIGN.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

ARCHITECTURE DOCUMENTATION

FILE REFERENCE:

docs/THREAT_MODEL.md

Category: Architecture documentation | Kind: documentation | Lines: 13 | SHA-256: 31552c138bd49a30...

Purpose

This documentation file records the repository's current guidance for Threat Model. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Threat Model. Its extracted size is 13 lines and 189 lexical tokens; the source digest begins 31552c138bd4.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Threat model. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: README.md. The containing category is Architecture documentation.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: docs/THREAT_MODEL.md is a documentation that owns a focused part of architecture documentation. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DEVELOPER AND OPERATIONAL TOOLING

FILE REFERENCE:

load/k6/README.md

Category: Developer and operational tooling | Kind: documentation | Lines: 46 | SHA-256: 5a8aaca01d0d03f8...

Purpose

This documentation file records the repository's current guidance for Readme. It is a maintained explanation surface rather than executable behavior and is cross-checked against implementation where this book makes claims.

Key responsibilities

It exists to provide a coherent documentation boundary for Readme. Its extracted size is 46 lines and 452 lexical tokens; the source digest begins 5a8aaca01d0d.

Important symbols

Important exported or declared symbols/sections identified by static inspection: Safe staging load tests, Prerequisites, Run progressively, 10,000-user evidence gate. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/docs/CONTEXT_ENGINEERING.md, backend/evals/cases/prompt-injection.json, backend/evals/cases/stale-readme.json, backend/tests/test_load_harness_safety.py. The containing category is Developer and operational tooling.

Simplified pseudocode

```
READ documented topic
ORGANIZE implementation-grounded guidance
LINK related source paths and constraints
DISTINGUISH verified behavior from limitations
GUIDE operator or contributor decisions
```

Workflow role

It participates in developer onboarding and operator decision making. Executable pseudocode is not applicable; its document flow records the contract, limitation, or procedure it explains.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: load/k6/README.md is a documentation that owns a focused part of developer and operational tooling. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DEVELOPER AND OPERATIONAL TOOLING

FILE REFERENCE:

load/k6/authenticated-session-read.js

Category: Developer and operational tooling | Kind: configuration | Lines: 48 | SHA-256: 99231a3b2e228b5f...

Purpose

This configuration is part of the developer and operational tooling layer. It supplies the repository behavior or operational contract associated with baseUrl, bearerToken, options.

Key responsibilities

It exists to provide a coherent configuration boundary for Authenticated Session Read. Its extracted size is 48 lines and 194 lexical tokens; the source digest begins 99231a3b2e22.

Important symbols

Important exported or declared symbols/sections identified by static inspection: baseUrl, bearerToken, options, response. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/tests/test_load_harness_safety.py, load/k6/README.md. The containing category is Developer and operational tooling.

Simplified pseudocode

```
READ load/k6/authenticated-session-read.js from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: load/k6/authenticated-session-read.js is a configuration that owns a focused part of developer and operational tooling. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DEVELOPER AND OPERATIONAL TOOLING

FILE REFERENCE:

load/k6/public-health.js

Category: Developer and operational tooling | Kind: configuration | Lines: 41 | SHA-256: 5e34ca26c33a1c74...

Purpose

This configuration is part of the developer and operational tooling layer. It supplies the repository behavior or operational contract associated with baseUrl, options, response.

Key responsibilities

It exists to provide a coherent configuration boundary for Public Health. Its extracted size is 41 lines and 159 lexical tokens; the source digest begins 5e34ca26c33a.

Important symbols

Important exported or declared symbols/sections identified by static inspection: baseUrl, options, response. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/tests/test_load_harness_safety.py, load/k6/README.md. The containing category is Developer and operational tooling.

Simplified pseudocode

```
READ load/k6/public-health.js from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: load/k6/public-health.js is a configuration that owns a focused part of developer and operational tooling. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DEPLOYMENT CONFIGURATION

FILE REFERENCE:

render.free.yaml

Category: Deployment configuration | Kind: YAML configuration | Lines: 103 | SHA-256: fe9fe8e13d295e7a...

Purpose

This YAML configuration is part of the deployment configuration layer. It supplies the repository behavior or operational contract associated with services.

Key responsibilities

It exists to provide a coherent YAML configuration boundary for Render.Free. Its extracted size is 103 lines and 341 lexical tokens; the source digest begins fe9fe8e13d29.

Important symbols

Important exported or declared symbols/sections identified by static inspection: services. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/tests/test_deployment_blueprints.py, backend/tests/test_render_blueprint.py, docs/DEPLOYMENT.md, docs/IMPLEMENTATION_PROGRESS.md. The containing category is Deployment configuration.

Simplified pseudocode

```
READ render.free.yaml from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: render.free.yaml is a YAML configuration that owns a focused part of deployment configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DEPLOYMENT CONFIGURATION

FILE REFERENCE:

render.mvp.yaml

Category: Deployment configuration | Kind: YAML configuration | Lines: 121 | SHA-256: 745c677c50e40aaf...

Purpose

This YAML configuration is part of the deployment configuration layer. It supplies the repository behavior or operational contract associated with services.

Key responsibilities

It exists to provide a coherent YAML configuration boundary for Render.Mvp. Its extracted size is 121 lines and 417 lexical tokens; the source digest begins 745c677c50e4.

Important symbols

Important exported or declared symbols/sections identified by static inspection: services. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: backend/tests/test_deployment_blueprints.py, backend/tests/test_render_blueprint.py, docs/DEPLOYMENT.md, docs/IMPLEMENTATION_PROGRESS.md. The containing category is Deployment configuration.

Simplified pseudocode

```
READ render.mvp.yaml from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: render.mvp.yaml is a YAML configuration that owns a focused part of deployment configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DEPLOYMENT CONFIGURATION

FILE REFERENCE:

render.yaml

Category: Deployment configuration | Kind: YAML configuration | Lines: 242 | SHA-256: 013e7b95e56c3511...

Purpose

This YAML configuration is part of the deployment configuration layer. It supplies the repository behavior or operational contract associated with services.

Key responsibilities

It exists to provide a coherent YAML configuration boundary for Render. Its extracted size is 242 lines and 969 lexical tokens; the source digest begins 013e7b95e56c.

Important symbols

Important exported or declared symbols/sections identified by static inspection: services. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .github/workflows/deployment-readiness.yml, backend/app/agent/graph.py, backend/app/core/access_logging.py, backend/app/services/product_assistant_service.py. The containing category is Deployment configuration.

Simplified pseudocode

```
READ render.yaml from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: render.yaml is a YAML configuration that owns a focused part of deployment configuration. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DEVELOPER AND OPERATIONAL TOOLING

FILE REFERENCE:

scripts/demo.ps1

Category: Developer and operational tooling | Kind: automation script | Lines: 7 | SHA-256: 414c1a47c98b1f19...

Purpose

This automation script is part of the developer and operational tooling layer. It supplies the repository behavior or operational contract associated with its declared configuration and content structure.

Key responsibilities

It exists to provide a coherent automation script boundary for Demo. Its extracted size is 7 lines and 88 lexical tokens; the source digest begins 414c1a47c98b.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, README.md, backend/app/services/product_assistant_service.py, backend/tests/test_github_repository_boundary.py. The containing category is Developer and operational tooling.

Simplified pseudocode

```
READ scripts/demo.ps1 from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: scripts/demo.ps1 is a automation script that owns a focused part of developer and operational tooling. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DEVELOPER AND OPERATIONAL TOOLING

FILE REFERENCE:

scripts/demo.sh

Category: Developer and operational tooling | Kind: automation script | Lines: 10 | SHA-256: 4fbe949ff2e1b36b...

Purpose

This automation script is part of the developer and operational tooling layer. It supplies the repository behavior or operational contract associated with its declared configuration and content structure.

Key responsibilities

It exists to provide a coherent automation script boundary for Demo. Its extracted size is 10 lines and 94 lexical tokens; the source digest begins 4fbe949ff2e1.

Important symbols

Important exported or declared symbols/sections identified by static inspection: no public symbol list was extracted from this file format. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, README.md, backend/app/services/product_assistant_service.py, backend/tests/test_github_repository_boundary.py. The containing category is Developer and operational tooling.

Simplified pseudocode

```
READ scripts/demo.sh from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: scripts/demo.sh is a automation script that owns a focused part of developer and operational tooling. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DEVELOPER AND OPERATIONAL TOOLING

FILE REFERENCE:

scripts/generate_engineering_book.py

Category: Developer and operational tooling | Kind: Python module | Lines: 971 | SHA-256: 3ff79534ae7bbe8c...

Purpose

This Python module is part of the developer and operational tooling layer. It supplies the repository behavior or operational contract associated with ROOT, REPORTS, PDF_PATH.

Key responsibilities

It exists to provide a coherent Python module boundary for Generate Engineering Book. Its extracted size is 971 lines and 10796 lexical tokens; the source digest begins 3ff79534ae7b.

Important symbols

Important exported or declared symbols/sections identified by static inspection: ROOT, REPORTS, PDF_PATH, MARKDOWN_PATH, INVENTORY_PATH, COVERAGE_PATH, GENERATION_REPORT_PATH. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: no simple textual dependent was resolved; framework registration or runtime discovery may still use it. The containing category is Developer and operational tooling.

Simplified pseudocode

```
READ scripts/generate_engineering_book.py from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: scripts/generate_engineering_book.py is a Python module that owns a focused part of developer and operational tooling. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DEVELOPER AND OPERATIONAL TOOLING

FILE REFERENCE:

scripts/verify.ps1

Category: Developer and operational tooling | Kind: automation script | Lines: 19 | SHA-256: 500decb9ad3cb9d5...

Purpose

This automation script is part of the developer and operational tooling layer. It supplies the repository behavior or operational contract associated with AgentGraph.

Key responsibilities

It exists to provide a coherent automation script boundary for Verify. Its extracted size is 19 lines and 104 lexical tokens; the source digest begins 500decb9ad3c.

Important symbols

Important exported or declared symbols/sections identified by static inspection: AgentGraph. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, .github/workflows/ci.yml, README.md. The containing category is Developer and operational tooling.

Simplified pseudocode

```
READ scripts/verify.ps1 from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: scripts/verify.ps1 is a automation script that owns a focused part of developer and operational tooling. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Source file reference

DEVELOPER AND OPERATIONAL TOOLING

FILE REFERENCE:

scripts/verify.sh

Category: Developer and operational tooling | Kind: automation script | Lines: 7 | SHA-256: 078c0b7e21276300...

Purpose

This automation script is part of the developer and operational tooling layer. It supplies the repository behavior or operational contract associated with AgentGraph.

Key responsibilities

It exists to provide a coherent automation script boundary for Verify. Its extracted size is 7 lines and 86 lexical tokens; the source digest begins 078c0b7e2127.

Important symbols

Important exported or declared symbols/sections identified by static inspection: AgentGraph. These names are a navigation aid, not a claim that every symbol is externally public.

Connections

Static in-repository dependencies: no direct in-repository import was statically resolved. Potential consumers found through bounded textual/import analysis: .commandcode/taste/node.js-scaffolding/taste.md, .commandcode/taste/taste.md, .github/workflows/ci.yml, README.md. The containing category is Developer and operational tooling.

Simplified pseudocode

```
READ scripts/verify.sh from its owning layer
ACCEPT typed inputs or declared configuration
APPLY the file's focused responsibility
USE only its declared collaborators and boundaries
RETURN typed data, rendered UI, a test assertion, or an operational contract
SURFACE limitations rather than granting new authority
```

Workflow role

It participates as a focused building block in the larger investigation, delivery, deployment, or verification flow described by its containing subsystem.

Engineering details

The file is part of the operational contract. Its details should be interpreted with the matching runtime/configuration conditions rather than as evidence that an external environment is live.

Brief summary

In one sentence: scripts/verify.sh is a automation script that owns a focused part of developer and operational tooling. Related files: no direct in-repository import was statically resolved. Why it matters: it supplies one explicit, reviewable boundary rather than hiding this responsibility in an unrelated module.

Chapter 24

IMPORTANT DESIGN DECISIONS

Important Design Decisions

The core design choice is evidence first. The system treats repository content and CI output as data, retains provenance, and exposes limitations before generating a conclusion. LangGraph is used to express investigation stages and state transitions rather than to replace deterministic enforcement. Human approval remains central because exact artifacts, branches, and delivery arguments matter more than a broad natural-language intent. Constrained patching reduces the surface of a generated mutation by selecting evidence-related paths and validating source hashes. The frontend/backend split permits an authenticated interactive review surface while preserving policy and token handling on the server. MongoDB supports document-shaped state but requires explicit indexes and reconnect behavior. The existing retrieval and graph components are bounded and advisory rather than a claim of omniscient repository understanding. Docker sandbox use is optional and fail-closed in production. These are design rationales inferred from implementation and documentation, not a statement of original author intent.

Chapter 25

ALTERNATIVE ARCHITECTURES

Alternative Architectures

A monolith with direct GitHub API calls could reduce deployment complexity but would blur interactive, worker, persistence, and privilege boundaries. A fully serverless model could lower idle cost but complicate durable leases, long repository work, and cold-start behavior. A traditional queue such as Redis/RQ or cloud service could improve throughput but add a dependency that this repository intentionally avoids as the sole event ledger. Microservices could isolate indexing, delivery, and model use but would introduce service-to-service authentication and operational overhead. A vector-only retrieval layer may improve semantic matching but loses deterministic lexical fallback and increases dependency sensitivity. A generic coding agent offers flexibility but does not inherently enforce exact-diff approval or failed-SHA separation. GitHub Actions-only automation is simpler for fixed workflows but does not provide cross-run evidence storage, tenant UI, or a guarded investigation workspace. The appropriate architecture depends on scale, compliance requirements, CI provider, and whether the organization prioritizes speed, control, or operational simplicity.

Chapter 26

SCALABILITY AND BOTTLENECKS

Scalability and Bottlenecks

Potential bottlenecks are more clearly identified than measured. GitHub API rate limits and log volume are controlled with repository scoping, cache TTLs, byte caps, excerpt caps, and job caps. LLM calls are bounded by context and retry policy, but provider latency remains external. Repository indexing is limited by file, byte, and chunk budgets and can move to workers, but large repositories may produce partial corpus coverage. MongoDB is the durable coordination point, so connection pools, indexes, and availability are significant. Background job throughput depends on worker availability and lease settings. Free Render can cold start and may not sustain always-on workers. Concurrent investigations are intentionally tenant-limited to contain cost. A future benchmark should measure per-stage duration, queue lag, Mongo queries, GitHub request count, retained bytes, context tokens, validation duration, and terminal outcomes under authorized load. No throughput number is claimed here.

Chapter 27

FAILURE MODES AND RECOVERY

Failure Modes and Recovery

Mongo startup failure degrades the service while a bounded reconnect loop retries and re-establishes correctness-critical indexes. Readiness remains false until durable state and required rate limiting are available. GitHub failures, permission denials, and unavailable logs become investigation limitations rather than hidden provider detail. OAuth state protects callbacks from replay and incorrect user association. Model failures have bounded retries and explicit failures. A missing parent revision, unavailable workflow file, or changed current state prevents causal overclaim and may block patch generation. Invalid patches fail path, hash, candidate, validation, risk, or approval checks. Approval conflicts and stale plans are invalidated instead of silently reusing an old approval. Worker lease/retry rules protect durable job recovery. Browser network/auth errors surface a recovery state. These mechanisms reduce common failure impacts but do not replace live incident response, provider observability, backups, or a production SLO.

Chapter 28

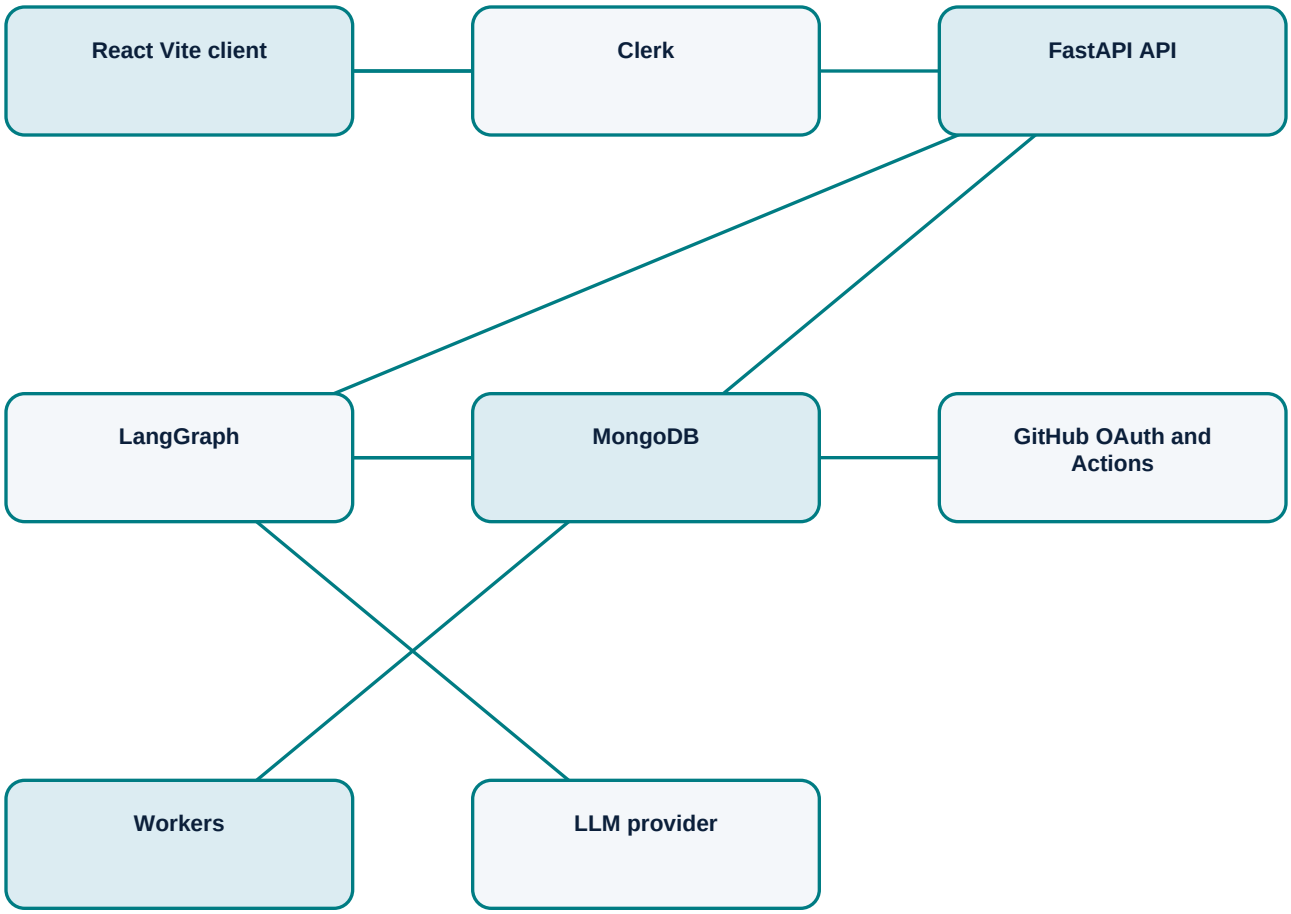
FUTURE IMPROVEMENTS

Future Improvements

The highest priority unfinished work is a real, opt-in candidate verification lifecycle: a GitHub App installation-token boundary, a reviewed repository-owned workflow, immutable VerificationRun records, and an honest result state. The next causality improvements are runner/environment and dependency-lockfile comparisons against prior success, still labeled as correlation. Historical intelligence needs promotion rules only after verified evidence thresholds, plus advisory PR warnings rather than merge blocks. Flaky intelligence needs richer test-level artifacts and controlled recurrence statistics; it must never infer a passing test from a workflow-level success. GitHub-native check/comment reporting requires explicit least-privilege permissions and notification preferences. Live integration testing should use disposable allowed repositories and no arbitrary writes. Product work should include accessible browser E2E coverage, real user workflow measurements, and reproducible screenshots. Each roadmap item should be accepted only with a measurable safety or usefulness criterion rather than a demo-only claim.

Diagram 1

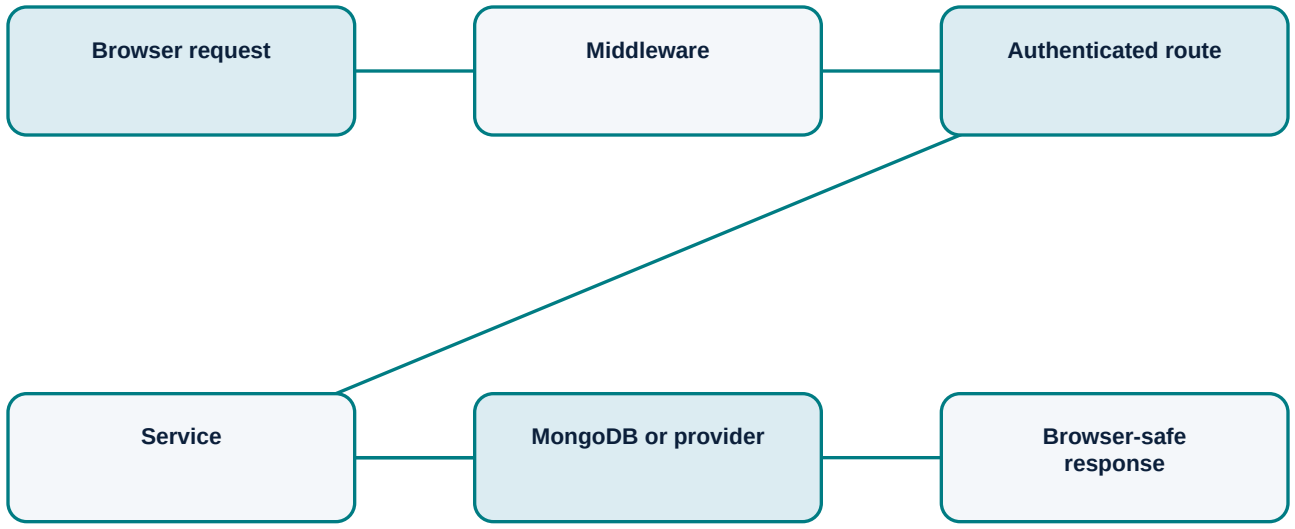
OVERALL SYSTEM ARCHITECTURE



This diagram is a conceptual representation of relationships verified in the repository. It does not imply a live deployment topology, provider permission, or runtime execution beyond the source and local test evidence documented in this edition.

Diagram 2

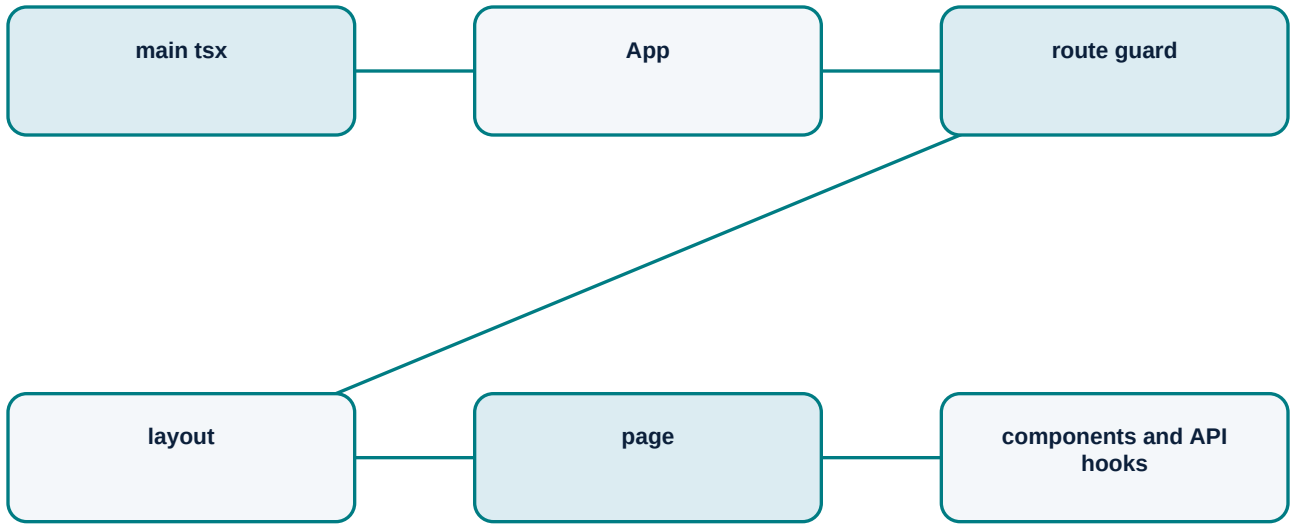
BACKEND REQUEST LIFECYCLE



This diagram is a conceptual representation of relationships verified in the repository. It does not imply a live deployment topology, provider permission, or runtime execution beyond the source and local test evidence documented in this edition.

Diagram 3

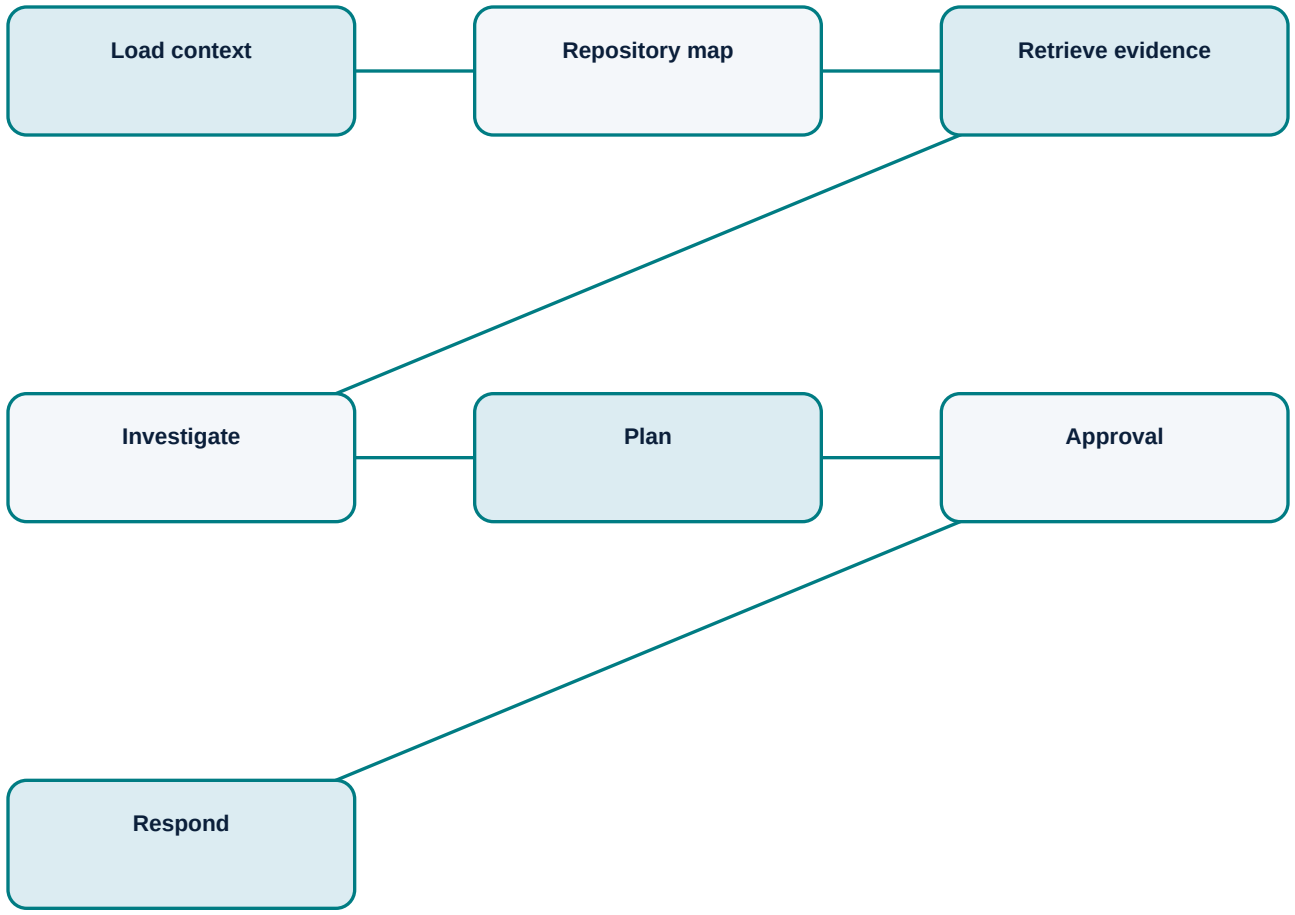
FRONTEND COMPONENT HIERARCHY



This diagram is a conceptual representation of relationships verified in the repository. It does not imply a live deployment topology, provider permission, or runtime execution beyond the source and local test evidence documented in this edition.

Diagram 4

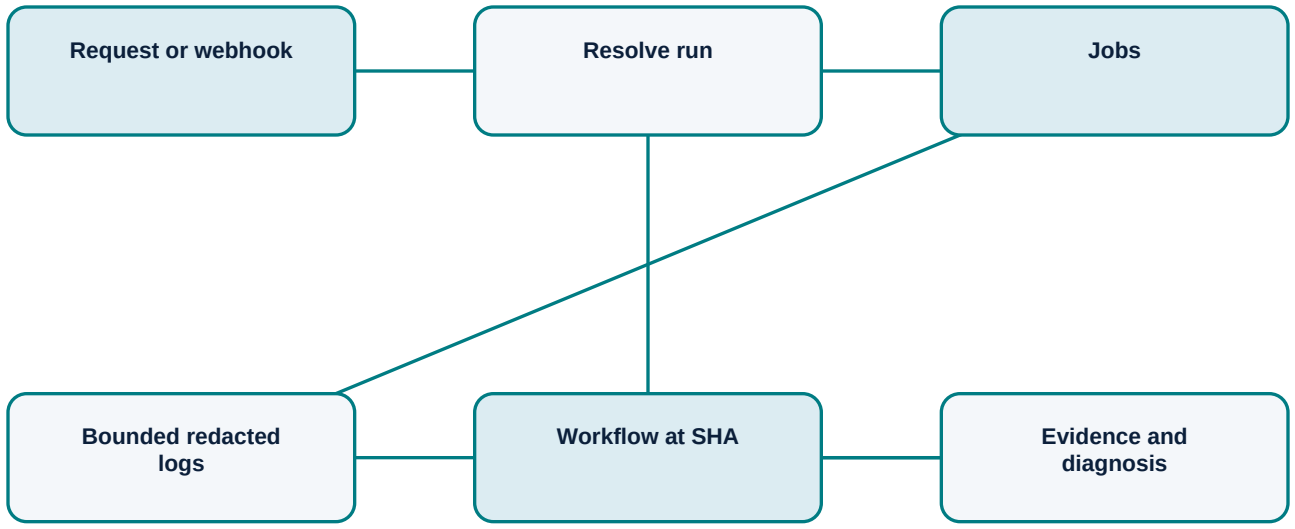
LANGGRAPH INVESTIGATION



This diagram is a conceptual representation of relationships verified in the repository. It does not imply a live deployment topology, provider permission, or runtime execution beyond the source and local test evidence documented in this edition.

Diagram 5

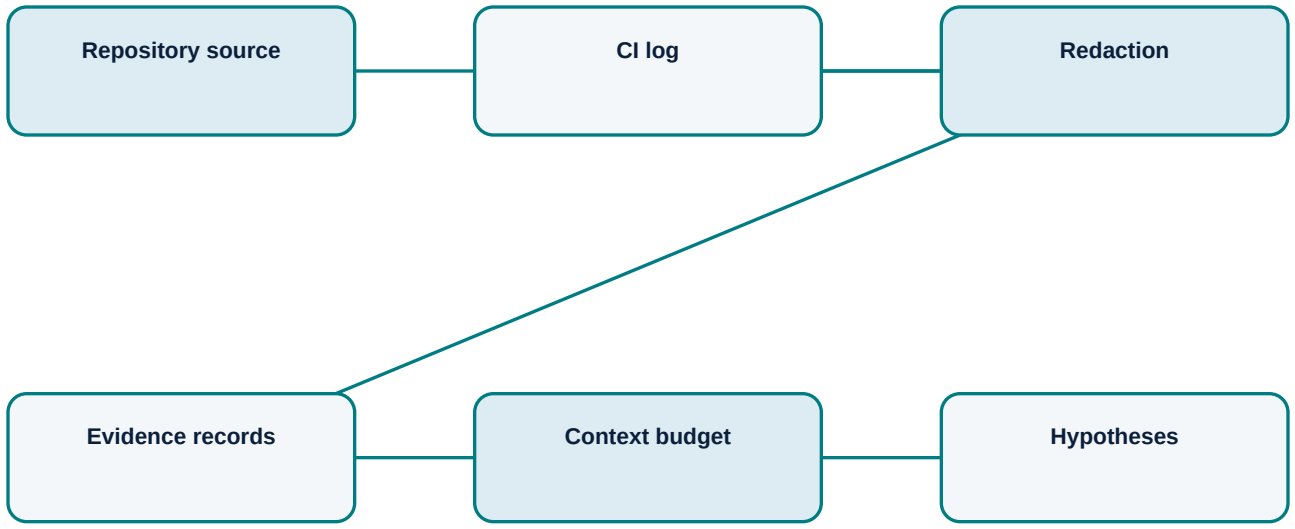
GITHUB ACTIONS INVESTIGATION



This diagram is a conceptual representation of relationships verified in the repository. It does not imply a live deployment topology, provider permission, or runtime execution beyond the source and local test evidence documented in this edition.

Diagram 6

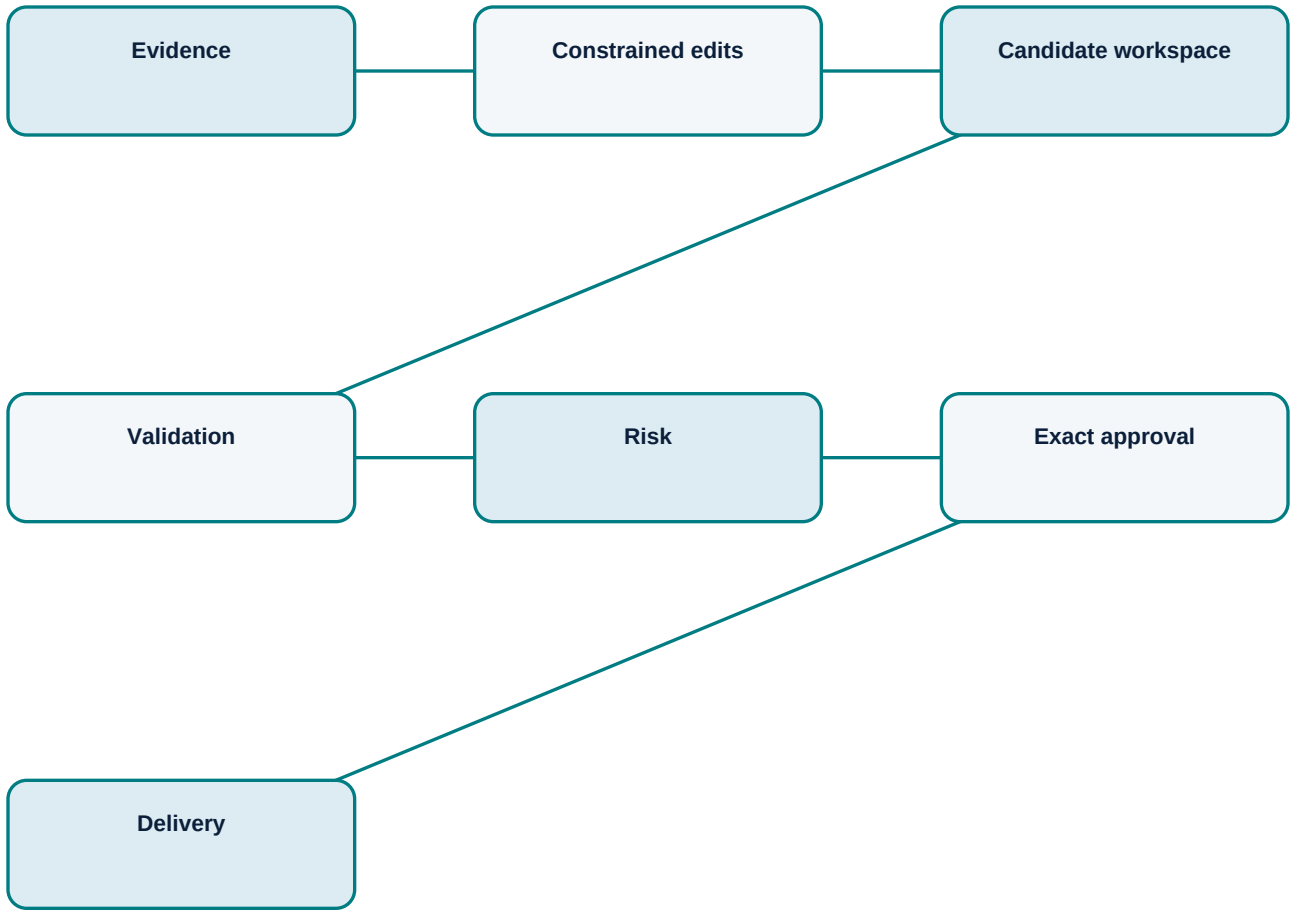
EVIDENCE RETRIEVAL



This diagram is a conceptual representation of relationships verified in the repository. It does not imply a live deployment topology, provider permission, or runtime execution beyond the source and local test evidence documented in this edition.

Diagram 7

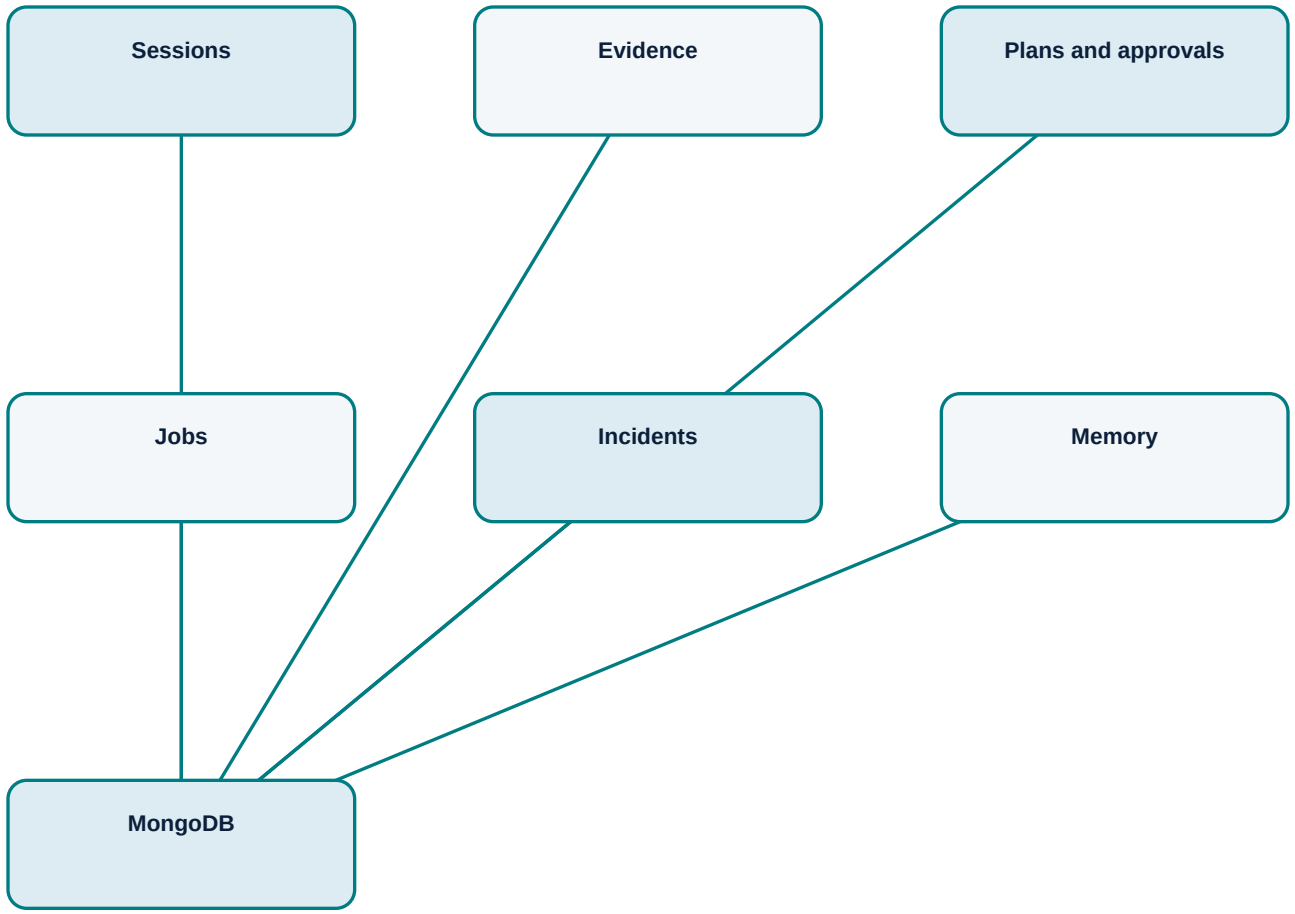
PATCH GENERATION AND APPROVAL



This diagram is a conceptual representation of relationships verified in the repository. It does not imply a live deployment topology, provider permission, or runtime execution beyond the source and local test evidence documented in this edition.

Diagram 8

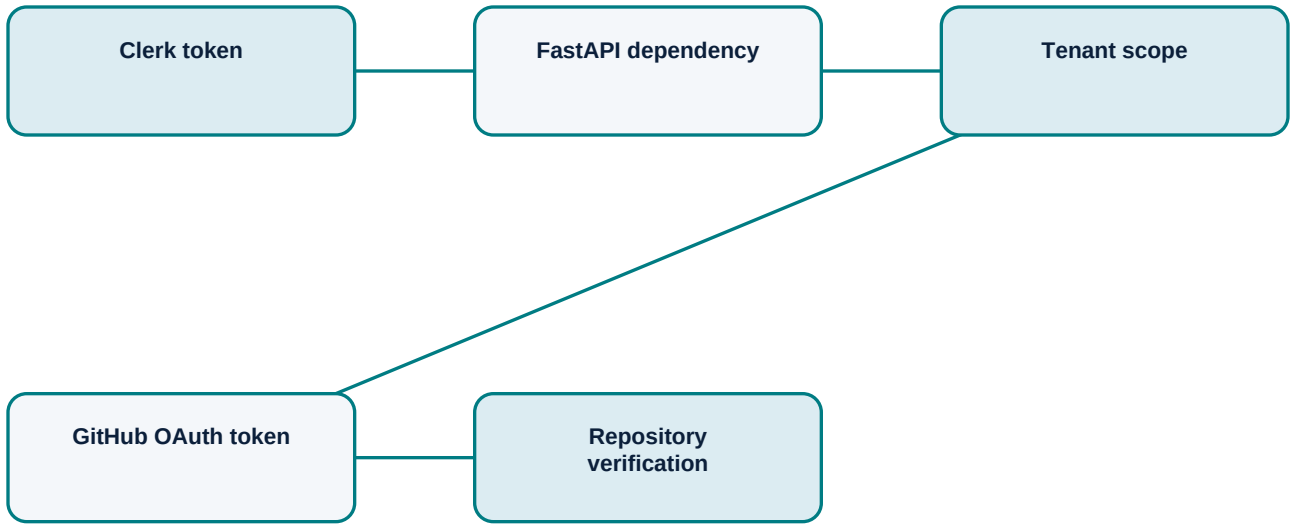
DATA PERSISTENCE



This diagram is a conceptual representation of relationships verified in the repository. It does not imply a live deployment topology, provider permission, or runtime execution beyond the source and local test evidence documented in this edition.

Diagram 9

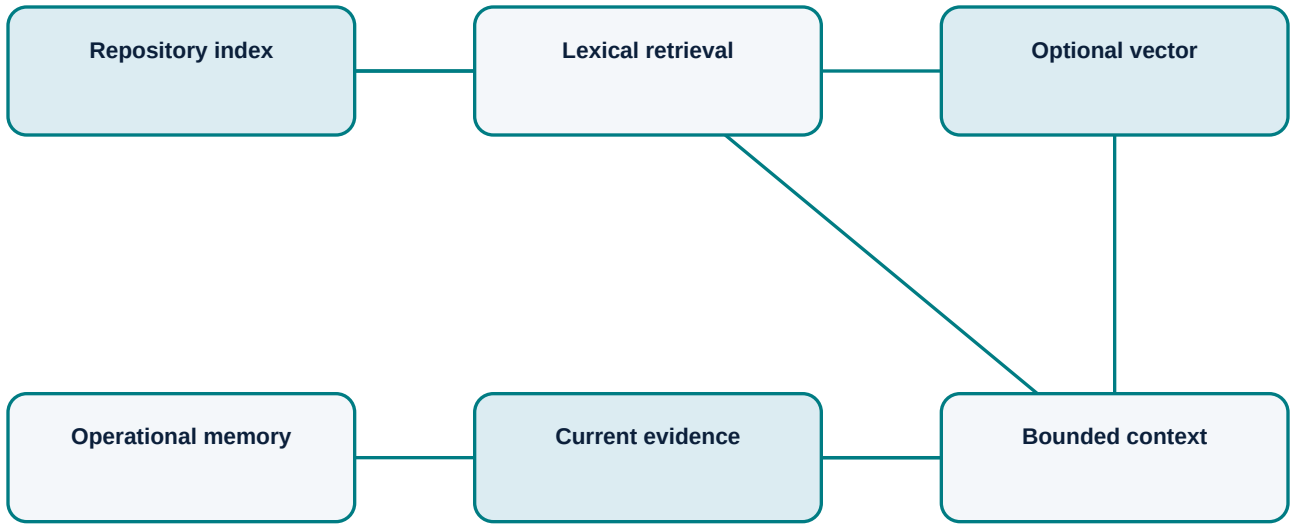
AUTHENTICATION AND AUTHORIZATION



This diagram is a conceptual representation of relationships verified in the repository. It does not imply a live deployment topology, provider permission, or runtime execution beyond the source and local test evidence documented in this edition.

Diagram 10

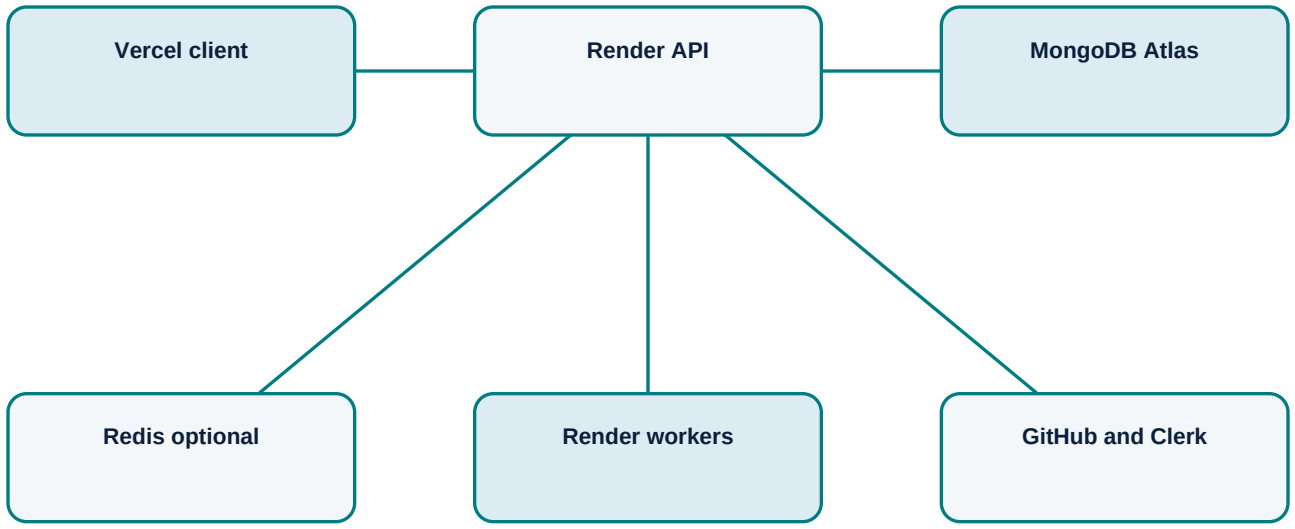
MEMORY AND RETRIEVAL



This diagram is a conceptual representation of relationships verified in the repository. It does not imply a live deployment topology, provider permission, or runtime execution beyond the source and local test evidence documented in this edition.

Diagram 11

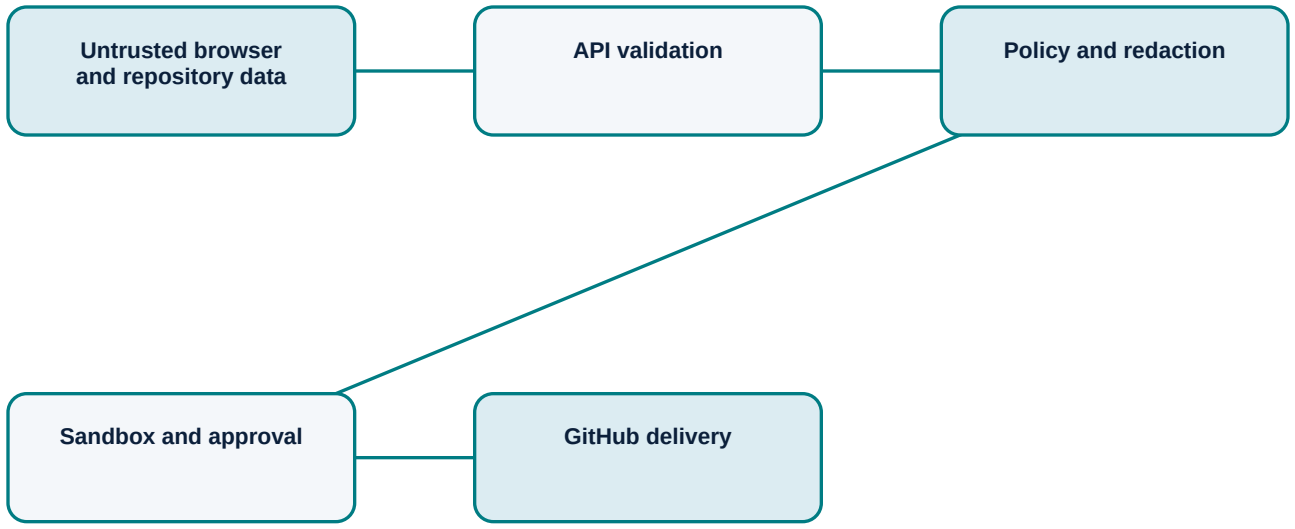
DEPLOYMENT TOPOLOGY



This diagram is a conceptual representation of relationships verified in the repository. It does not imply a live deployment topology, provider permission, or runtime execution beyond the source and local test evidence documented in this edition.

Diagram 12

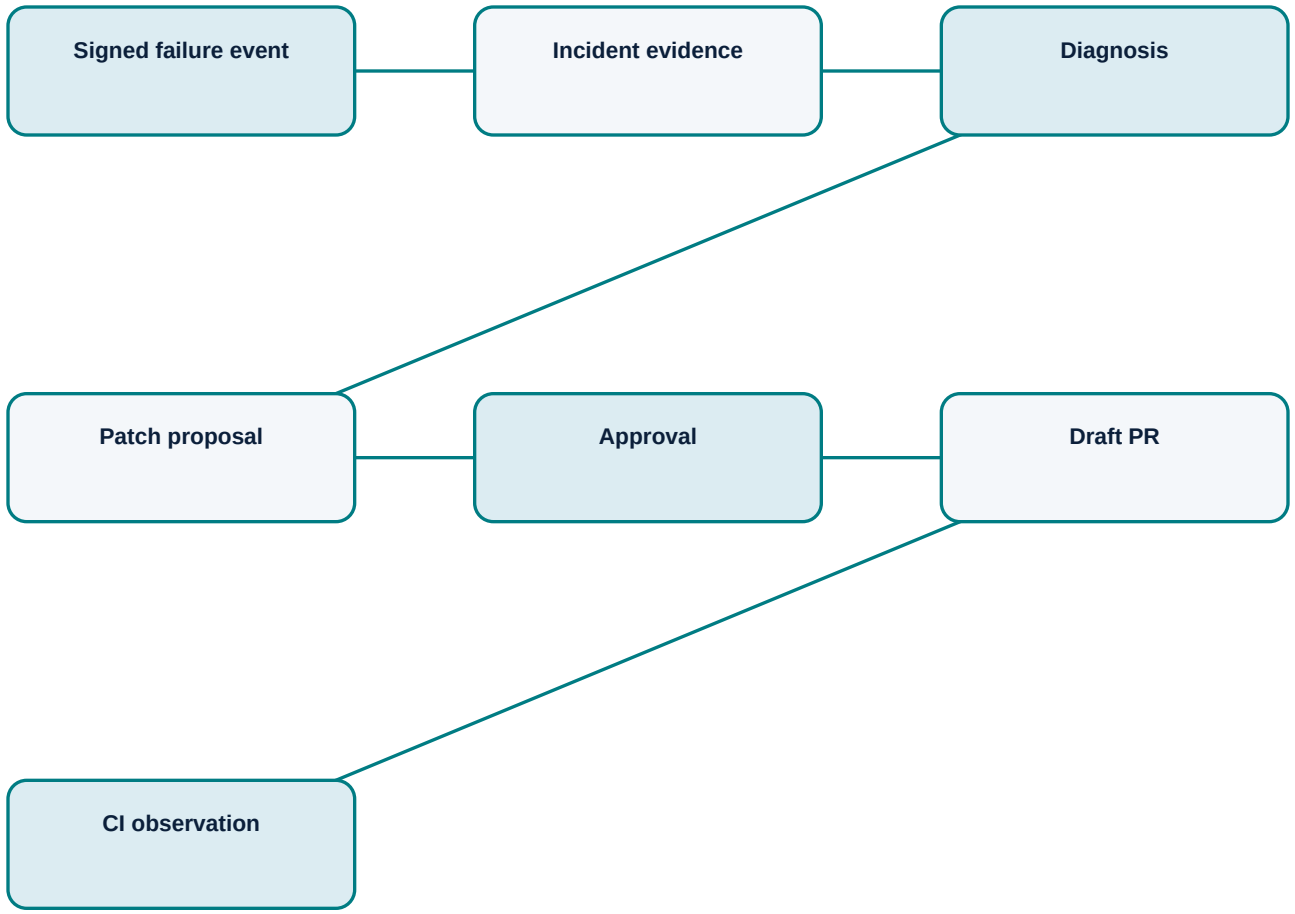
TRUST BOUNDARIES



This diagram is a conceptual representation of relationships verified in the repository. It does not imply a live deployment topology, provider permission, or runtime execution beyond the source and local test evidence documented in this edition.

Diagram 13

INCIDENT TO DELIVERY WORKFLOW



This diagram is a conceptual representation of relationships verified in the repository. It does not imply a live deployment topology, provider permission, or runtime execution beyond the source and local test evidence documented in this edition.

Glossary

REFERENCE

Evidence
A persisted redacted record carrying source and revision provenance.

DeliveryPlan
The exact proposal bound to source state, diff, validation, risk, and delivery arguments.

Failed run SHA
The Git commit associated with an Actions run, never silently treated as current HEAD.

Applicability
A comparison conclusion about whether a historical condition remains present.

Candidate workspace
An isolated workspace used to derive and validate a constrained patch.

Operational memory
Evidence-gated historical context that is lower precedence than current evidence and cannot authorize mutation.

Untrusted data
Repository/log/workflow/model content that cannot alter policy, credentials, or command selection.

References and verification boundaries

REFERENCE

External references

GitHub Actions documentation: https://docs.github.com/en/actions (accessed 2026-10-14).
GitHub workflow concepts: https://docs.github.com/en/actions/concepts/workflows-and-actions (accessed 2026-10-14).
Clerk React ClerkProvider reference: https://clerk.com/docs/reference/clerk-react/clerkprovider (accessed 2026-10-14).
Sentry Seer: https://docs.sentry.io/product/ai-in-sentry/seer (accessed 2026-10-14).
Primary repository references: README.md and the tracked documents in docs/ and backend/docs/ at the analyzed commit.

Verification boundary

External product descriptions are cited as official documentation snapshots, not a claim of complete competitive parity. Live GitHub, Atlas, Clerk, OpenAI, Redis, Render, Vercel, and GitHub App behavior is not inferred from local tests. The book names such items as unverified whenever it discusses them.